Aspirra — Student Success & Career Planner

Project Milestone 1 (PM1) — Software Design & DevOps

Course: CSE454 — Advanced Software Engineering
Institution: Alamein International University
Faculty: Computer Science & Engineering
Repository: https://github.com/AIU-SoftWave/Aspirra
Jira project: https://aspirra.atlassian.net (project `SCRUM`, board 1)
Submission date: 2026-10-28

Team

Member	GitHub	Owned services
Ahmed Mohamed Ahmed (Team Leader)	`AbooSalh`	Student, Academic, Course, Assessment
Yomna Mohamed Fekry	`YomnaElmenawy`	Goal Planner, Study Planner, Scheduling, Adaptation
Arwa Mohey Hamdy	`ArwaMohey`	Career, Skill, Portfolio, Job Preparation
Mostafa Khamis Abozead	`Mostafa-abozead`	Habit, Analytics, Prediction, AI Coach
Muhammed Hany Hamdy	`muhammedhanii`	Dashboard, Notification, Study Group, Administration

Tools used

Jira · GitHub · Docker & Docker Compose · GitHub Container Registry (GHCR) · GitHub Actions

---

Contents

1. Executive Summary
2. Part A — Design
    - A.1 System Context
    - A.2 Interfaces
    - A.3 Container Diagram
    - A.4 Component Diagrams
    - A.5 Class Diagrams
    - A.6 ERDs
3. Part B — DevOps
4. Part C — Project Management
5. Part D — Traceability
6. Build and Verification
7. Team Ownership
8. Appendix A — Full Interfaces Specification

---

1. Executive Summary

Aspirra is an AI-powered student success and career planner. A student states a goal such as “reach a 3.5 GPA” or “land a software internship”; Aspirra turns it into an editable plan of milestones, tasks, habits, deadlines, and career preparation steps. The student reviews and approves the plan before it becomes active. When progress slips, Aspirra recommends changes and waits for approval before applying them.

The system is designed as a **modular monolith** with 20 logical service boundaries. Each service has one provided interface, documented required interfaces, its own database schema, and a named owner. Sprint 1 implements six services end to end — Student, Academic, Course, Assessment, Goal Planner, and Habit — with 74 automated tests. The other 14 services are fully designed and scheduled for later sprints.

PM1 area	Result
Part A — Design	System Context, Interfaces, Container, 20 Component diagrams, system + 20 Class diagrams, 20 ERDs
Part B — DevOps	20 multi-stage Dockerfiles + API Gateway, single `docker compose up`, GHCR images, passing GitHub Actions
Part C — Project Management	Sprint 1: 16 stories, 46 planned SP, 46 completed SP, 0% estimation error
Part D — Traceability	Every Sprint 1 story mapped to code, diagrams, migrations, Docker, and Jira

Technology stack:

Area	Technology
Frontend (planned)	React / Next.js + TypeScript
Backend	ASP.NET Core Web API, C# / .NET 10
Data access	EF Core 10 + Npgsql
Database	PostgreSQL 16, one schema per service
Containers	Multi-stage Dockerfiles, Docker Compose
Registry	GitHub Container Registry (GHCR)
CI	GitHub Actions
Project management	Jira Scrum (`SCRUM`, board 1)

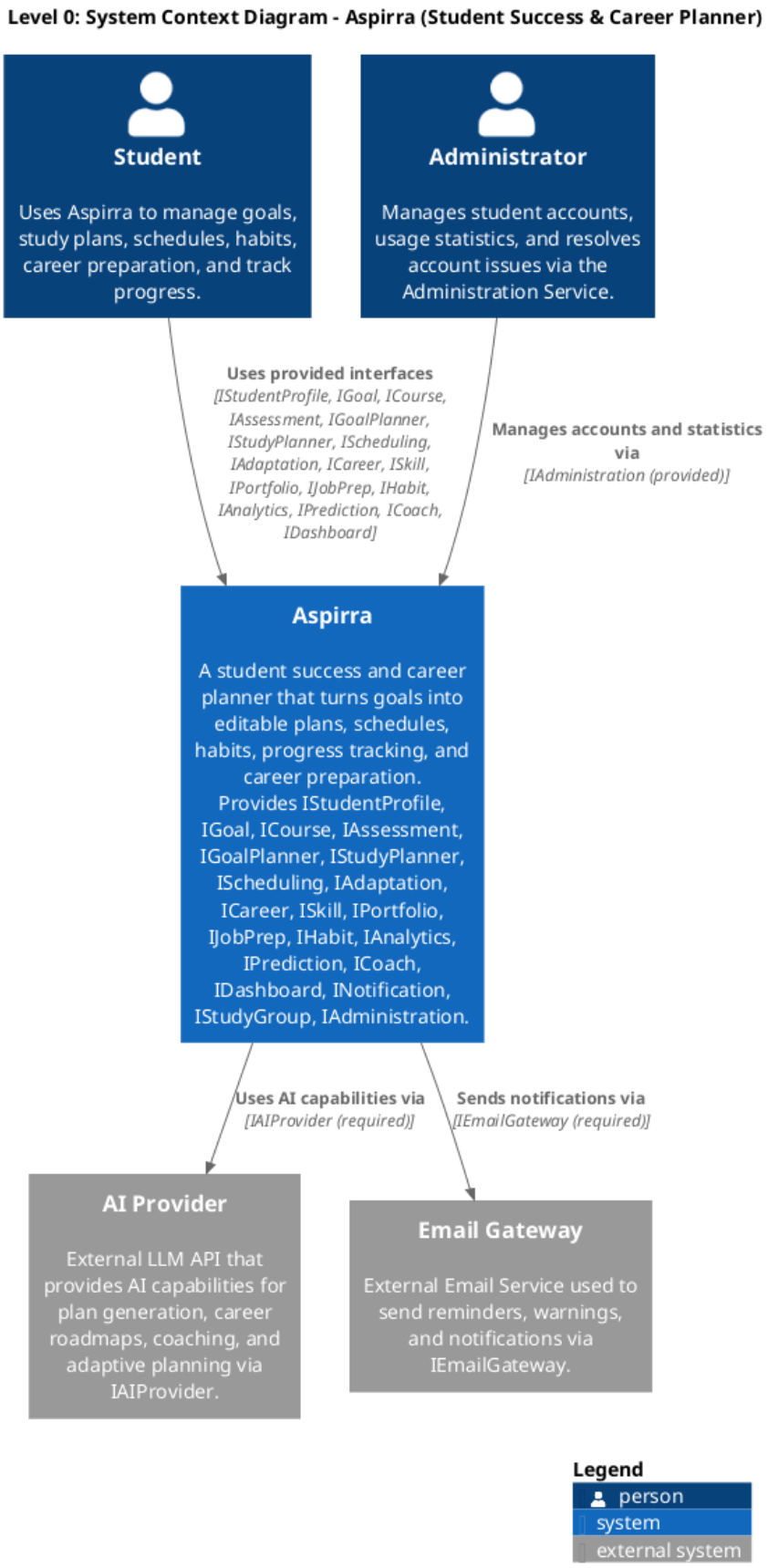
---

2. Part A — Design

A.1 System Context (Level 0)

The System Context diagram shows:

- The **Student** actor, who manages goals, plans, schedules, habits, career preparation, and progress.
- The **Administrator** actor, who manages student accounts, usage statistics, and account issues through the Administration Service.
- The **Aspirra** system box listing all 20 provided interfaces: `IStudentProfile`, `IGoal`, `ICourse`, `IAssessment`, `IGoalPlanner`, `IStudyPlanner`, `IScheduling`, `IAdaptation`, `ICareer`, `ISkill`, `IPortfolio`, `IJobPrep`, `IHabit`, `IAnalytics`, `IPrediction`, `ICoach`, `IDashboard`, `INotification`, `IStudyGroup`, and `IAdministration`.
- The external **AI Provider**, connected through the required interface `IAIProvider`.
- The external **Email Gateway**, connected through the required interface `IEmailGateway`.



Source: `System Context diagram.puml`

A.2 Interfaces (Level 0)

The interface specification documents every provided and required interface. For each interface it gives the high-level functional requirement and the low-level functional requirements grouped under it. Every

required-interface operation is annotated with the low-level FR that uses it.

Provided interfaces (20):

Interface	Service	Owner	High-level FR
`IStudentProfile`	Student Service	Ahmed Mohamed Ahmed	FR-01 Manage Student Profile
`IGoal`	Academic Service	Ahmed Mohamed Ahmed	FR-02 Manage Academic Goals
`ICourse`	Course Service	Ahmed Mohamed Ahmed	FR-03 Manage Courses
`IAssessment`	Assessment Service	Ahmed Mohamed Ahmed	FR-04 Manage Assignments and Exams
`IGoalPlanner`	Goal Planner Service	Yomna Mohamed Fekry	FR-05 Plan Goals with AI
`IStudyPlanner`	Study Planner Service	Yomna Mohamed Fekry	FR-06 Generate Study Plans
`IScheduling`	Scheduling Service	Yomna Mohamed Fekry	FR-07 Enable Smart Scheduling
`IAdaptation`	Adaptation Service	Yomna Mohamed Fekry	FR-08 Enable Adaptive Planning
`ICareer`	Career Service	Arwa Mohey Hamdy	FR-09 Manage Career Roadmaps
`ISkill`	Skill Service	Arwa Mohey Hamdy	FR-10 Track Skills
`IPortfolio`	Portfolio Service	Arwa Mohey Hamdy	FR-11 Manage Projects and Portfolios
`IJobPrep`	Job Preparation Service	Arwa Mohey Hamdy	FR-12 Manage Internship and Job Preparation
`IHabit`	Habit Service	Mostafa Khamis Abozead	FR-13 Manage Habits
`IAnalytics`	Analytics Service	Mostafa Khamis Abozead	FR-14 Analyze Study Workload
`IPrediction`	Prediction Service	Mostafa Khamis Abozead	FR-15 Predict and Warn Early
`ICoach`	AI Coach Service	Mostafa Khamis Abozead	FR-16 Coach Students with AI
`IDashboard`	Dashboard Service	Muhammed Hany Hamdy	FR-17 Display Student Dashboard
`INotification`	Notification Service	Muhammed Hany Hamdy	FR-18 Send Notifications
`IStudyGroup`	Study Group Service	Muhammed Hany	FR-19 Manage Study Groups and

		Hamdy	Accountability
`IAdministration`	Administration Service	Muhammed Hany Hamdy	FR-20 Manage Administration

Required interfaces (14):

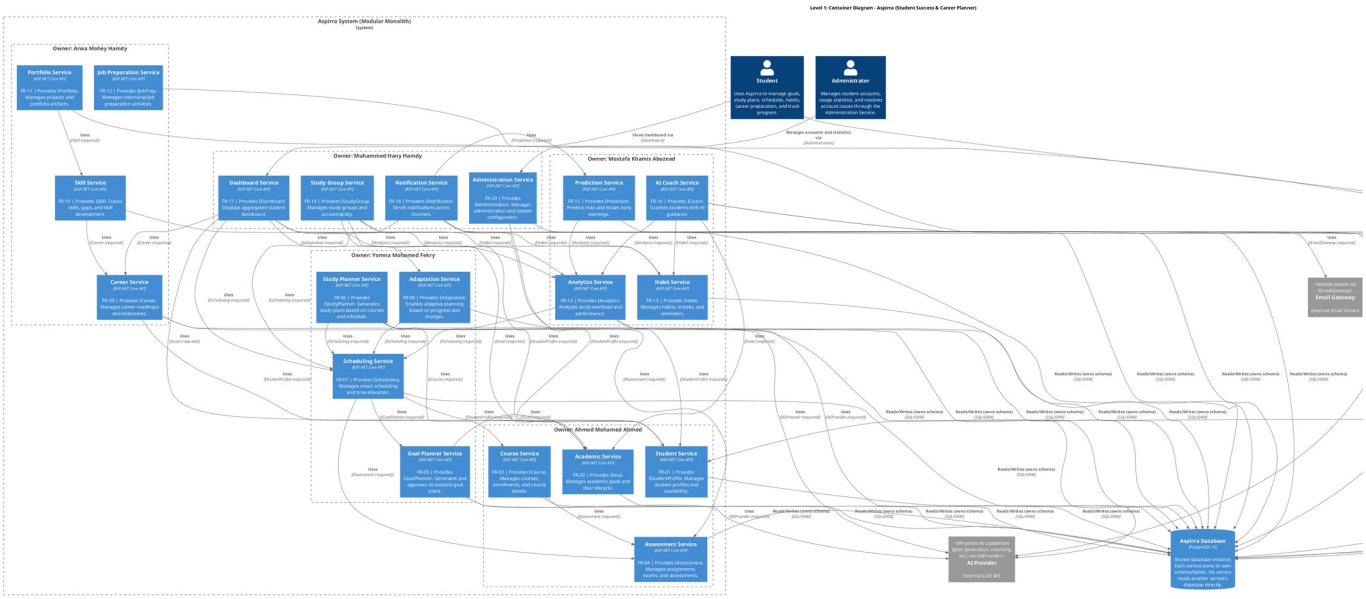
Interface	Consumed by	Supports
`IAIProvider`	Goal Planner, Study Planner, Career, AI Coach	FR-05, FR-06, FR-09, FR-16
`IEmailGateway`	Notification	FR-18
`IStudentProfile`	Scheduling, Career, Analytics, Study Group, Administration	FR-07, FR-09, FR-14, FR-19, FR-20
`IGoal`	Goal Planner, AI Coach, Dashboard, Study Group	FR-05, FR-16, FR-17, FR-19
`ICourse`	Study Planner	FR-06
`IAssessment`	Course, Scheduling, Notification	FR-03, FR-07, FR-18
`IGoalPlanner`	Scheduling	FR-07
`IScheduling`	Study Planner, Adaptation, Analytics, Dashboard, Notification	FR-06, FR-08, FR-14, FR-17, FR-18
`IAdaptation`	Dashboard	FR-17
`ICareer`	Skill, Dashboard	FR-10, FR-17
`ISkill`	Portfolio	FR-11
`IHabit`	AI Coach, Dashboard, Notification	FR-16, FR-17, FR-18
`IAnalytics`	Prediction, AI Coach, Dashboard, Study Group	FR-15, FR-16, FR-17, FR-19
`IPrediction`	Notification	FR-18

The complete interface specification, including every low-level operation and `(used by: FR-XX.Y)` annotation, is reproduced in **Appendix A**. It is also available at `Interfaces.md`.

A.3 Container Diagram (Level 1)

The container diagram shows one container per service, grouped by owner, connected through their provided and required interfaces. It also shows:

- The Student and Administrator actors.
- The external AI Provider and Email Gateway.
- The shared PostgreSQL 16 database instance, where each service owns its own schema and tables. No service reads another service's schema directly.
- The cross-service dependencies: Course Service reads assessment data through `IAssessment`; Goal Planner Service reads goal existence through `IGoal`; Study Planner, Scheduling, Adaptation, Career, Skill, Portfolio, Analytics, Prediction, AI Coach, Dashboard, Notification, Study Group, and Administration dependencies are documented in the interfaces specification.



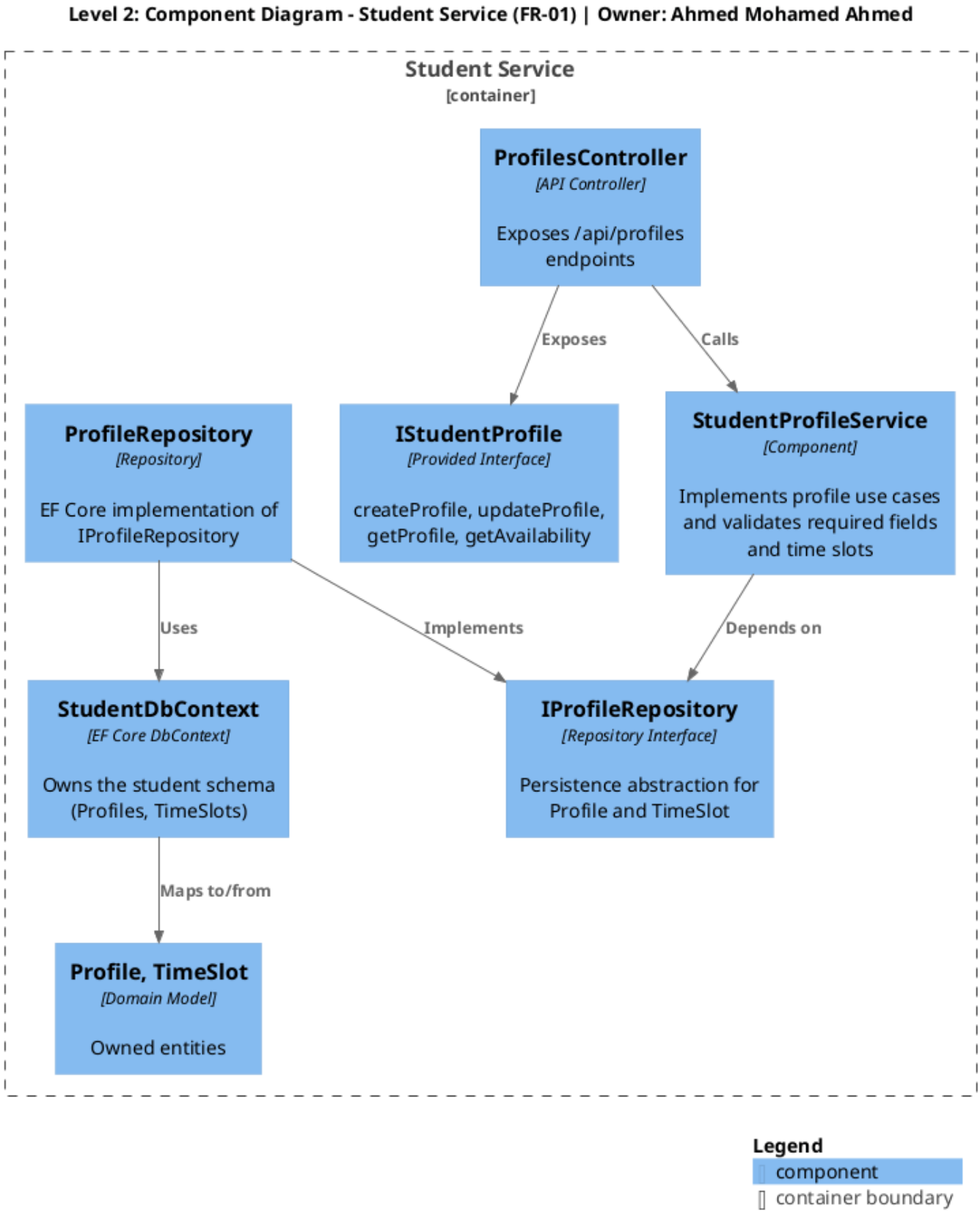
Source: `Aspirra-Container-Diagram.puml`

A.4 Component Diagrams (Level 2)

Each service has a component diagram prepared by its owner. The six implemented services are aligned with their real controllers, services, repositories, adapters, DbContexts, and domain models. The remaining 14 diagrams describe the planned internal structure.

01. Student Service — FR-01

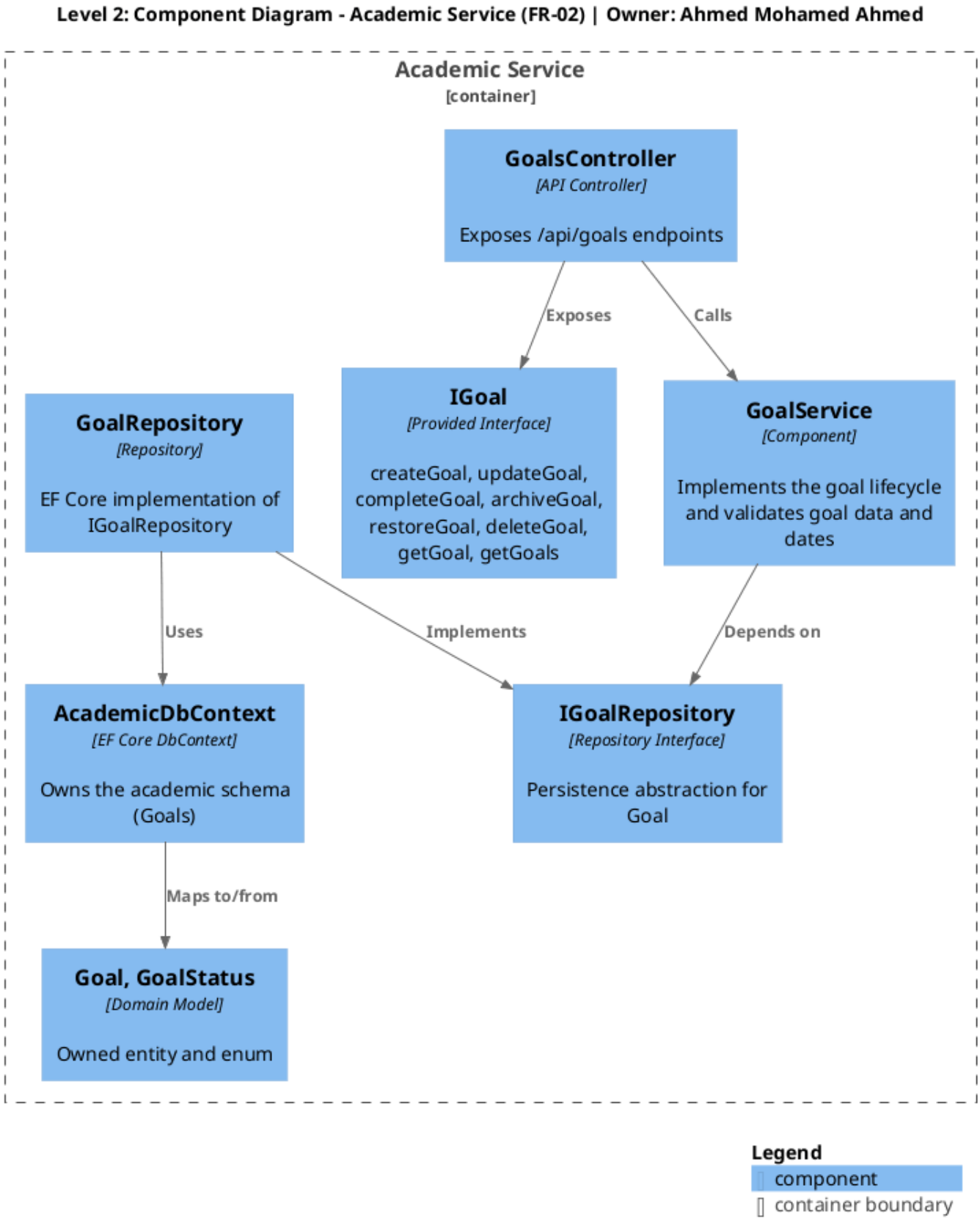
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `01-student-service-component.puml`

02. Academic Service — FR-02

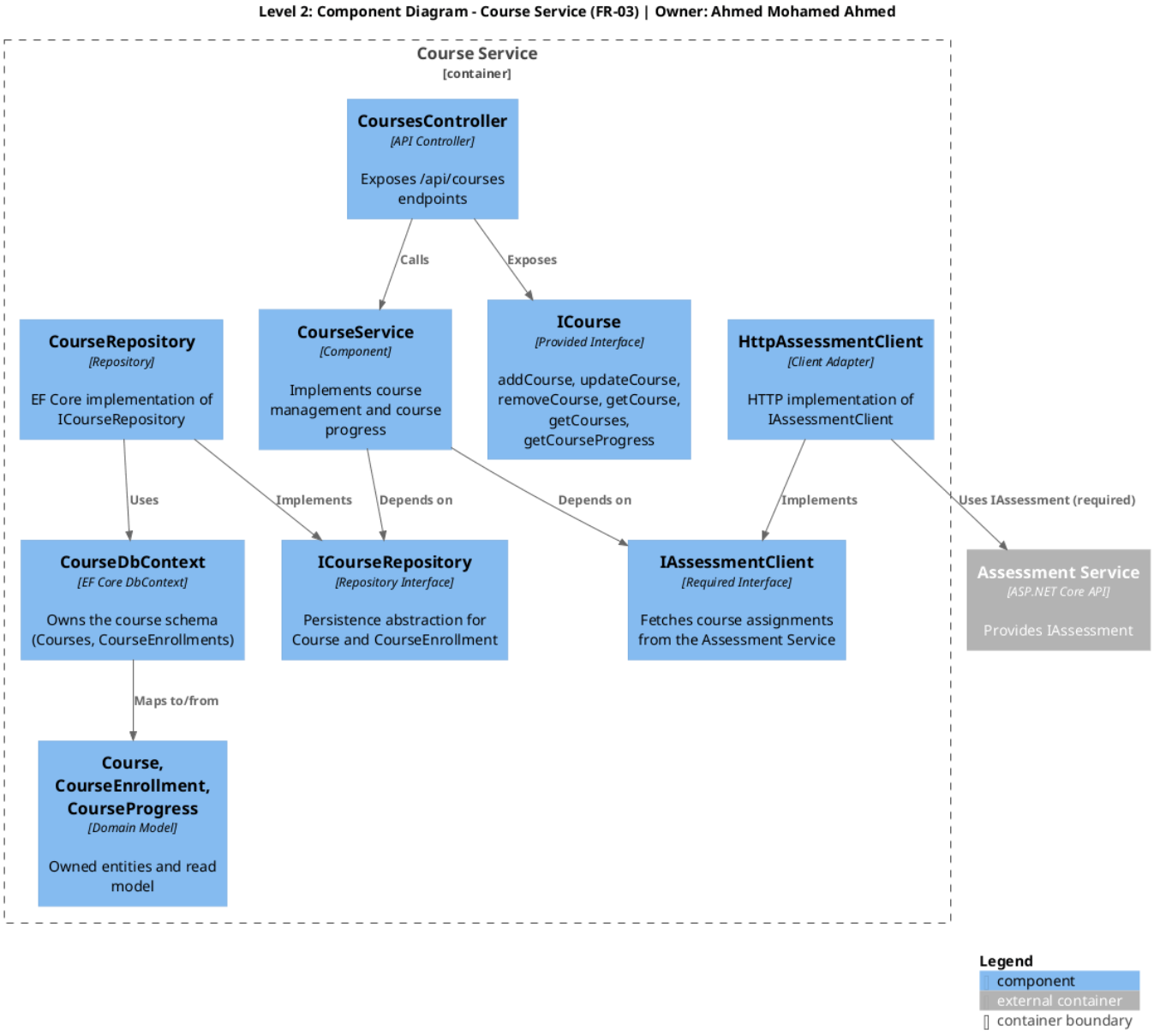
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `02-academic-service-component.puml`

03. Course Service — FR-03

Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1

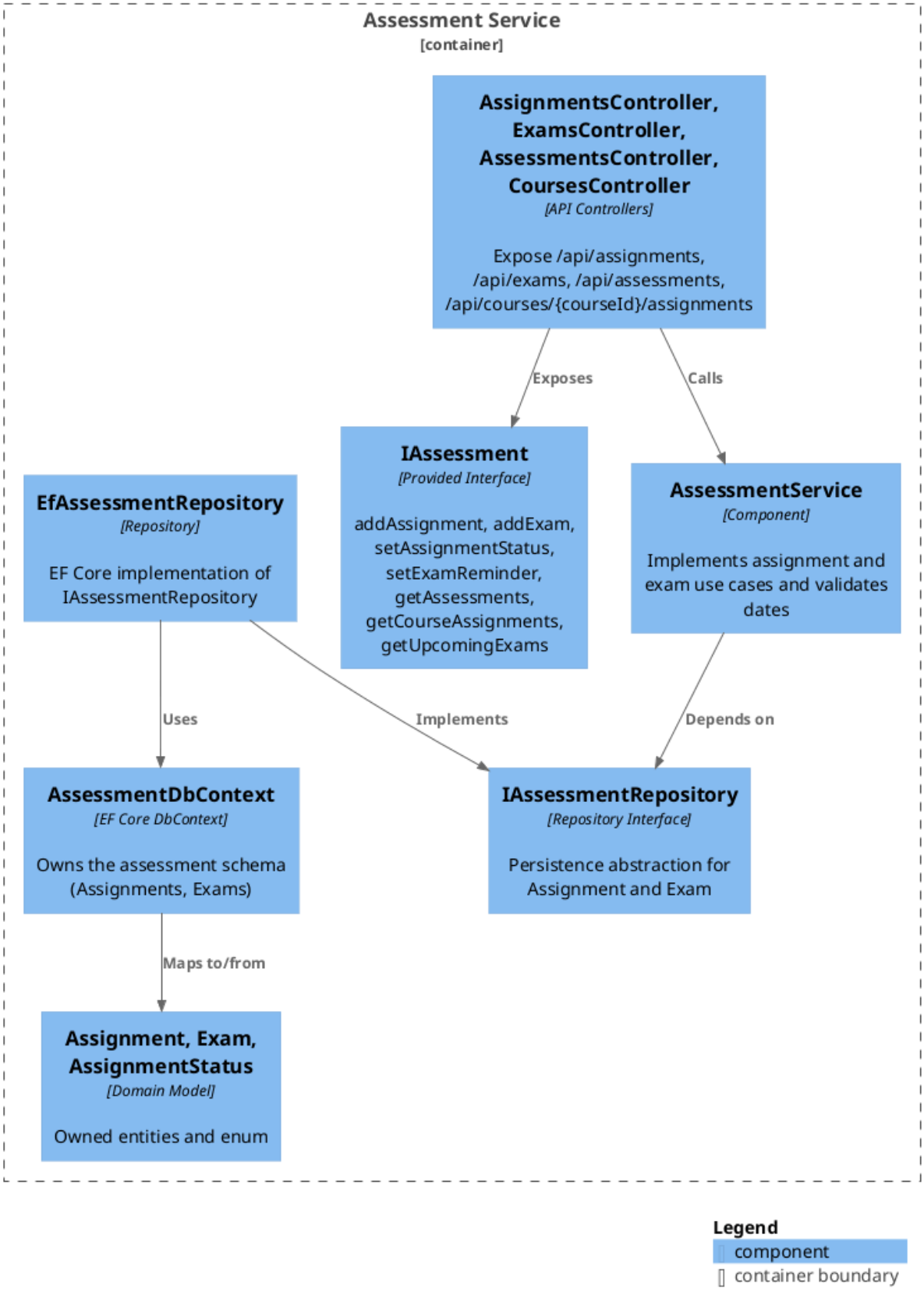


Source: `03-course-service-component.puml`

04. Assessment Service — FR-04

Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1

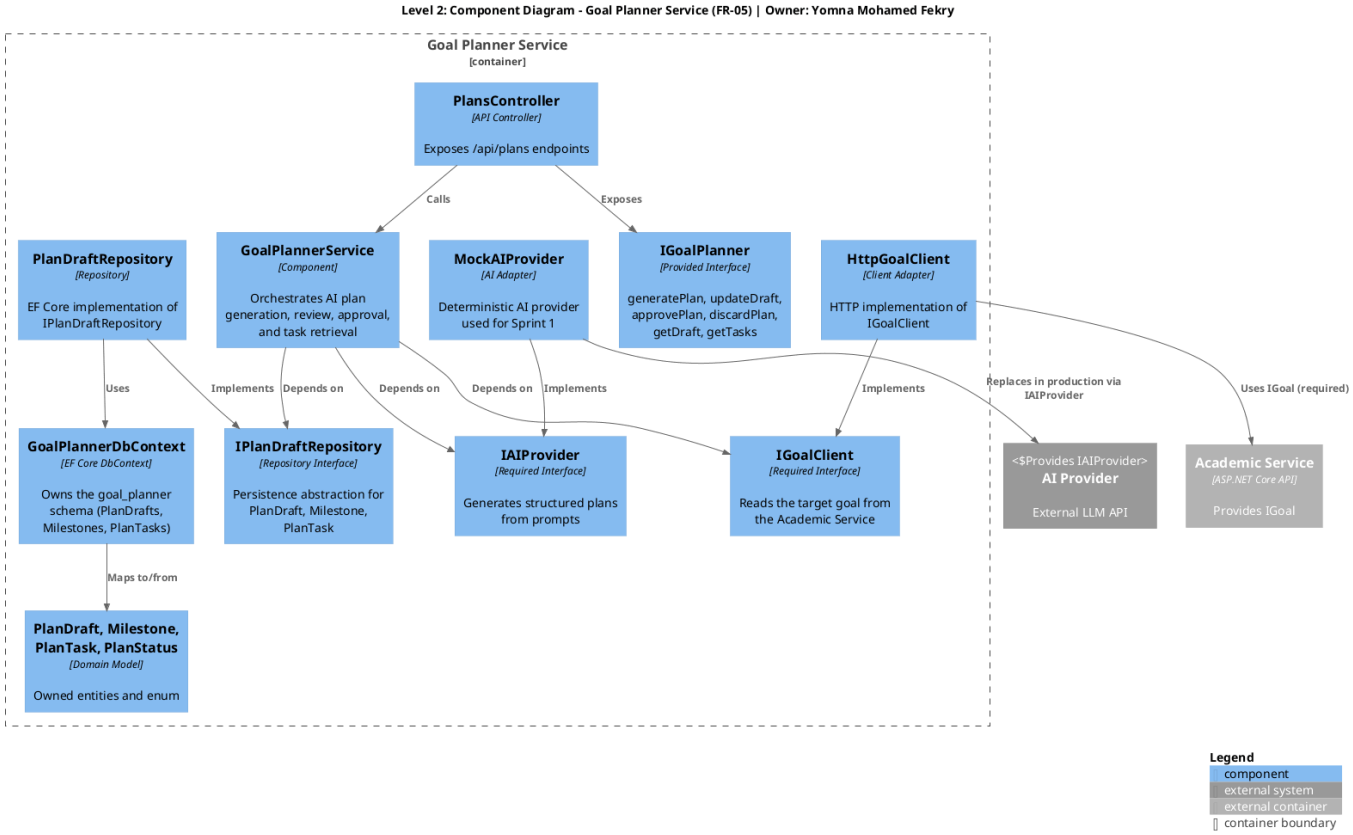
Level 2: Component Diagram - Assessment Service (FR-04) | Owner: Ahmed Mohamed Ahmed



Source: `04-assessment-service-component.puml`

05. Goal Planner Service — FR-05

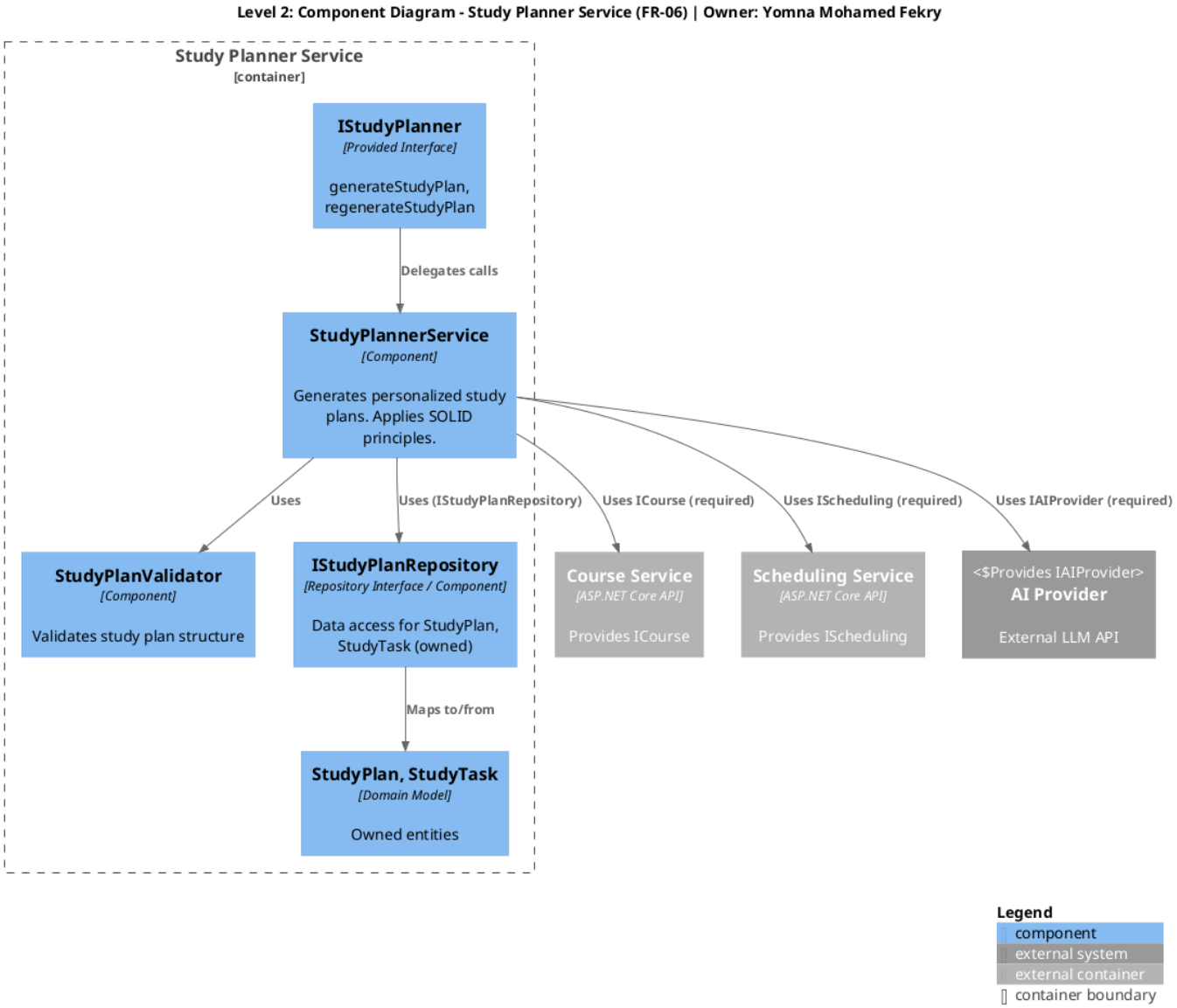
Owner: Yomna Mohamed Fekry · **Status:** Implemented in Sprint 1



Source: `05-goal-planner-service-component.puml`

06. Study Planner Service — FR-06

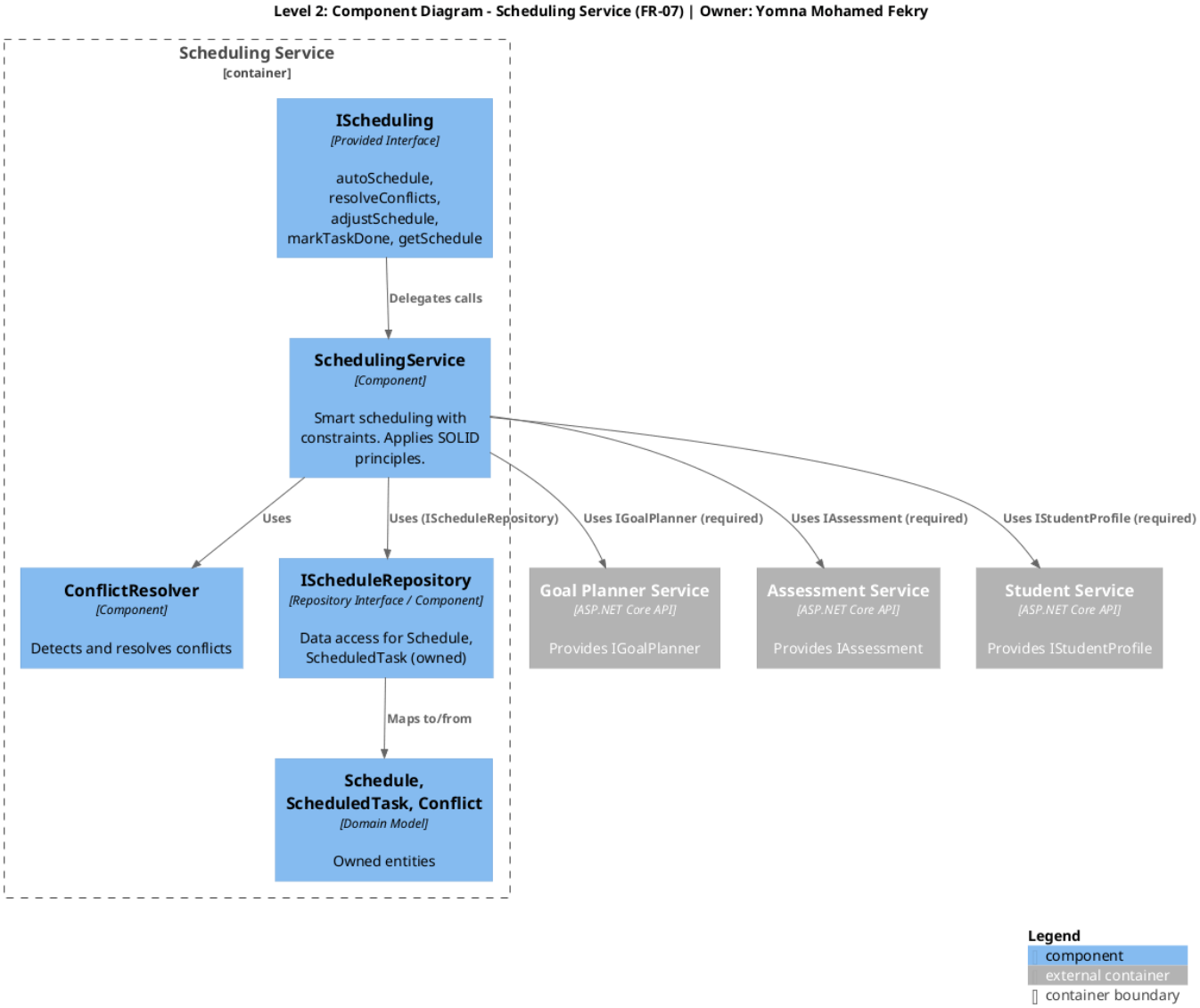
Owner: Yomna Mohamed Fekry · **Status:** Design complete; implementation scheduled for a later sprint



Source: `06-study-planner-service-component.puml`

07. Scheduling Service — FR-07

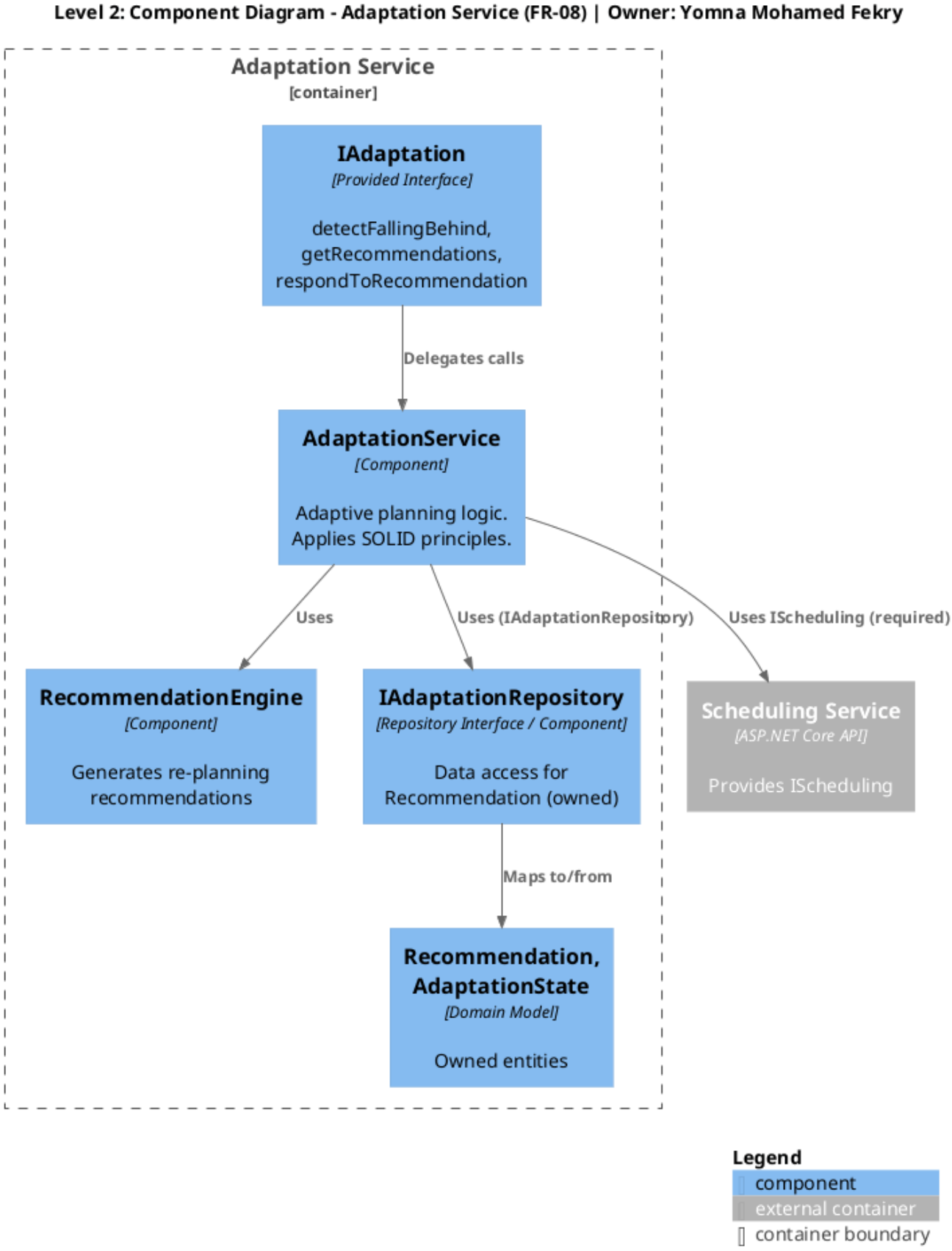
Owner: Yomna Mohamed Fekry · **Status:** Design complete; implementation scheduled for a later sprint



Source: `07-scheduling-service-component.puml`

08. Adaptation Service — FR-08

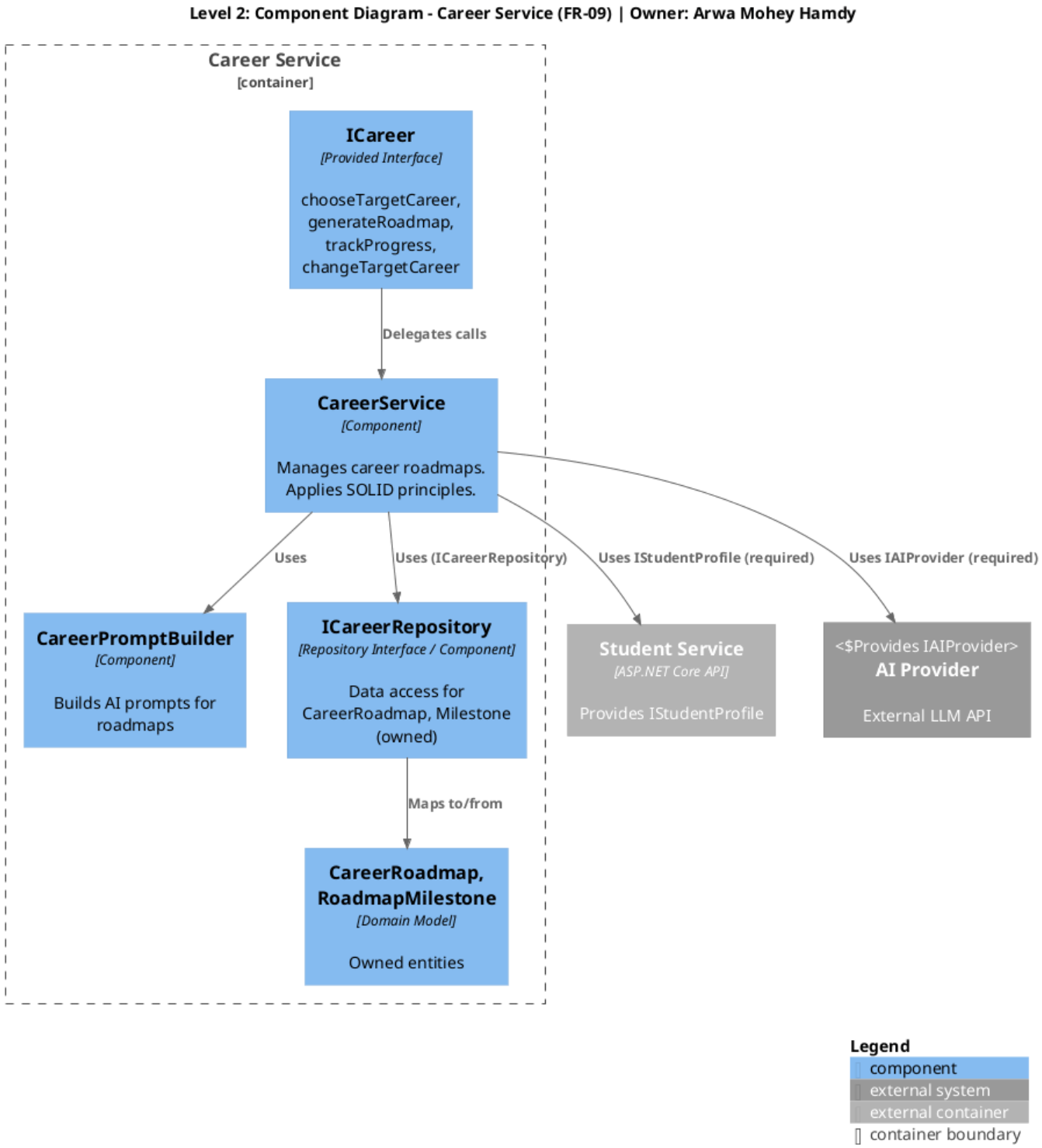
Owner: Yomna Mohamed Fekry · **Status:** Design complete; implementation scheduled for a later sprint



Source: `08-adaptation-service-component.puml`

09. Career Service — FR-09

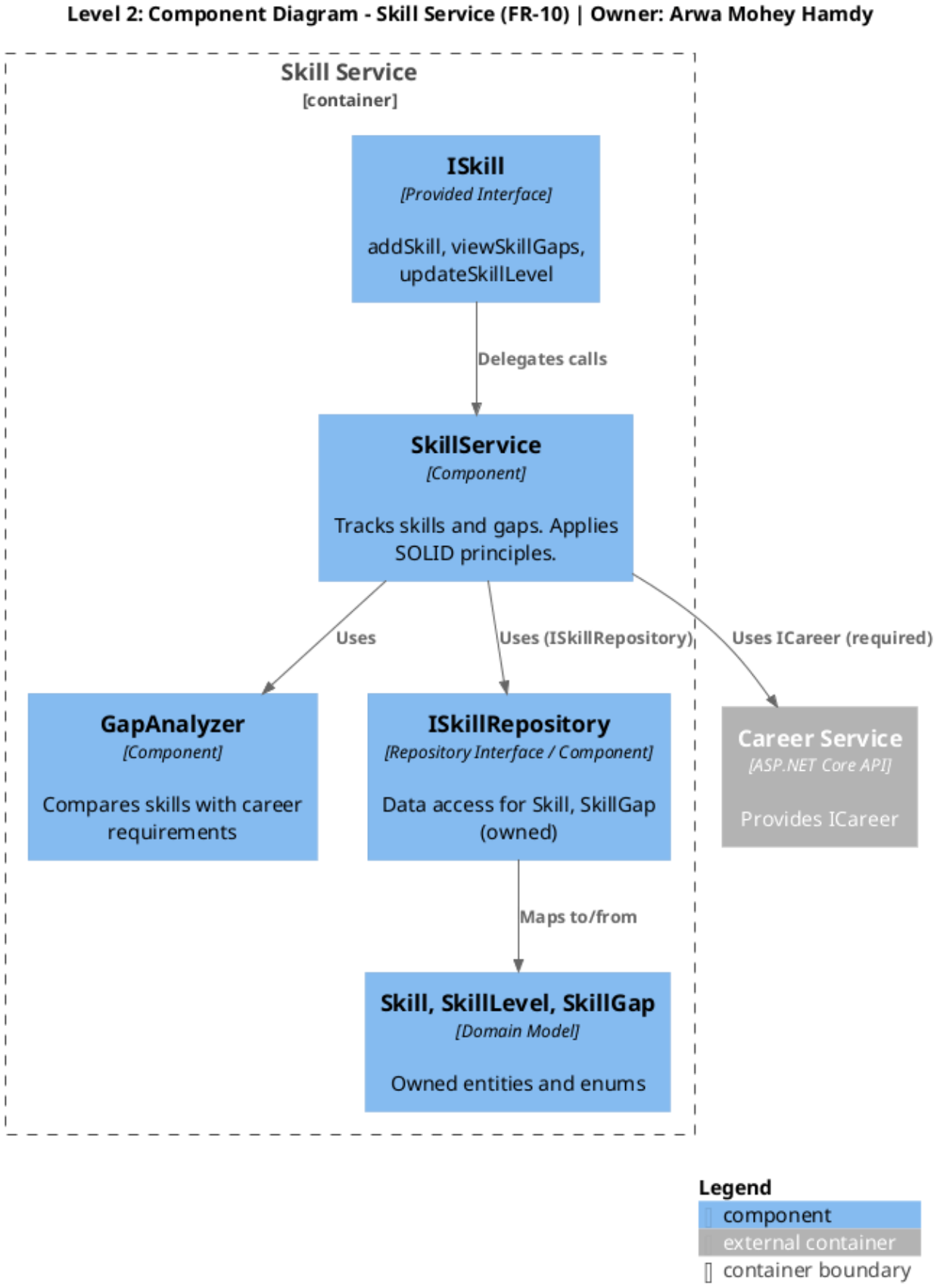
Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `09-career-service-component.puml`

10. Skill Service — FR-10

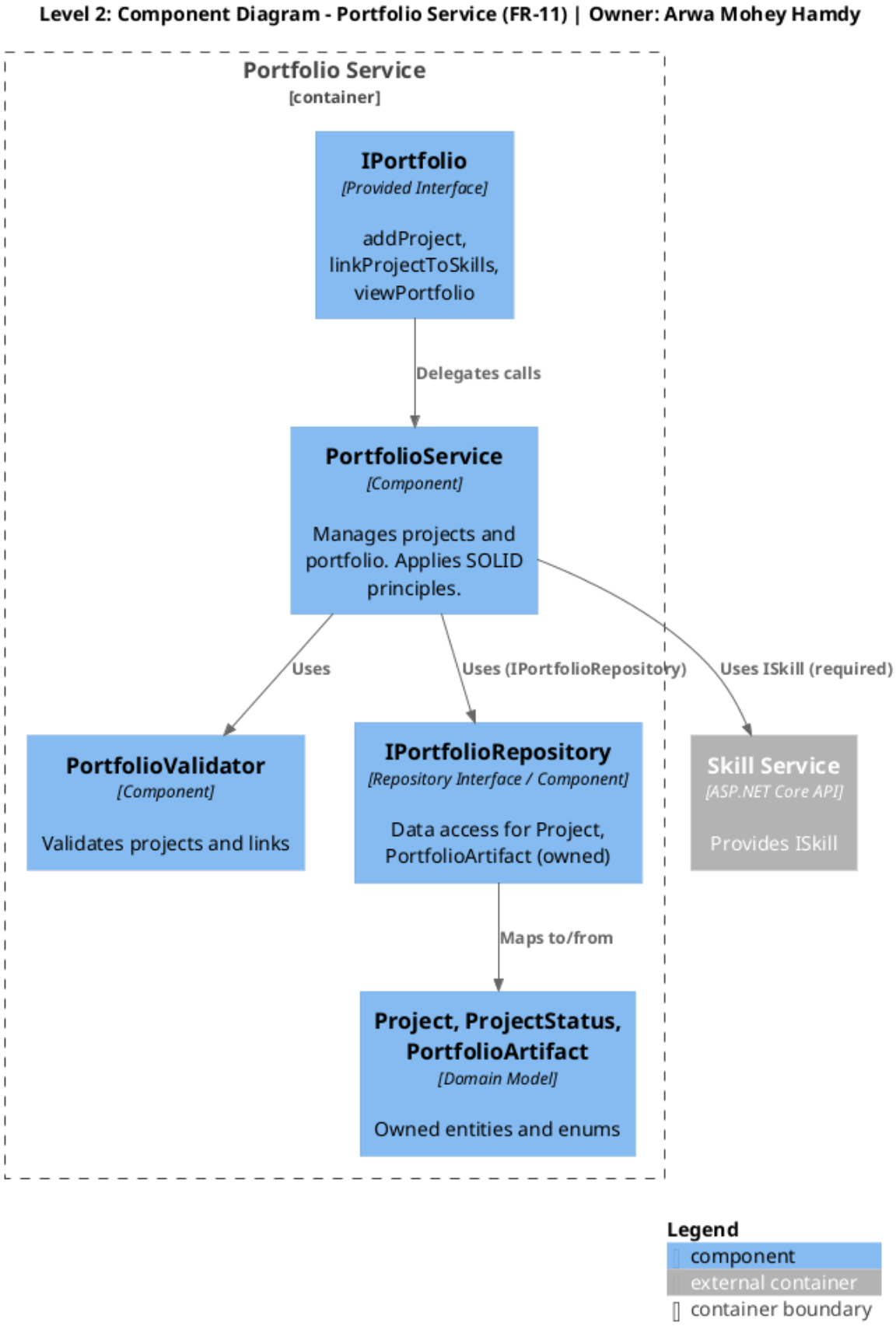
Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `10-skill-service-component.puml`

11. Portfolio Service — FR-11

Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint

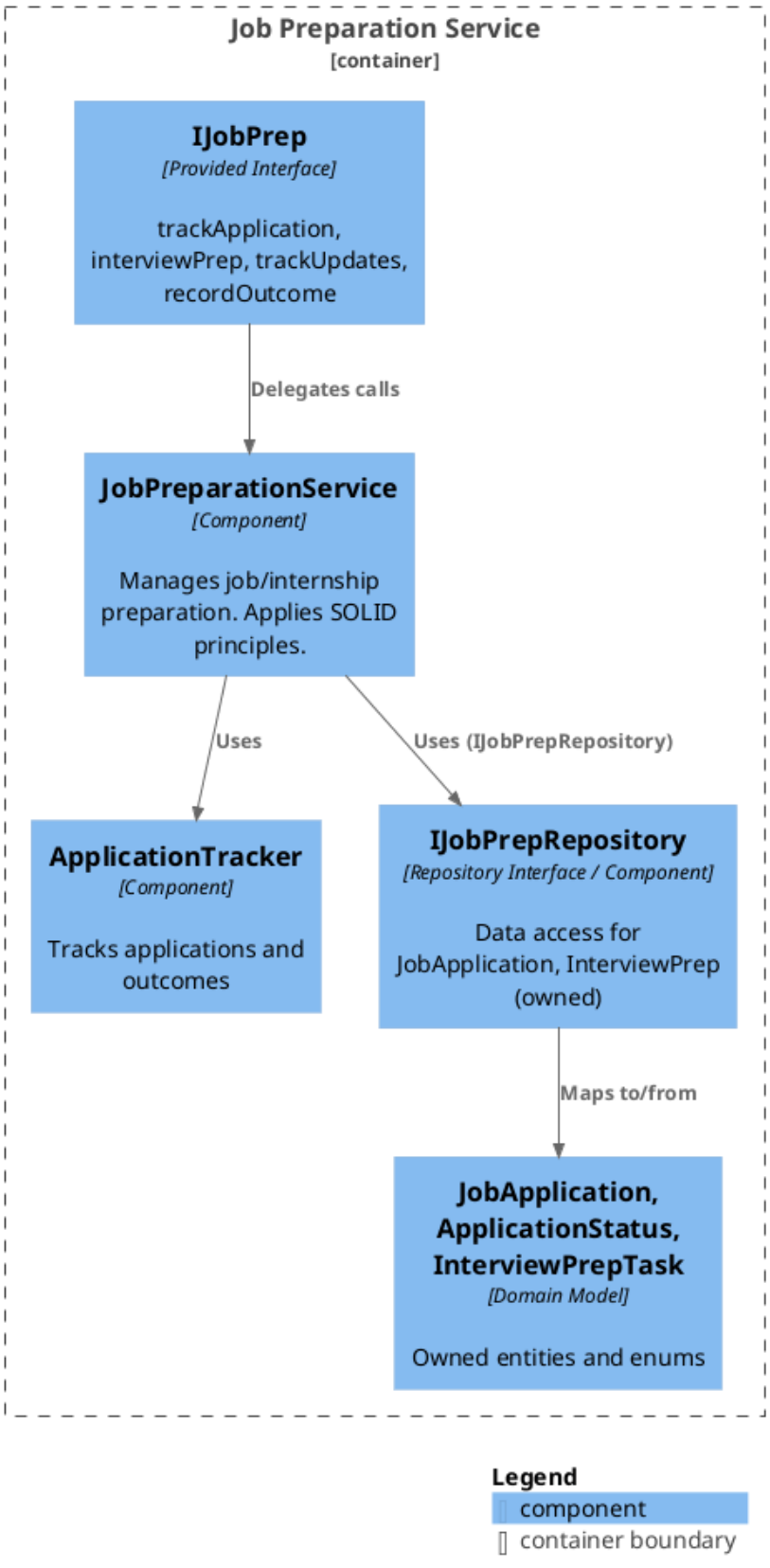


Source: `11-portfolio-service-component.puml`

12. Job Preparation Service — FR-12

Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint

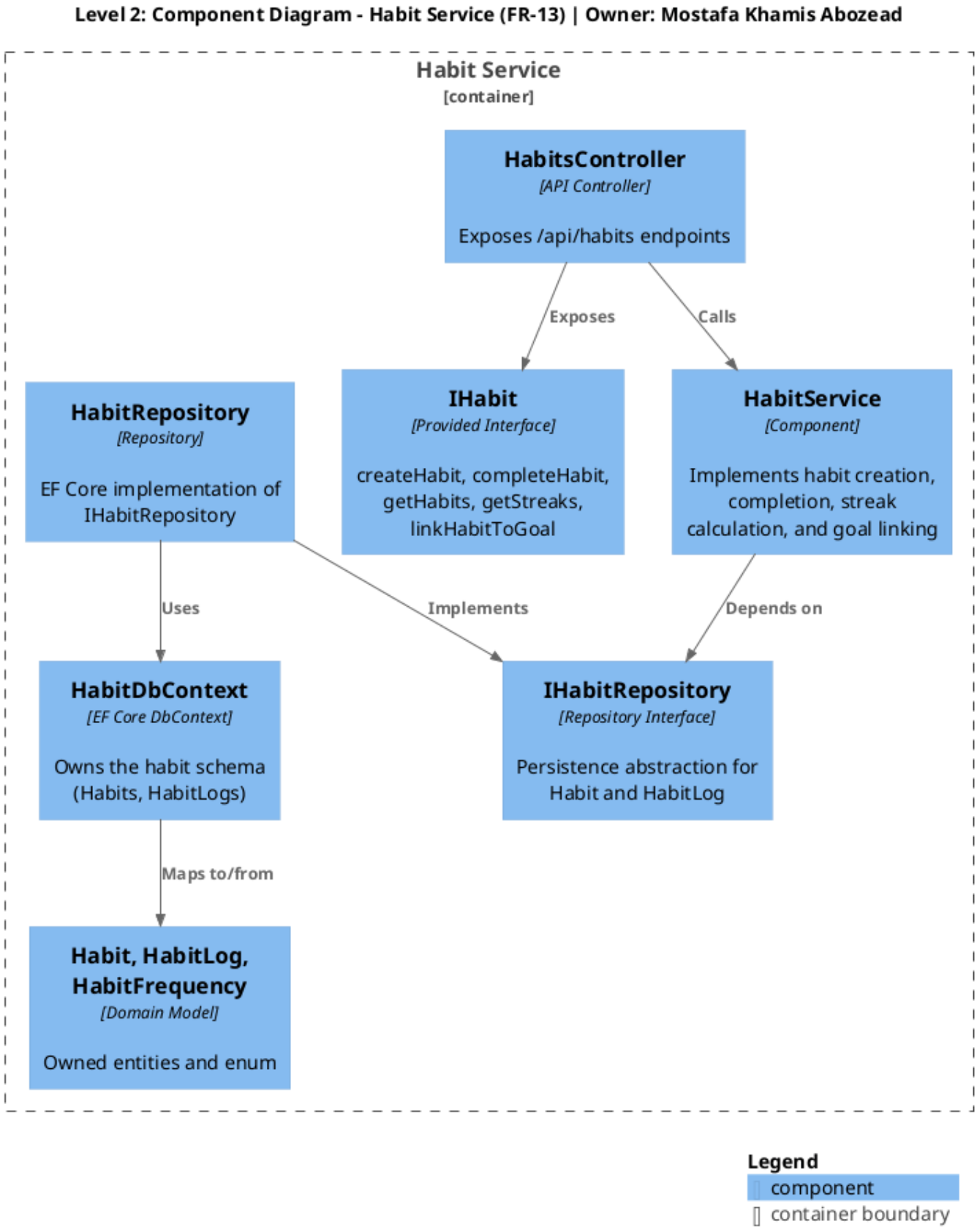
Level 2: Component Diagram - Job Preparation Service (FR-12) | Owner: Arwa Mohey Hamdy



Source: `12-job-preparation-service-component.puml`

13. Habit Service — FR-13

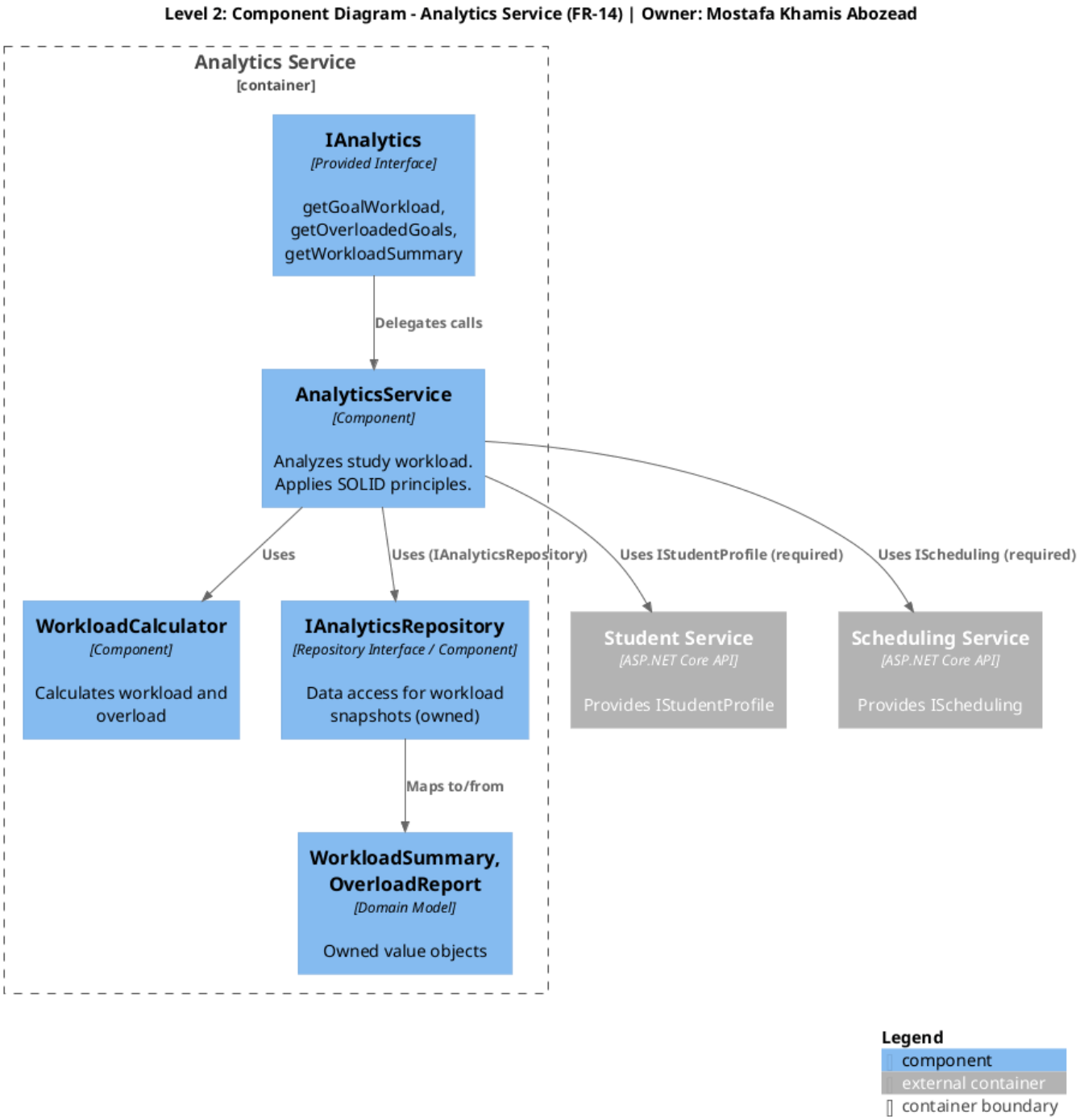
Owner: Mostafa Khamis Abozead · **Status:** Implemented in Sprint 1



Source: `13-habit-service-component.puml`

14. Analytics Service — FR-14

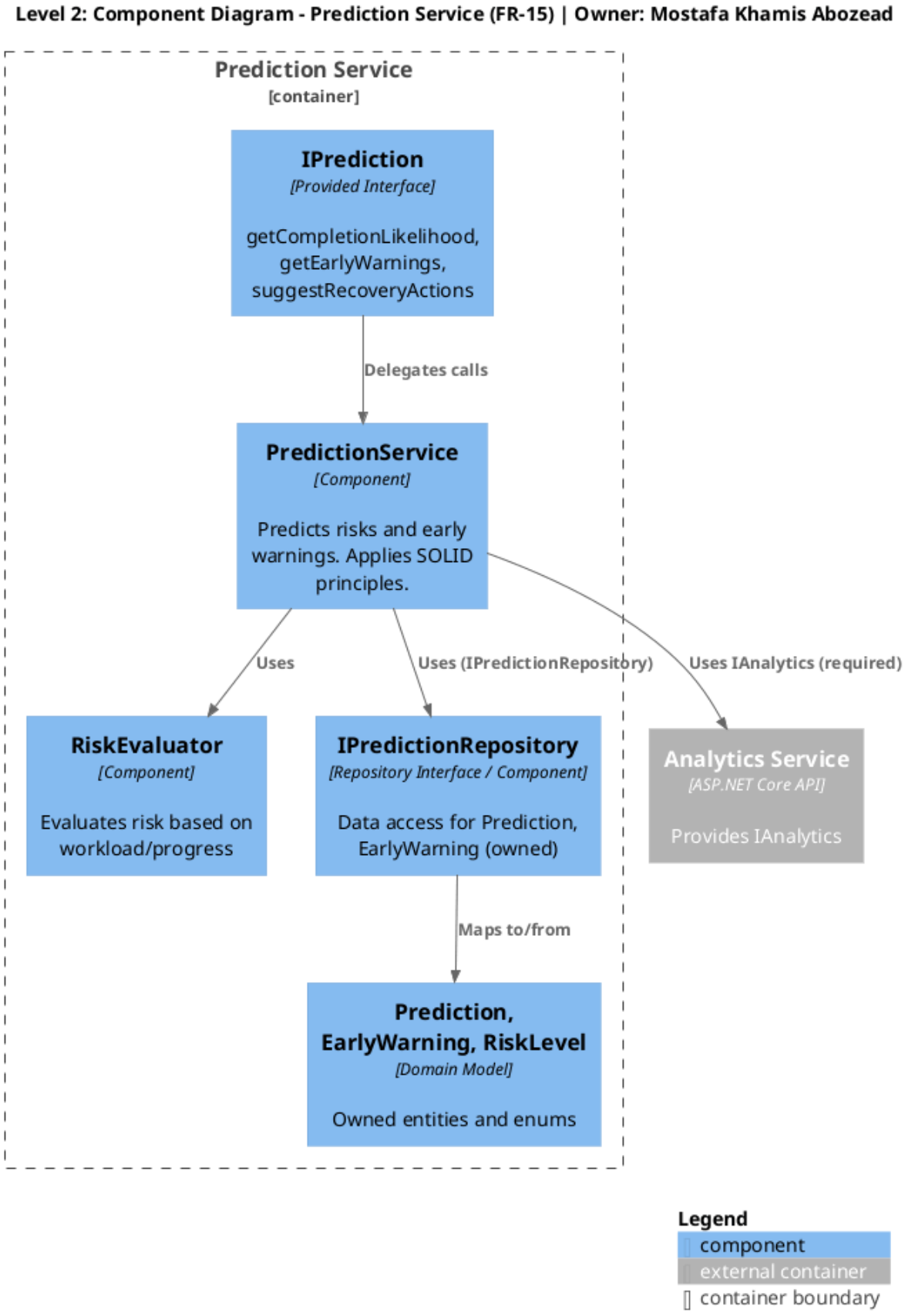
Owner: Mostafa Khamis Abozead · **Status:** Design complete; implementation scheduled for a later sprint



Source: `14-analytics-service-component.puml`

15. Prediction Service — FR-15

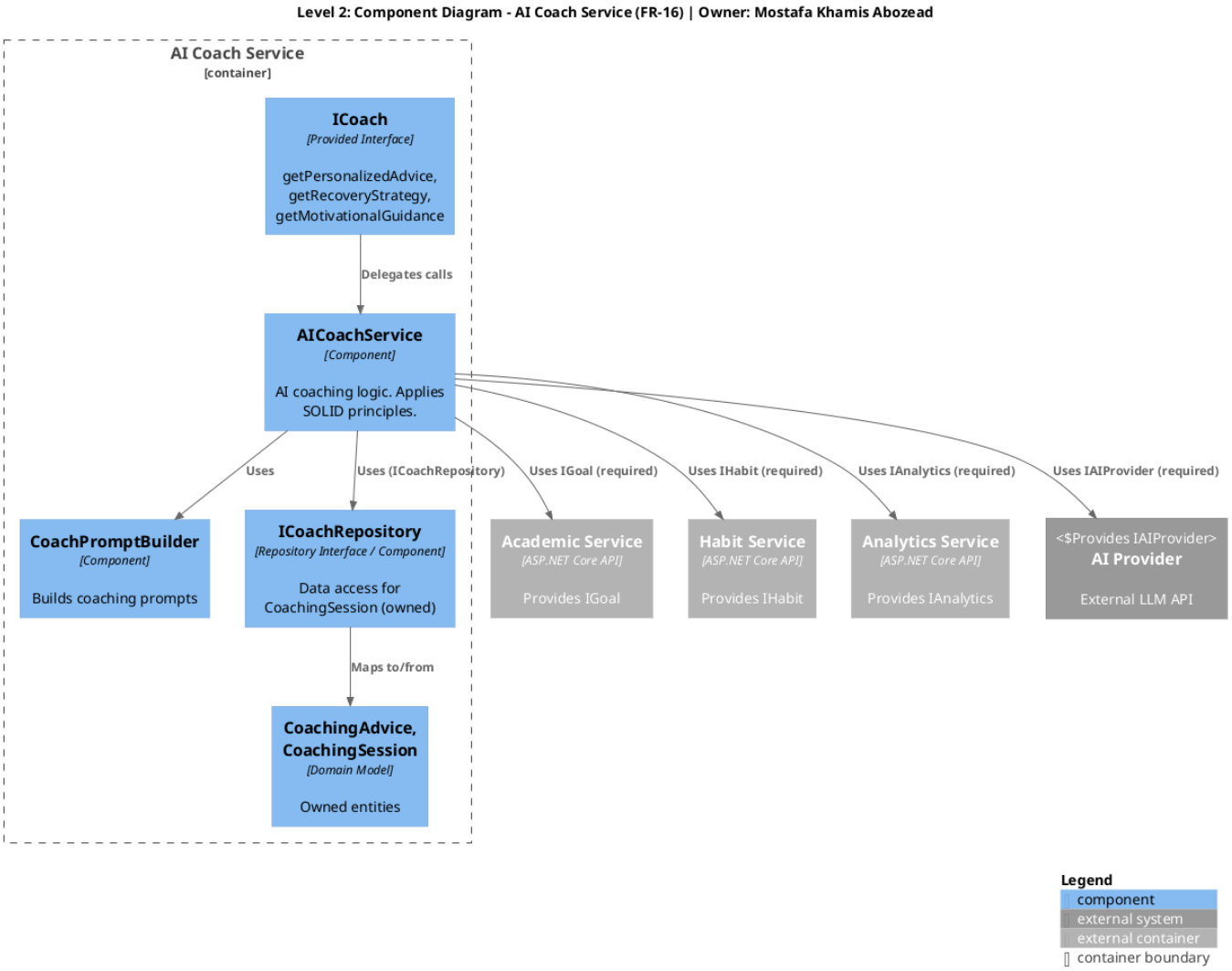
Owner: Mostafa Khamis Abozead · **Status:** Design complete; implementation scheduled for a later sprint



Source: `15-prediction-service-component.puml`

16. AI Coach Service — FR-16

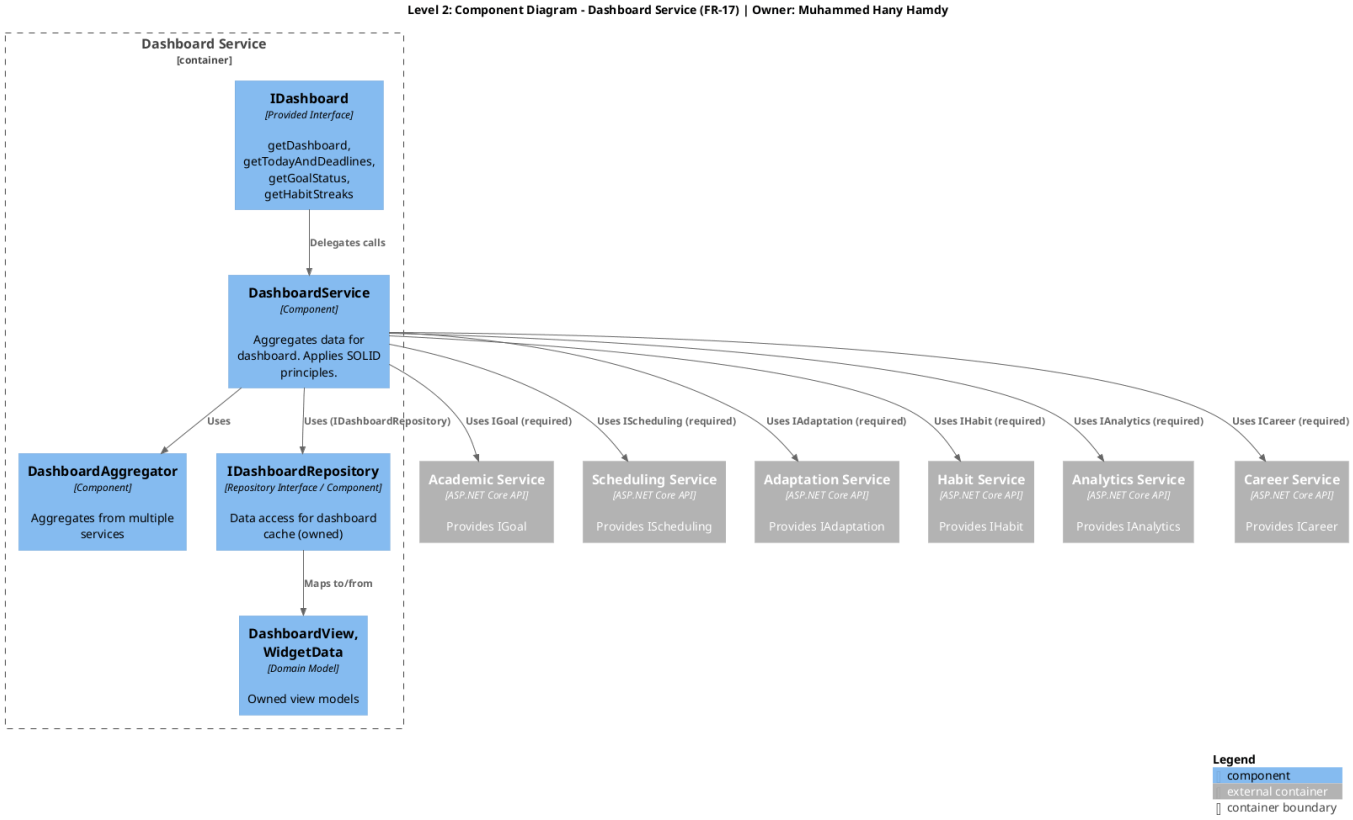
Owner: Mostafa Khamis Abozead · **Status:** Design complete; implementation scheduled for a later sprint



Source: `16-ai-coach-service-component.puml`

17. Dashboard Service — FR-17

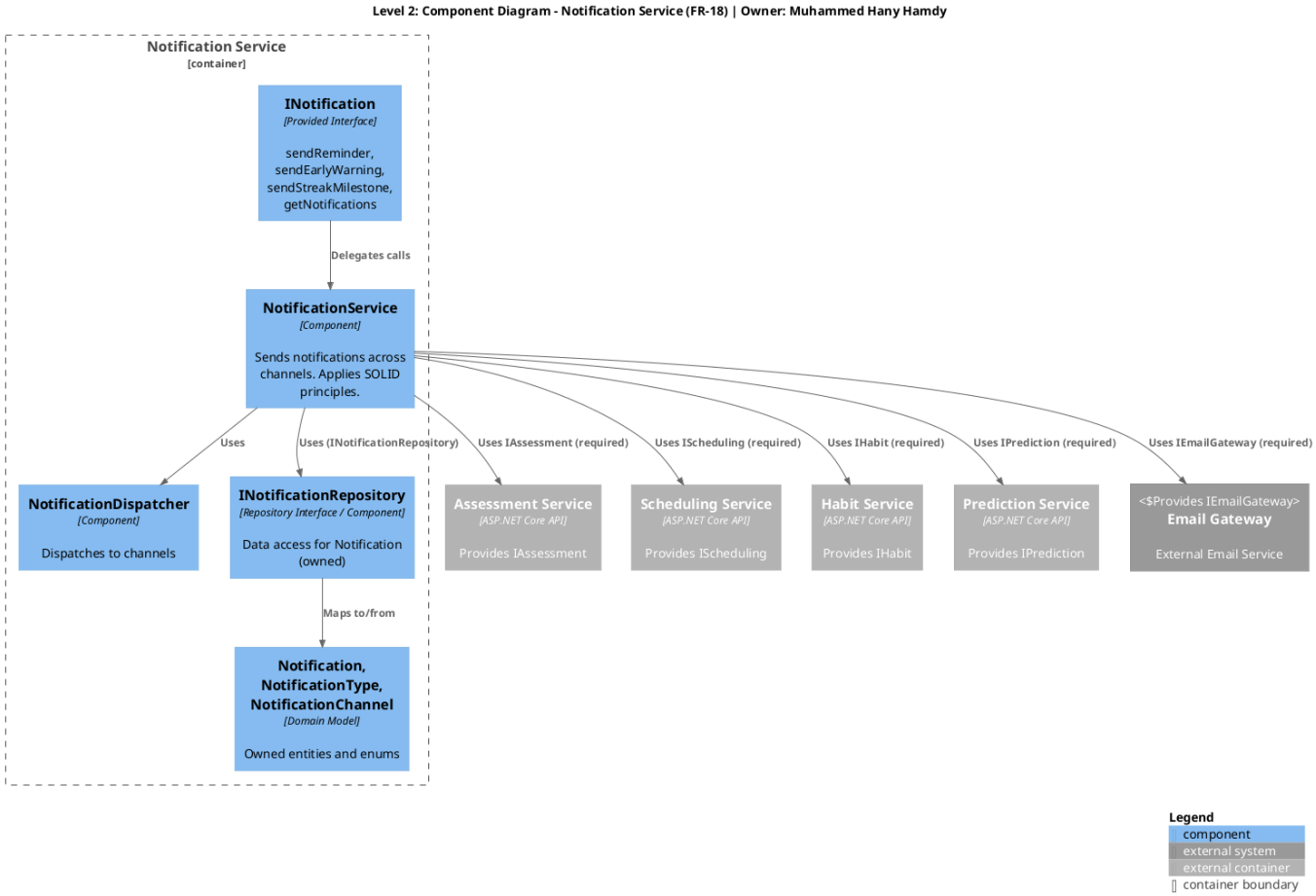
Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `17-dashboard-service-component.puml`

18. Notification Service — FR-18

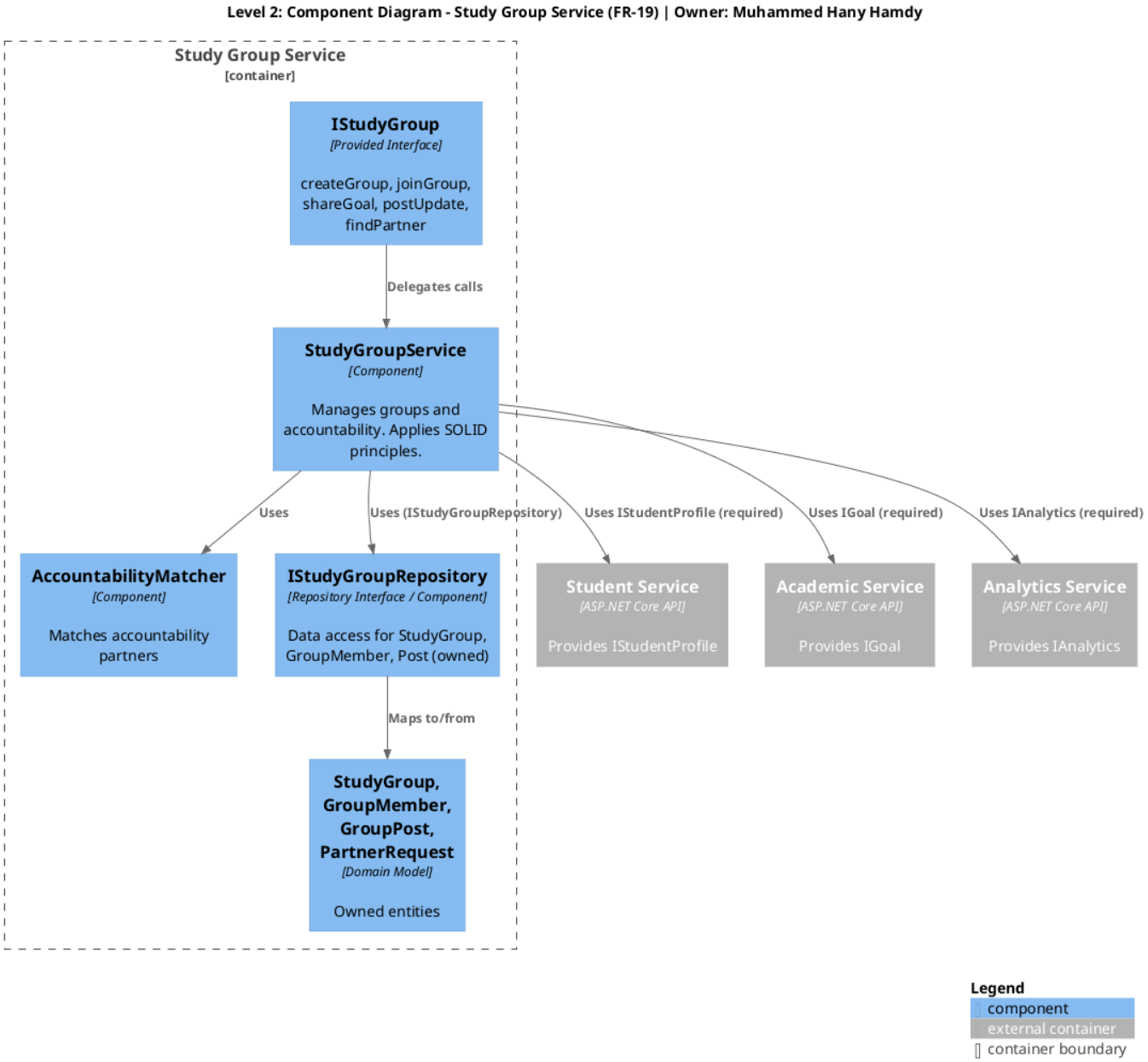
Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `18-notification-service-component.puml`

19. Study Group Service — FR-19

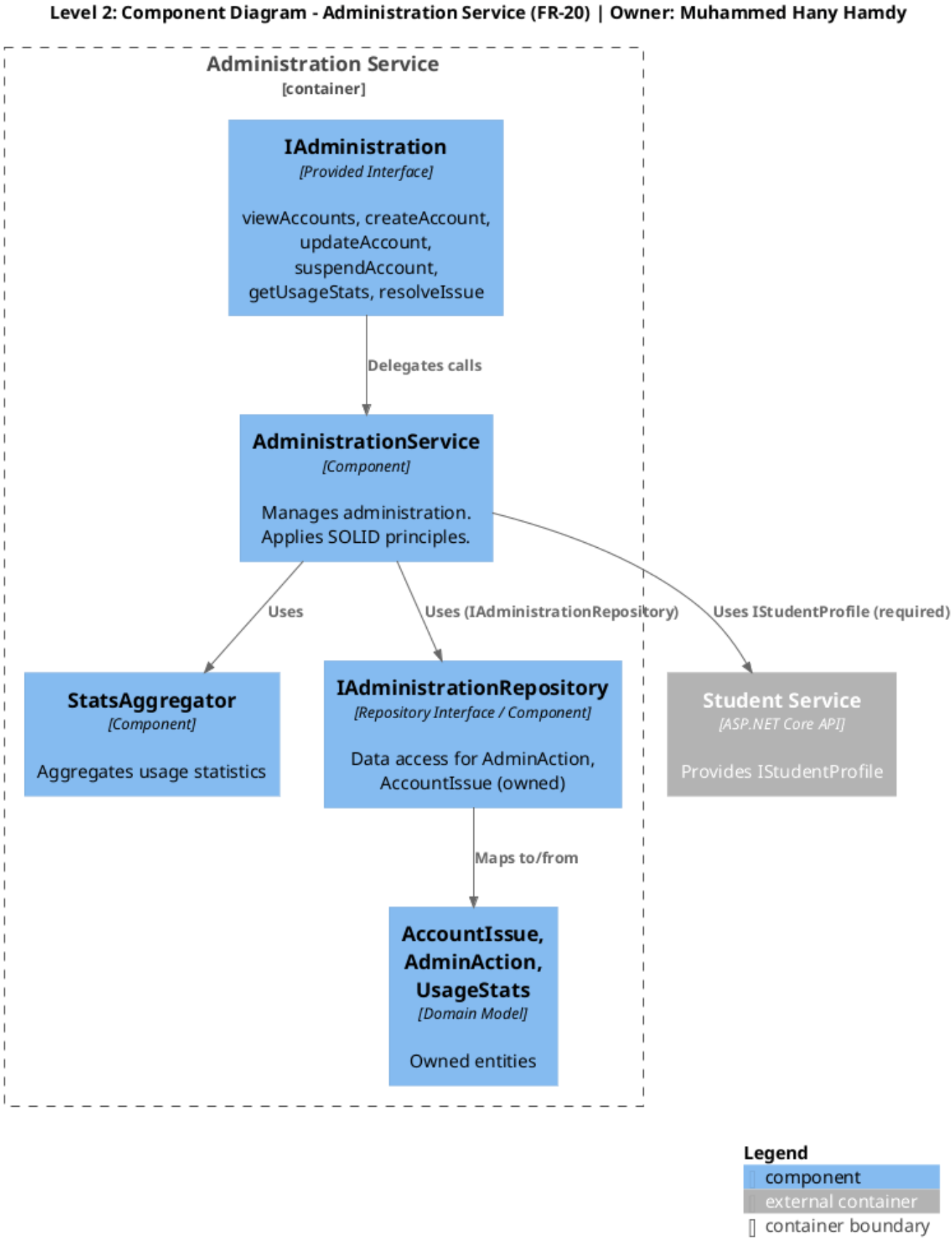
Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `19-study-group-service-component.puml`

20. Administration Service — FR-20

Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint

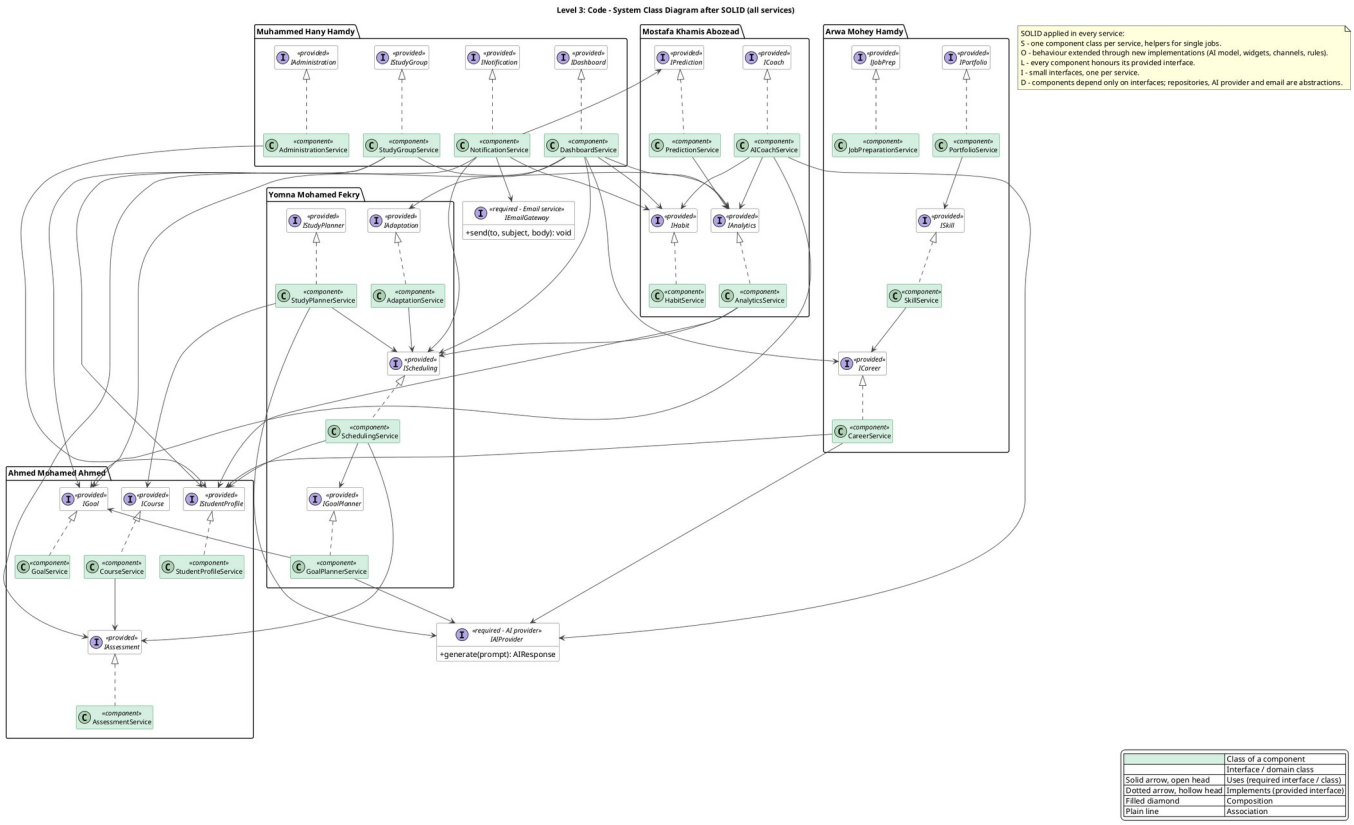


Source: `20-administration-service-component.puml`

A.5 Class Diagrams (Level 3)

The system class diagram applies SOLID principles across all services. Each service then has its own class diagram showing its provided interface, service class, required interfaces, repositories, adapters, and domain model. The six implemented services reflect the real classes and method names in code.

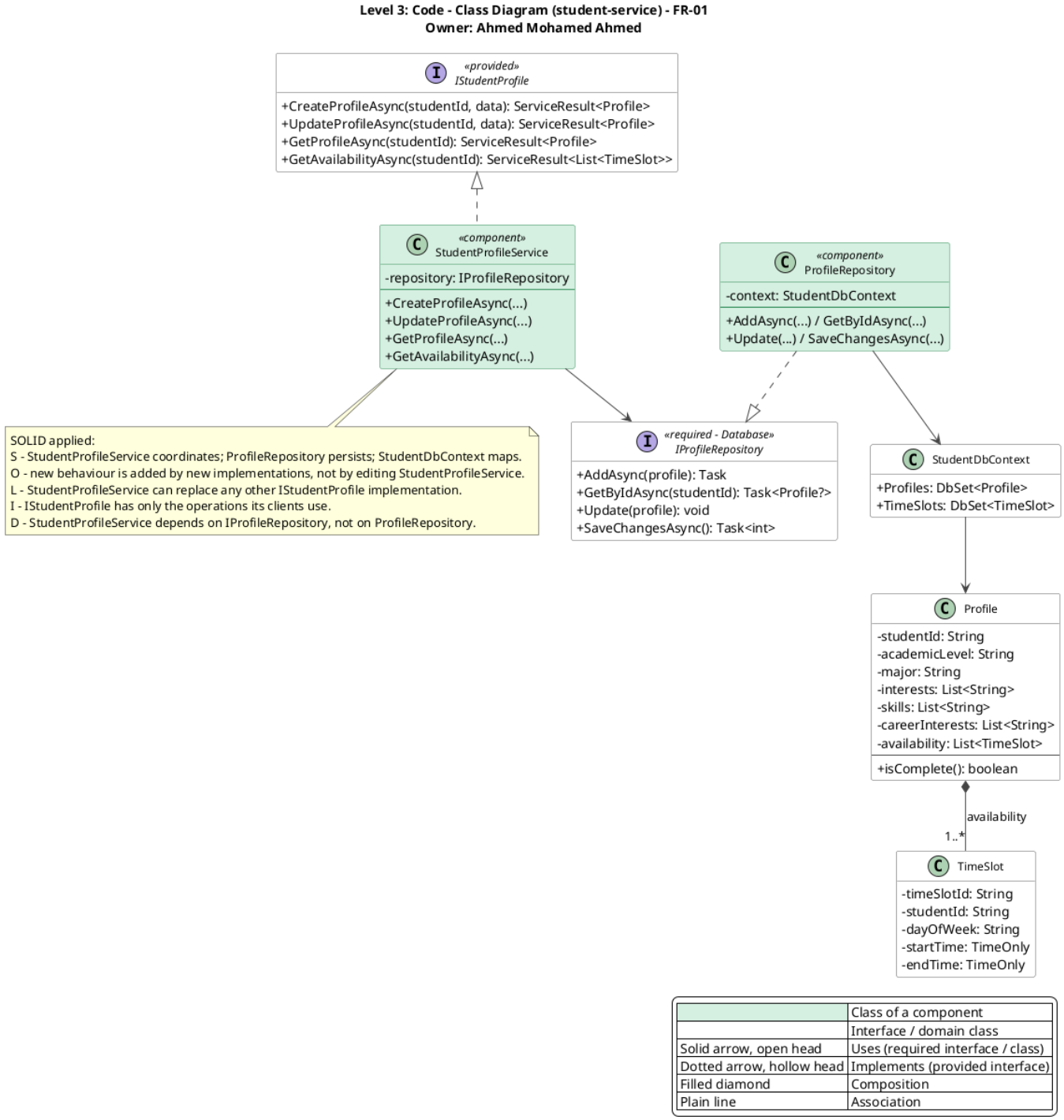
System Class Diagram



Source: `00-system-class-diagram.puml`

01. Student Service — FR-01

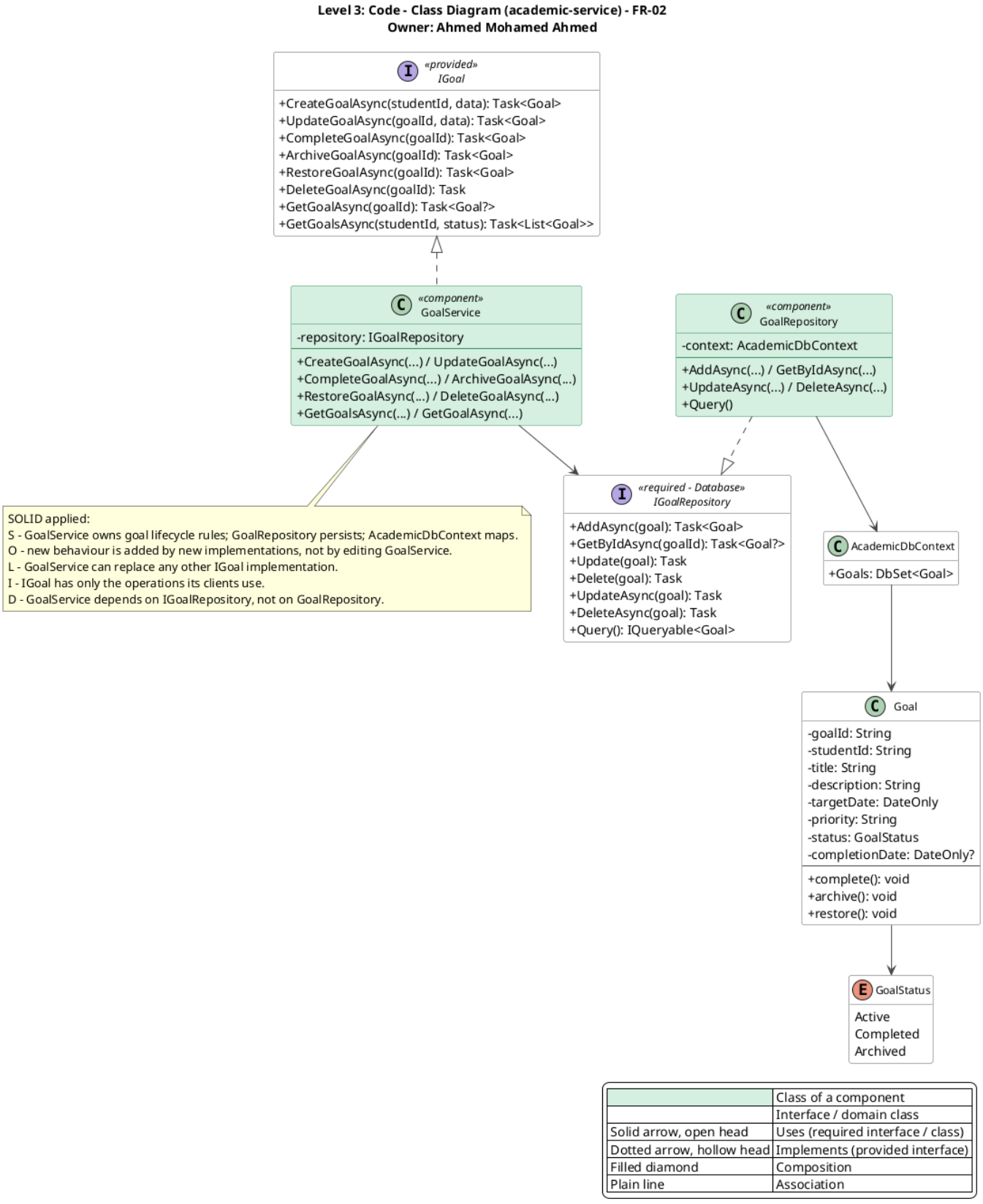
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `01-student-service.puml`

02. Academic Service — FR-02

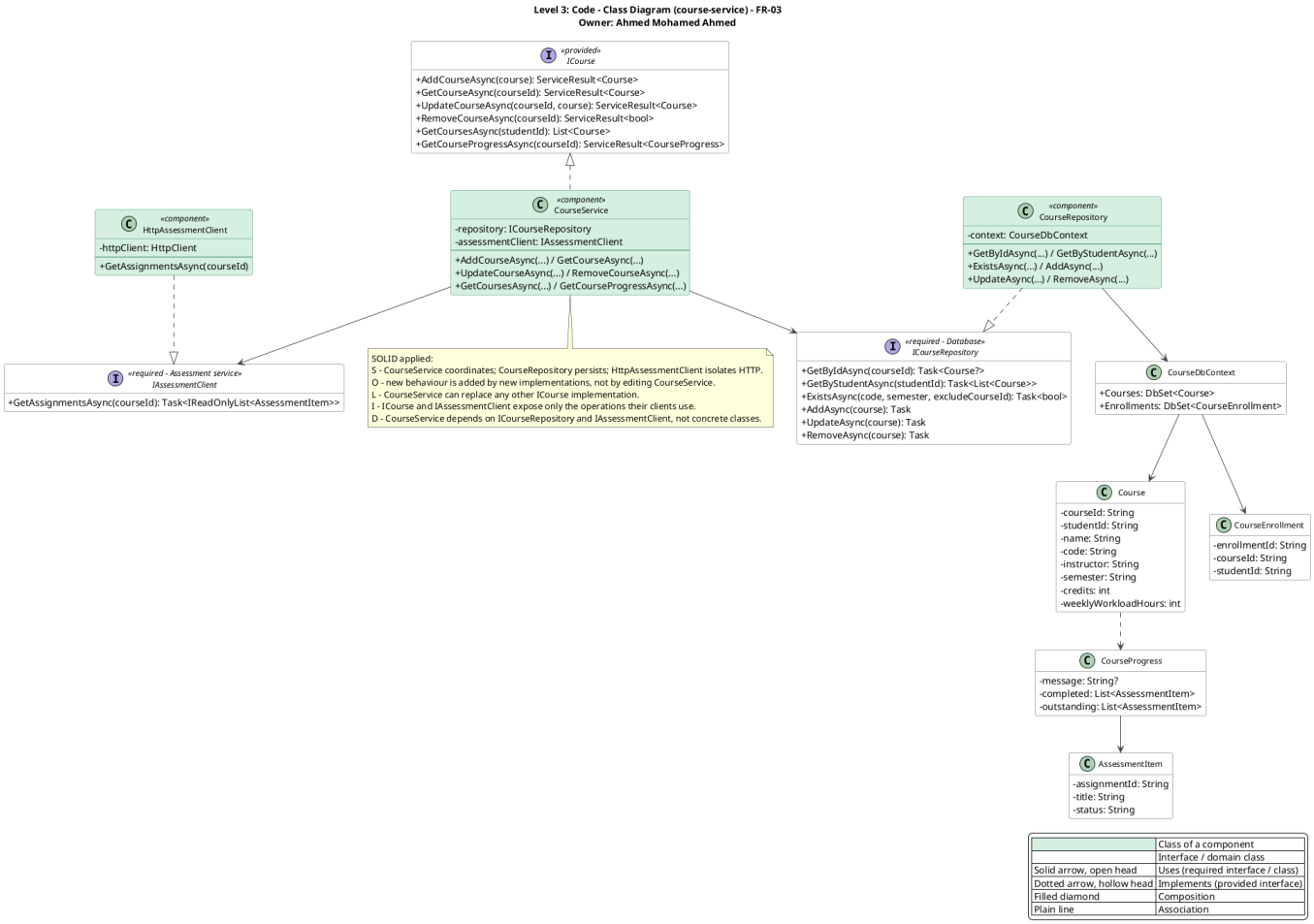
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `02-academic-service.puml`

03. Course Service — FR-03

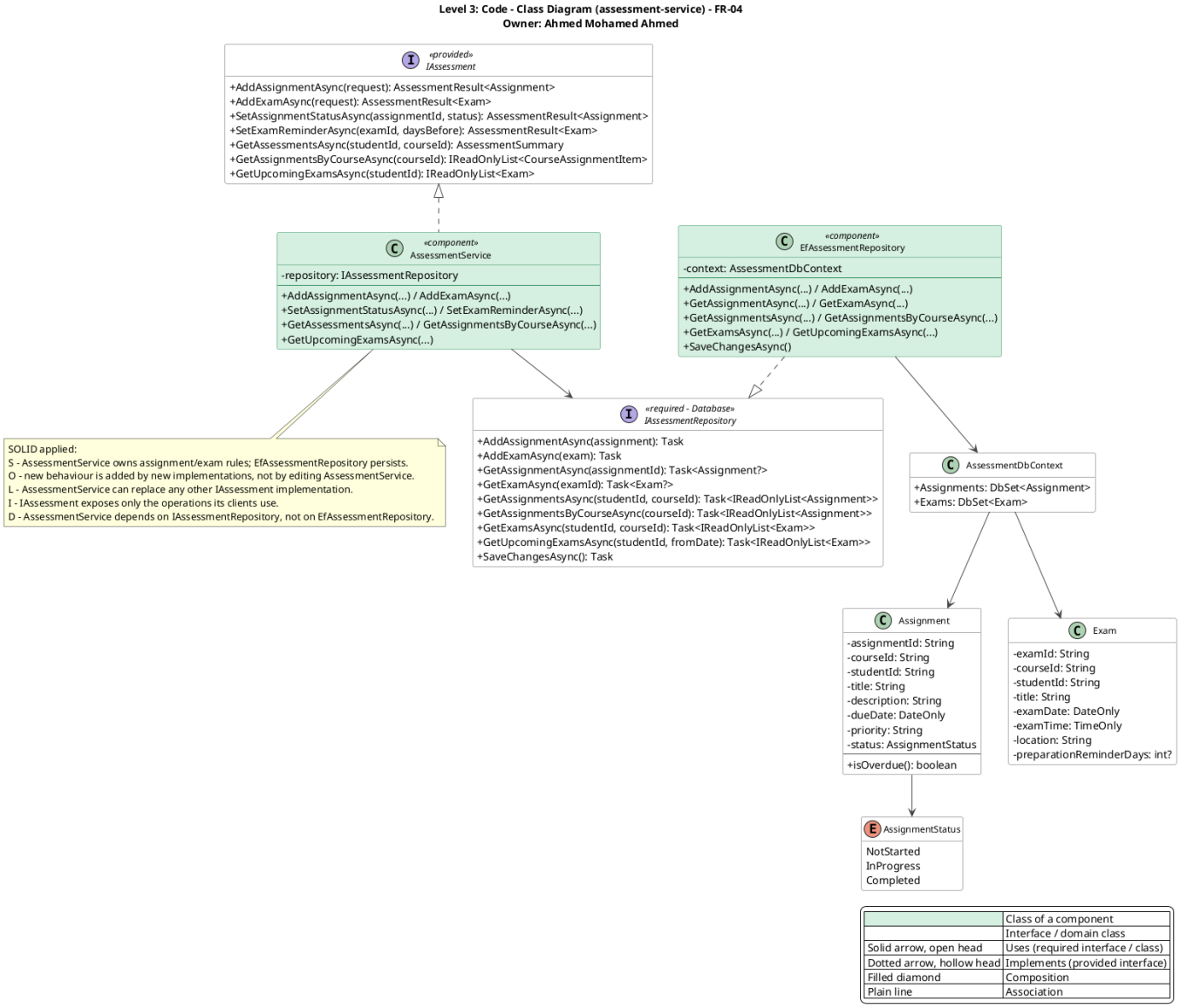
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `03-course-service.puml`

04. Assessment Service — FR-04

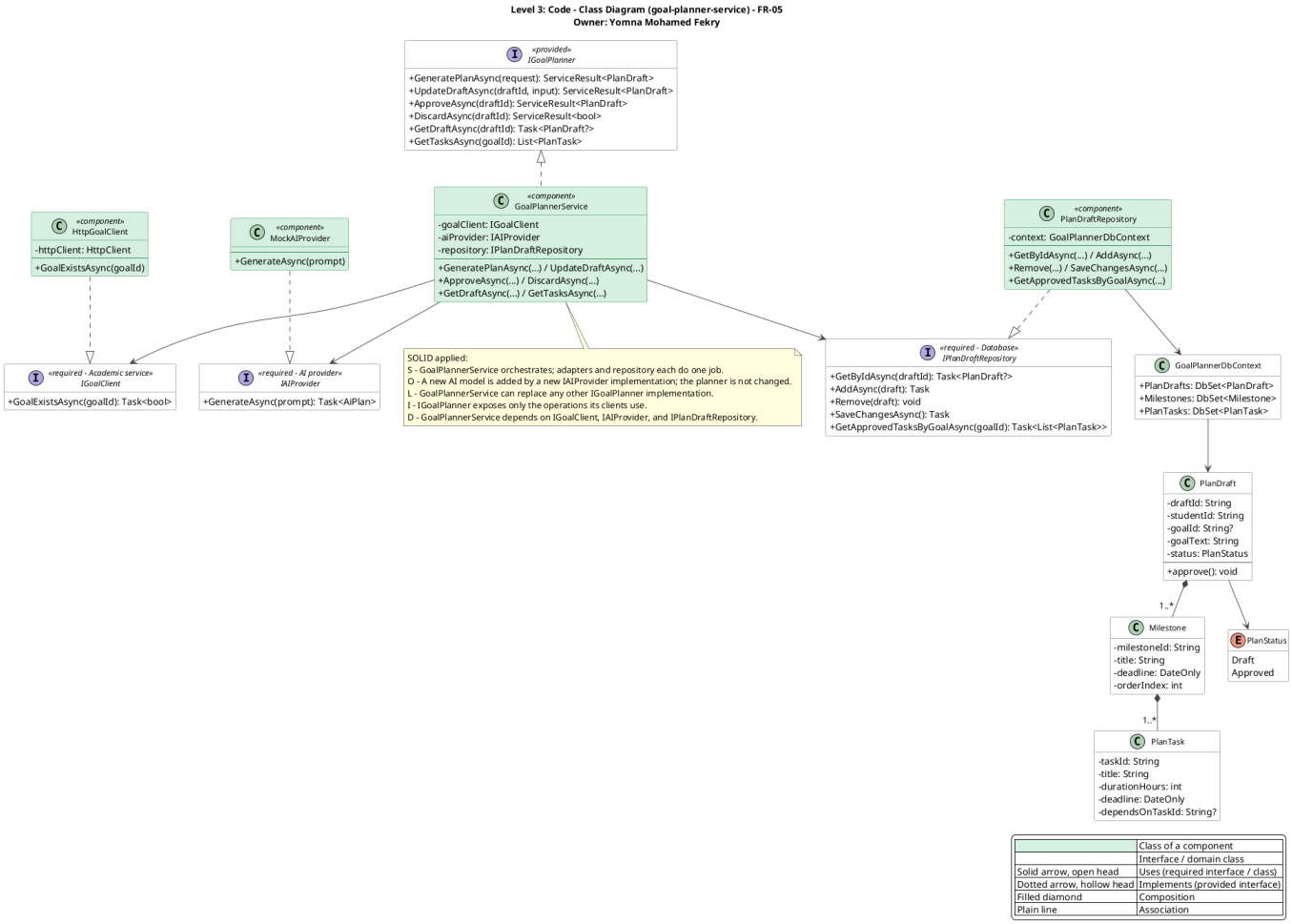
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `04-assessment-service.puml`

05. Goal Planner Service — FR-05

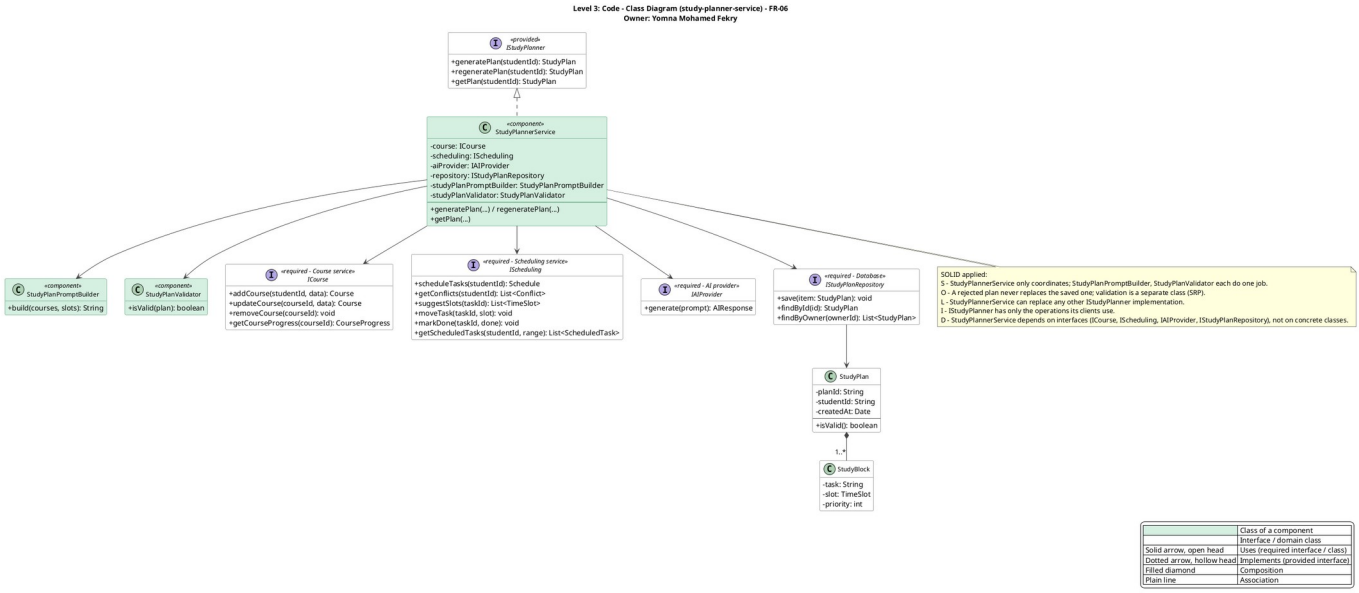
Owner: Yomna Mohamed Fekry · **Status:** Implemented in Sprint 1



Source: `05-goal-planner-service.puml`

06. Study Planner Service — FR-06

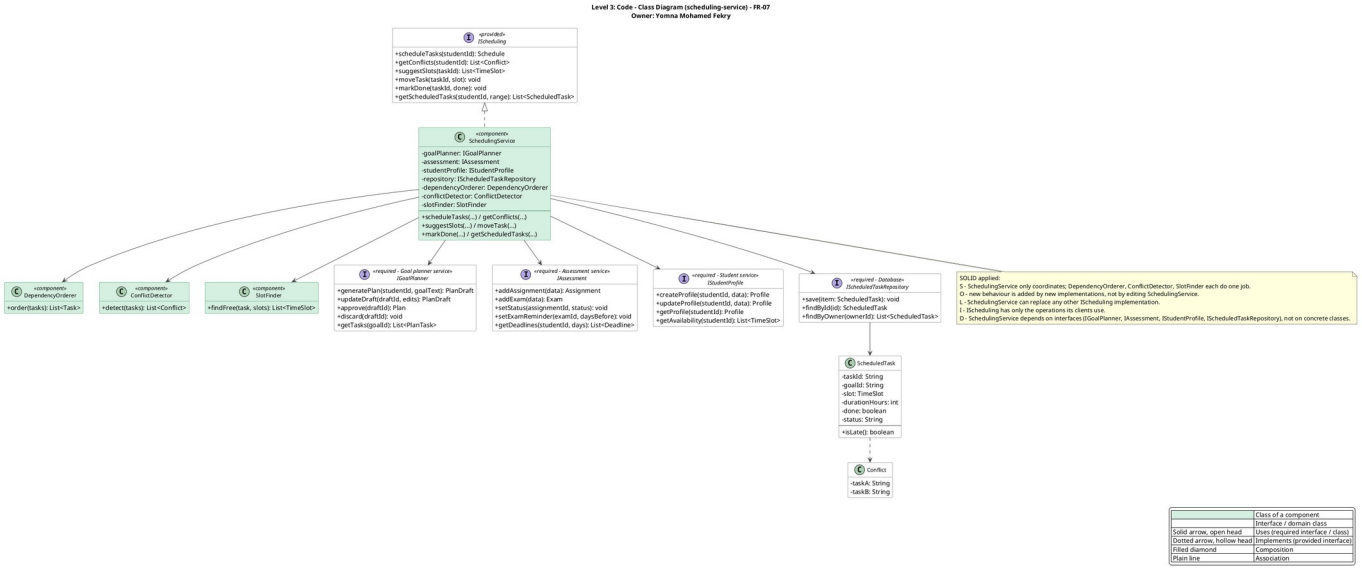
Owner: Yomna Mohamed Fekry · **Status:** Design complete; implementation scheduled for a later sprint



Source: `06-study-planner-service.puml`

07. Scheduling Service — FR-07

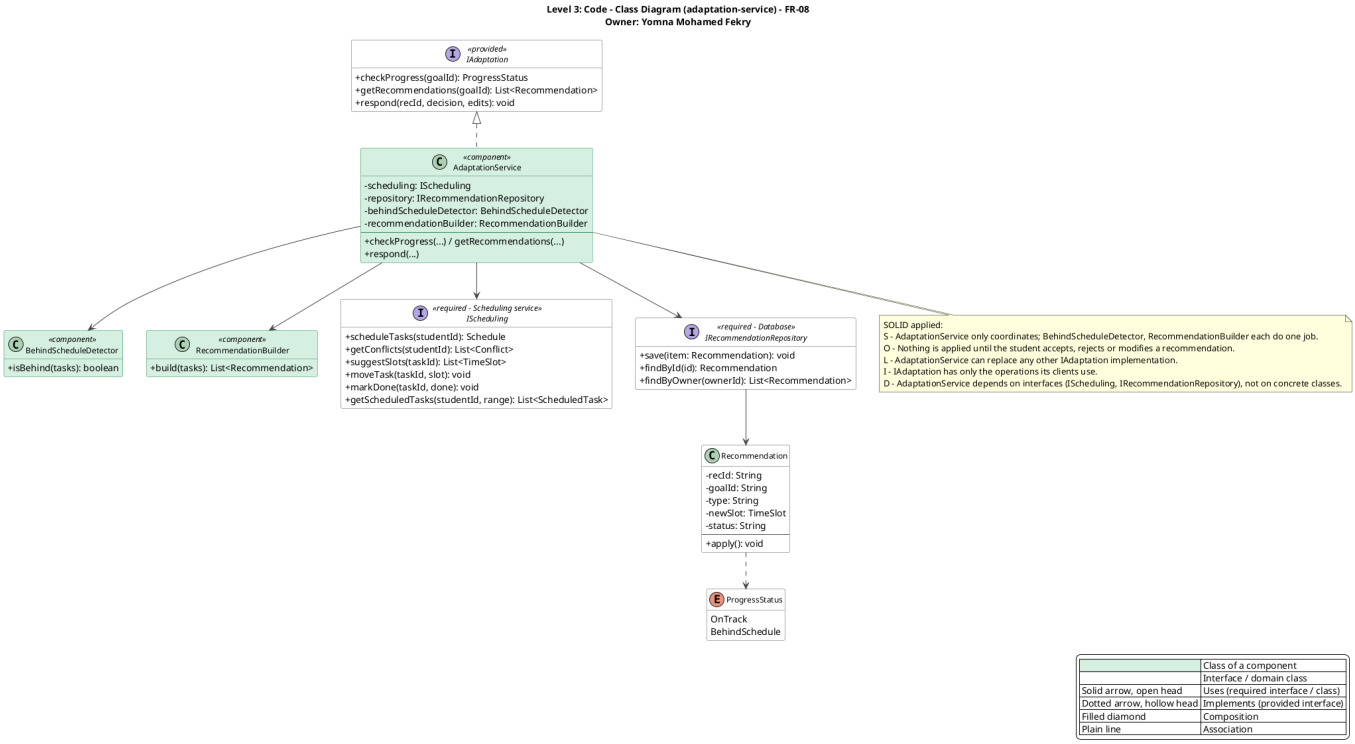
Owner: Yomna Mohamed Fekry · **Status:** Design complete; implementation scheduled for a later sprint



Source: `07-scheduling-service.puml`

08. Adaptation Service — FR-08

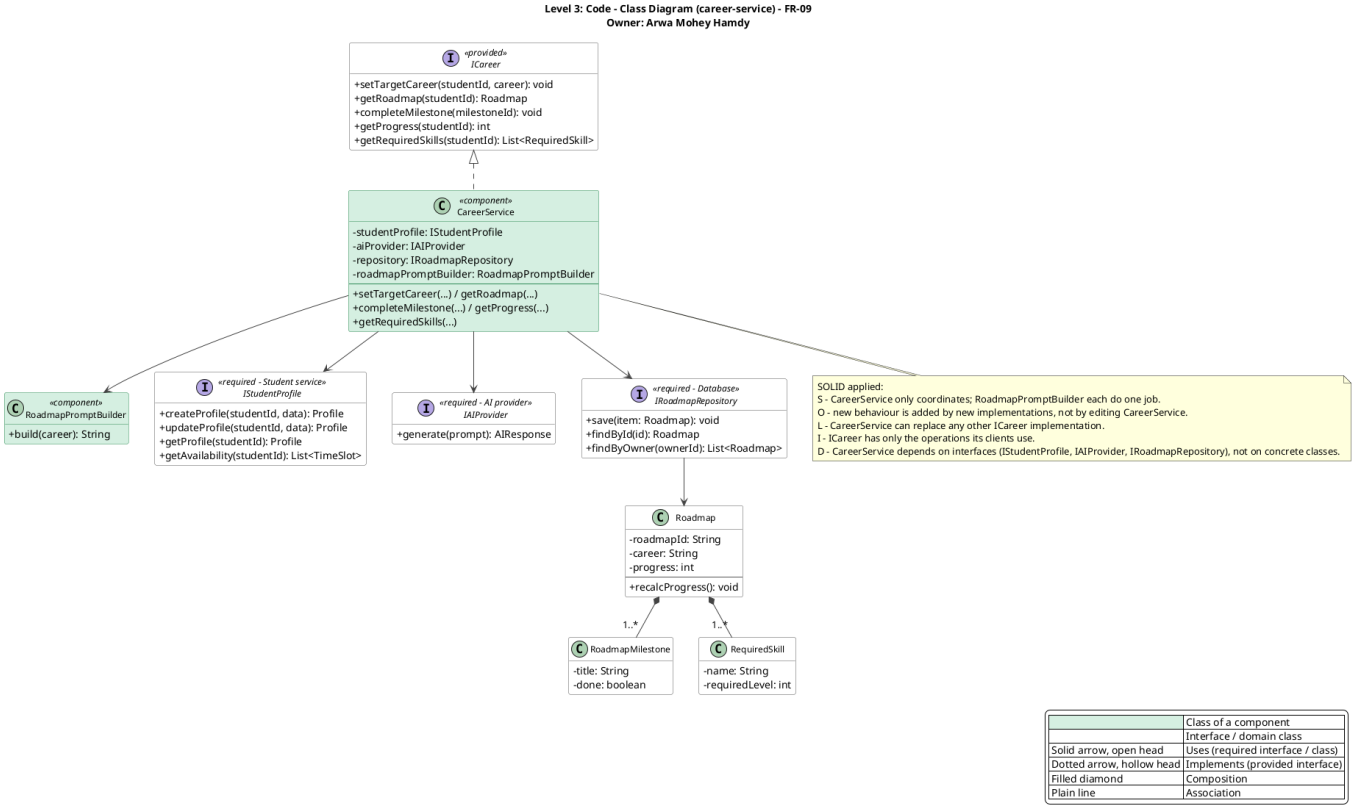
Owner: Yomna Mohamed Fekry · **Status:** Design complete; implementation scheduled for a later sprint



Source: `08-adaptation-service.puml`

09. Career Service — FR-09

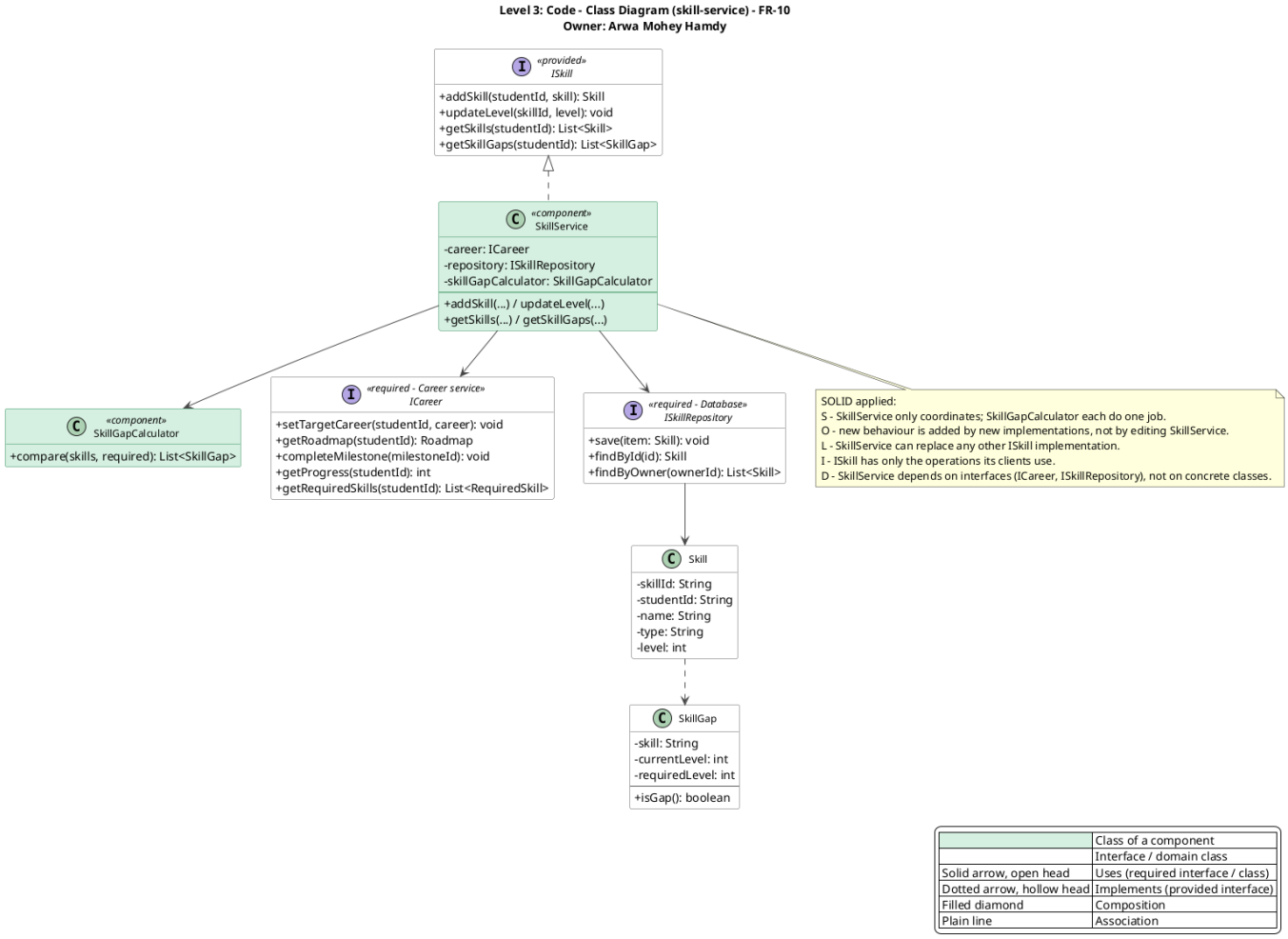
Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `09-career-service.puml`

10. Skill Service — FR-10

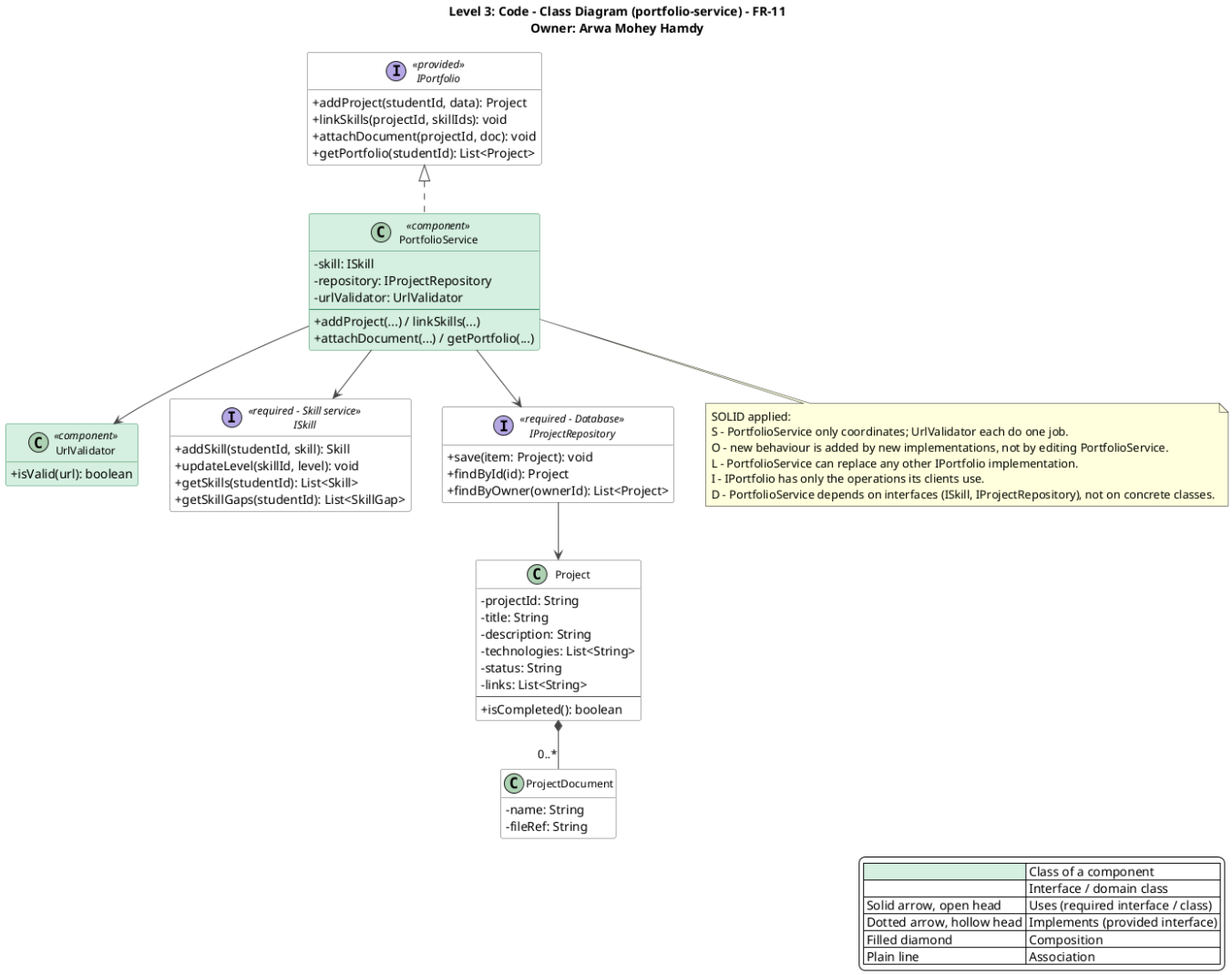
Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `10-skill-service.puml`

11. Portfolio Service — FR-11

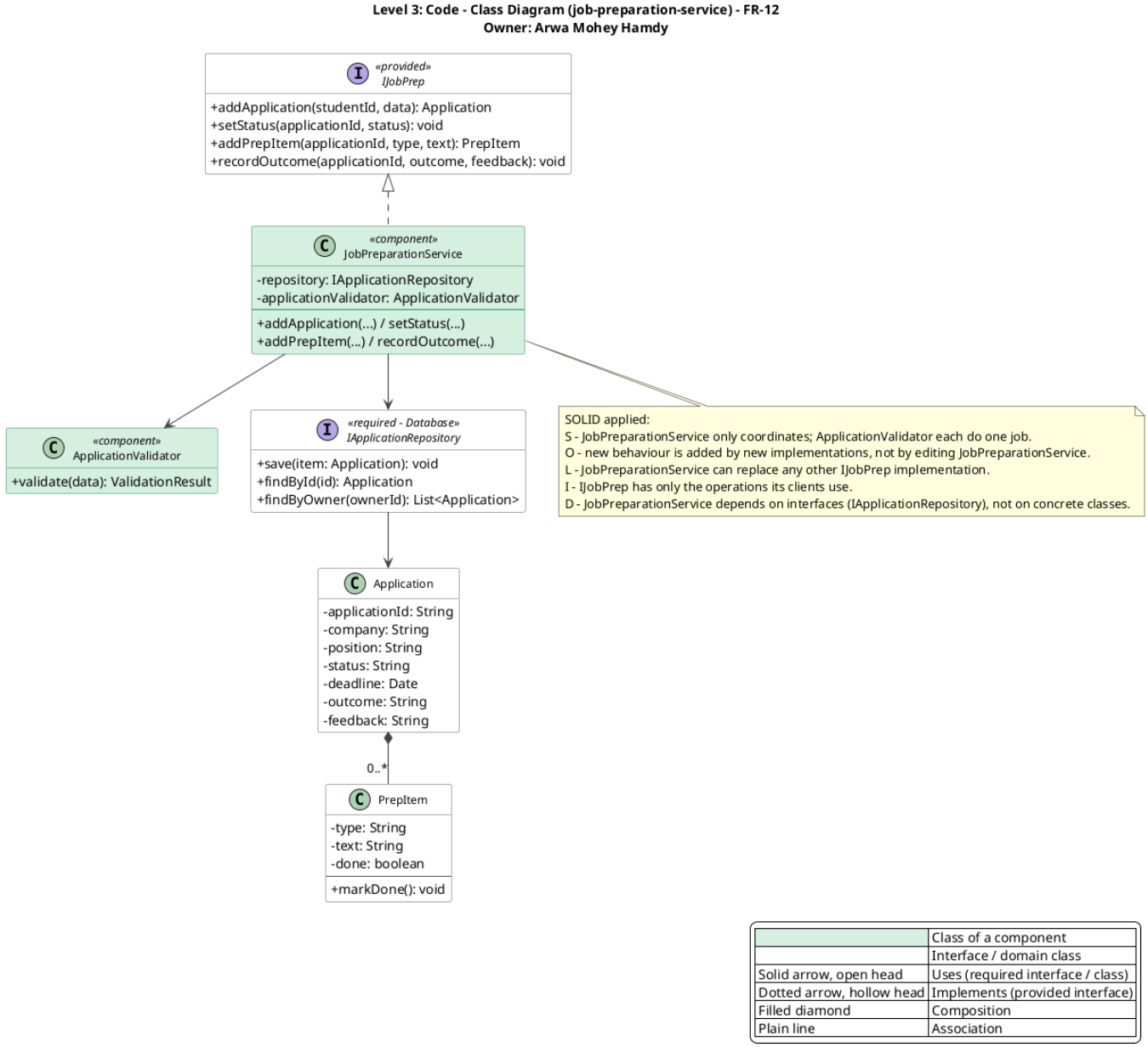
Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `11-portfolio-service.puml`

12. Job Preparation Service — FR-12

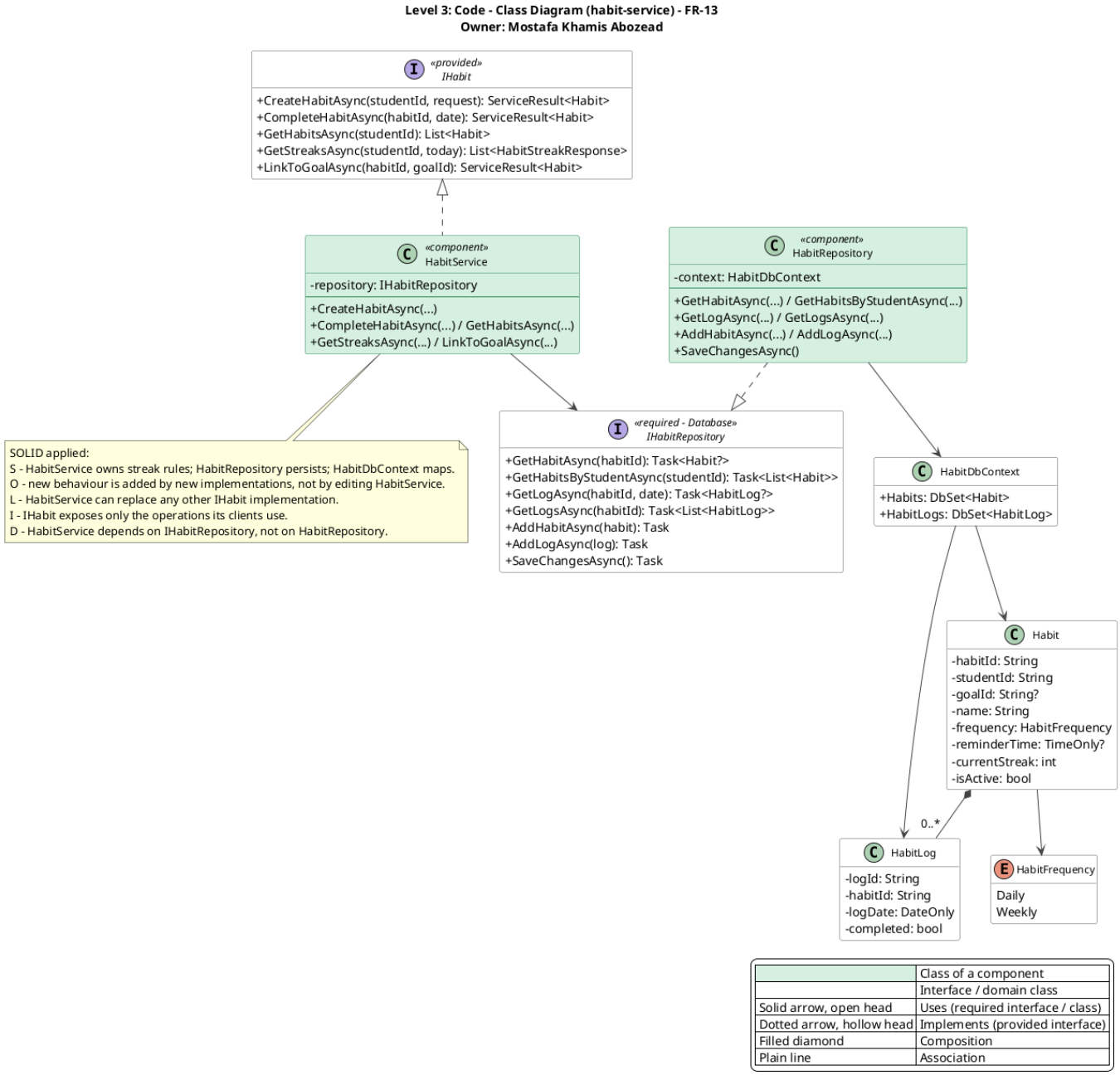
Owner: Arwa Mohey Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `12-job-preparation-service.puml`

13. Habit Service — FR-13

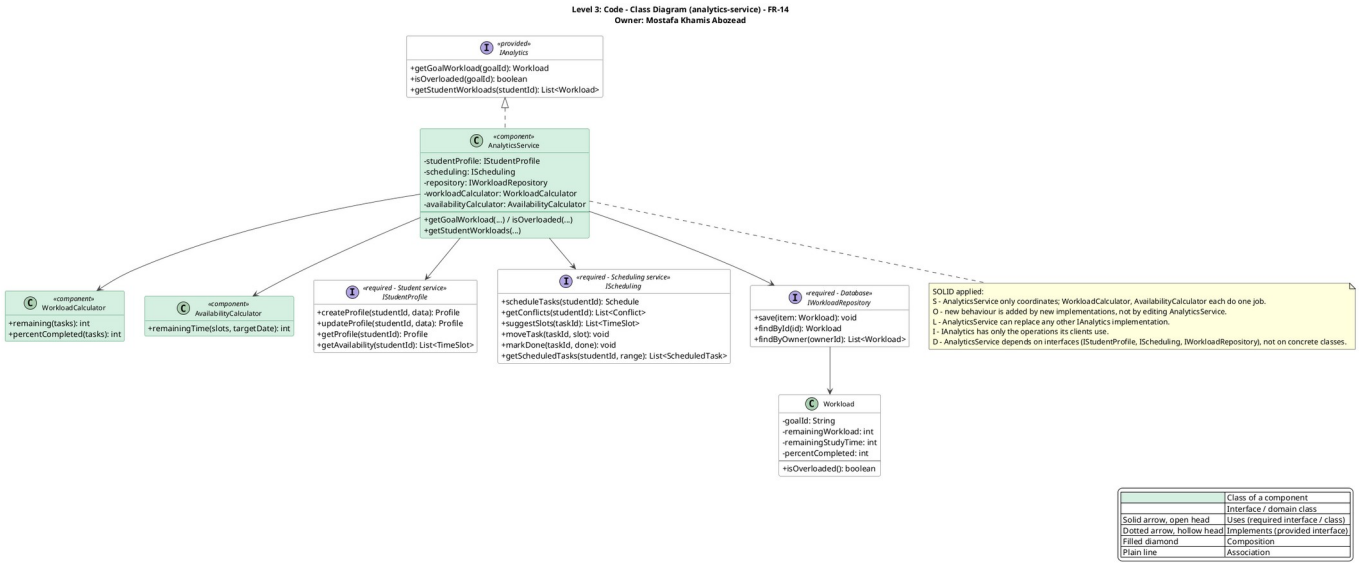
Owner: Mostafa Khamis Abozead · **Status:** Implemented in Sprint 1



Source: `13-habit-service.puml`

14. Analytics Service — FR-14

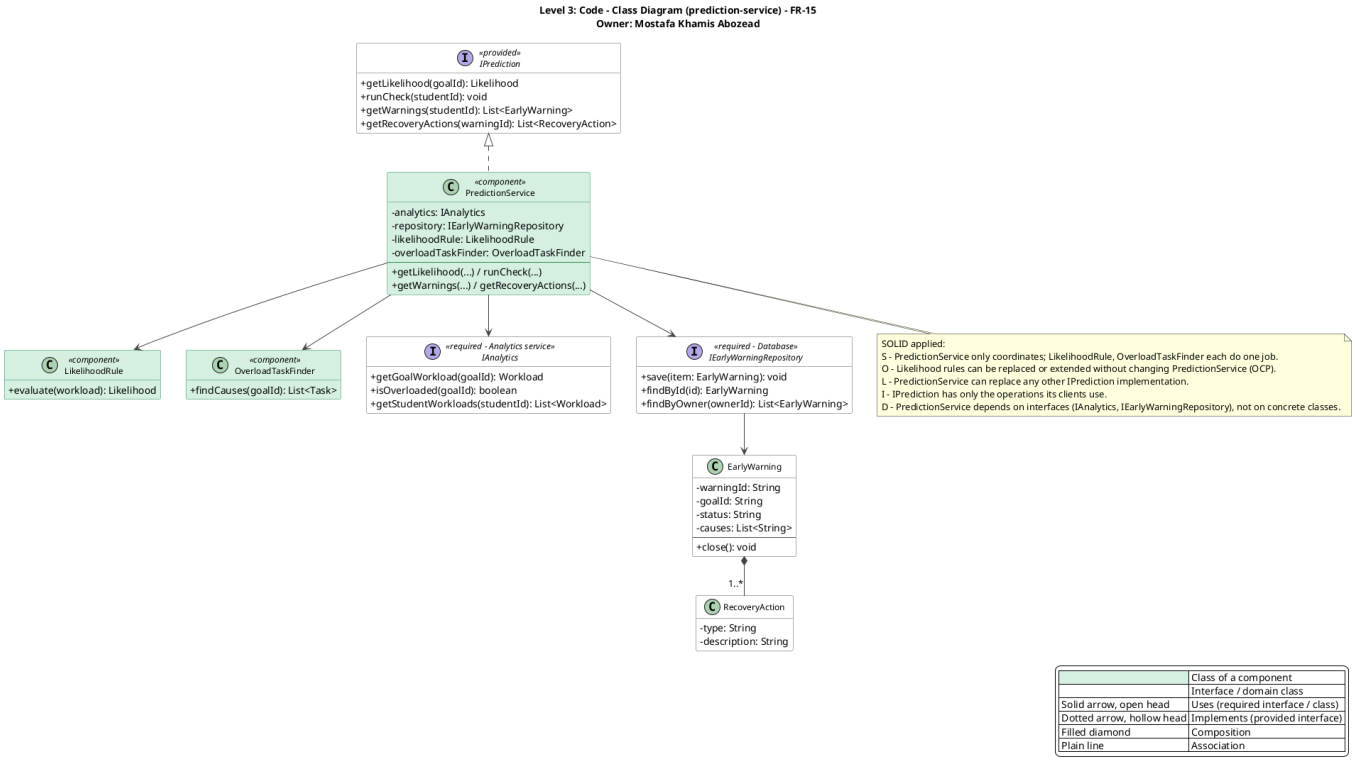
Owner: Mostafa Khamis Abozead · **Status:** Design complete; implementation scheduled for a later sprint



Source: `14-analytics-service.puml`

15. Prediction Service — FR-15

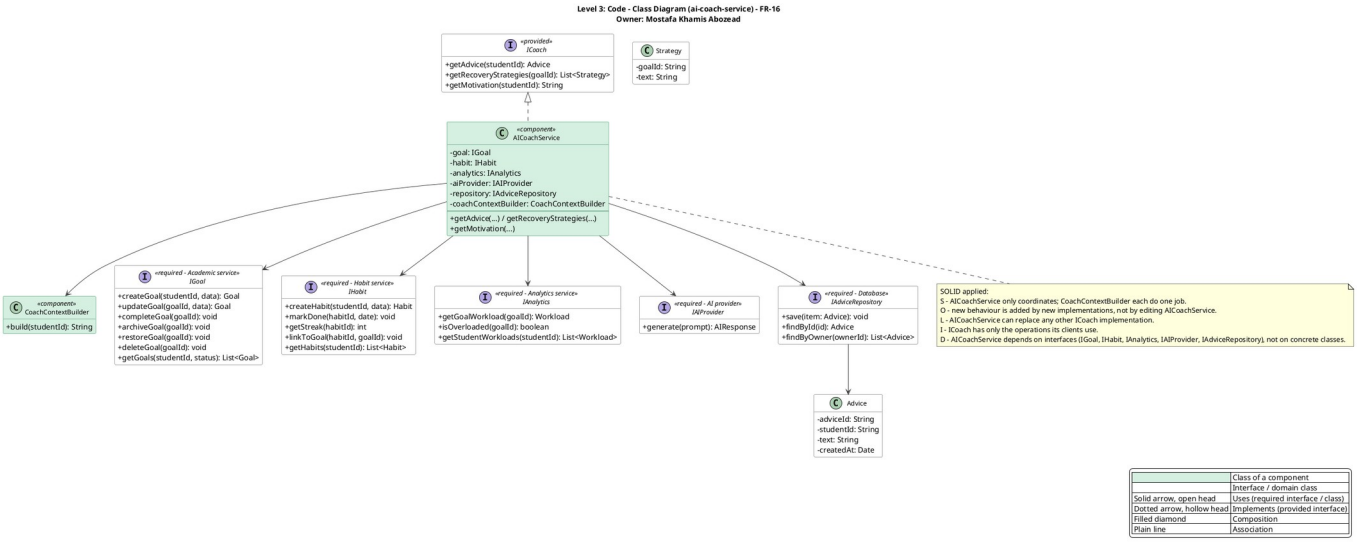
Owner: Mostafa Khamis Abozead · **Status:** Design complete; implementation scheduled for a later sprint



Source: `15-prediction-service.puml`

16. AI Coach Service — FR-16

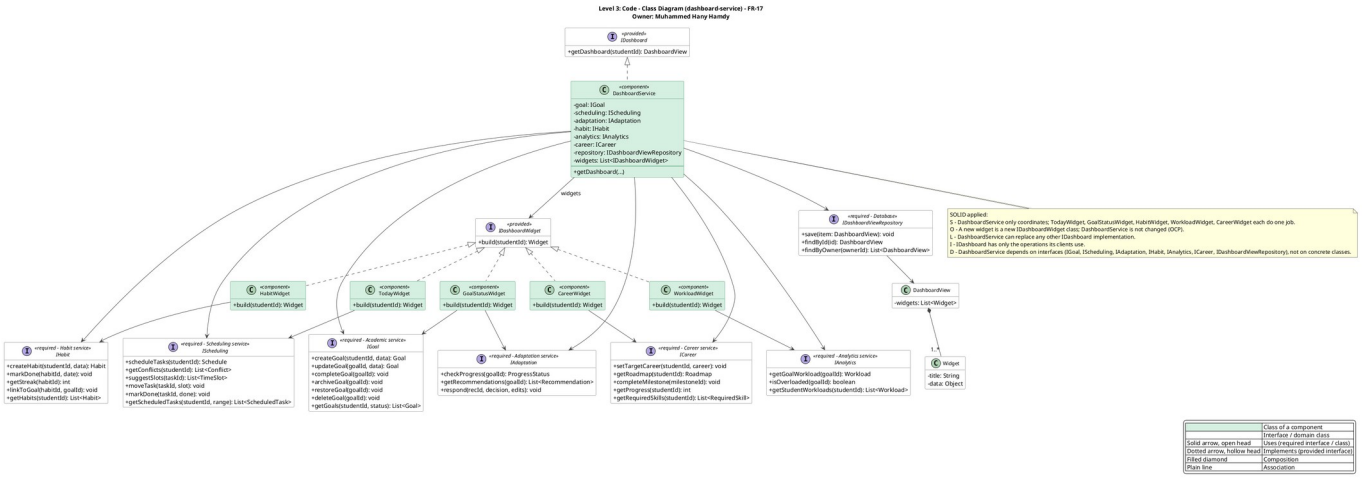
Owner: Mostafa Khamis Abozead · **Status:** Design complete; implementation scheduled for a later sprint



Source: `16-ai-coach-service.puml`

17. Dashboard Service — FR-17

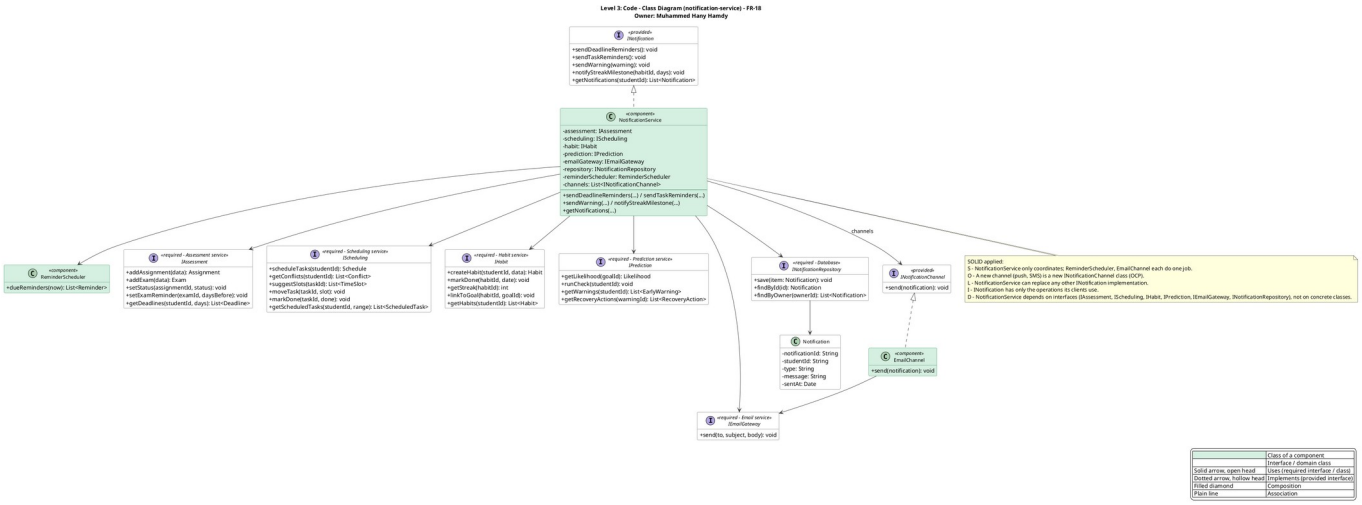
Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `17-dashboard-service.puml`

18. Notification Service — FR-18

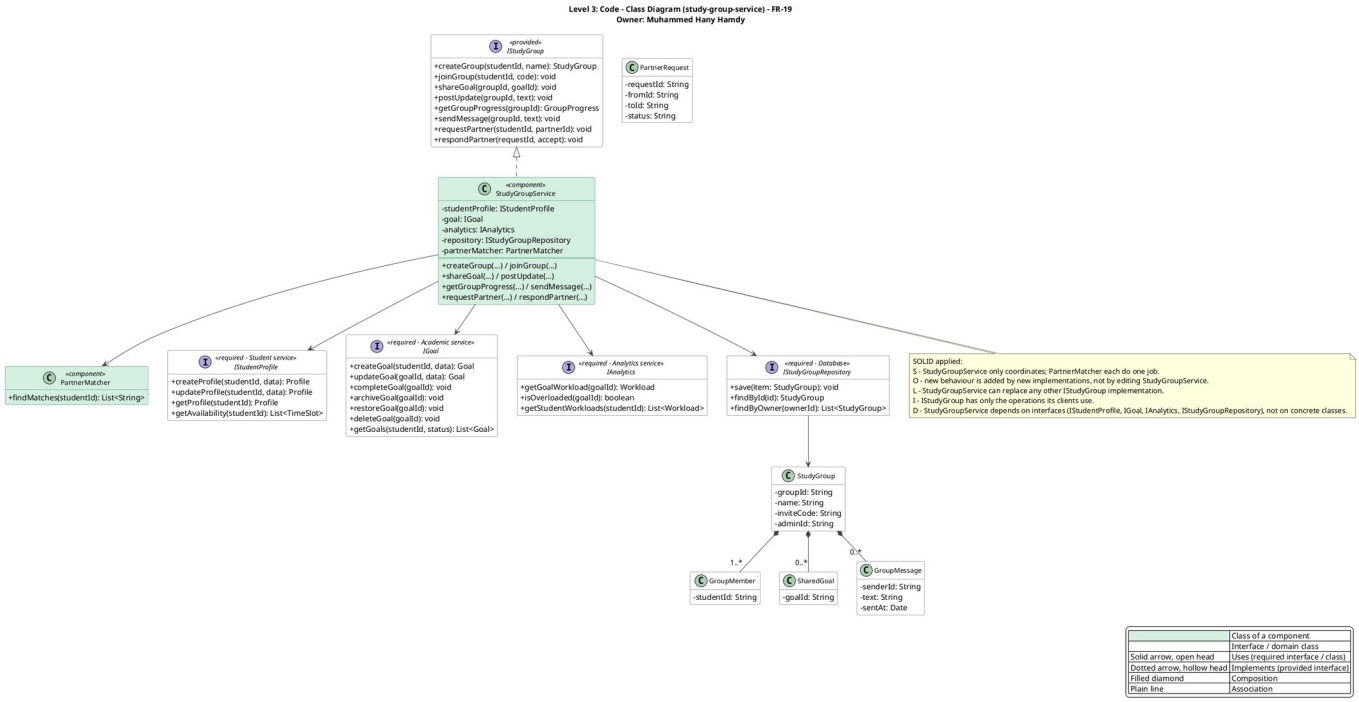
Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `18-notification-service.puml`

19. Study Group Service — FR-19

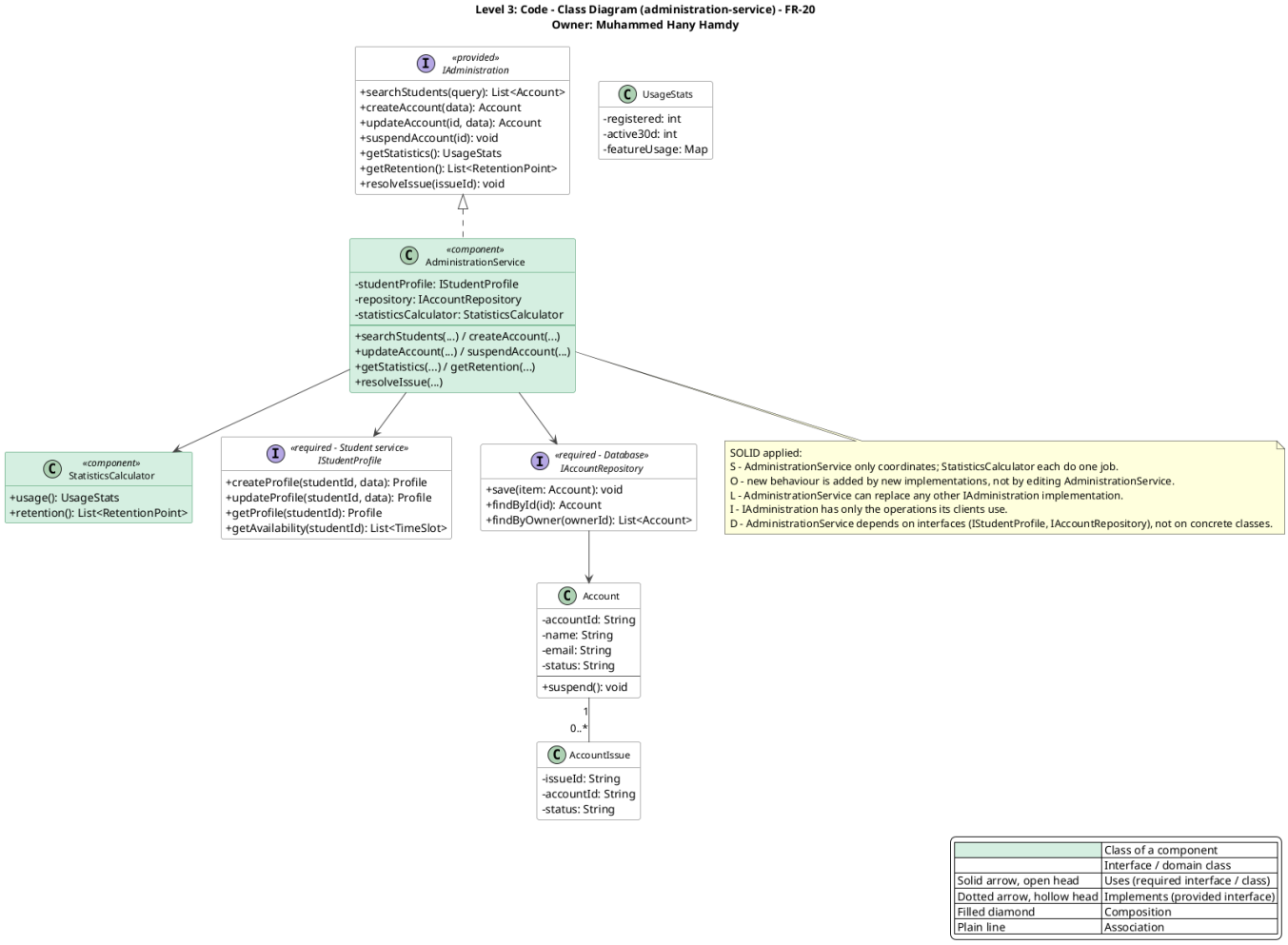
Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



Source: `19-study-group-service.puml`

20. Administration Service — FR-20

Owner: Muhammed Hany Hamdy · **Status:** Design complete; implementation scheduled for a later sprint



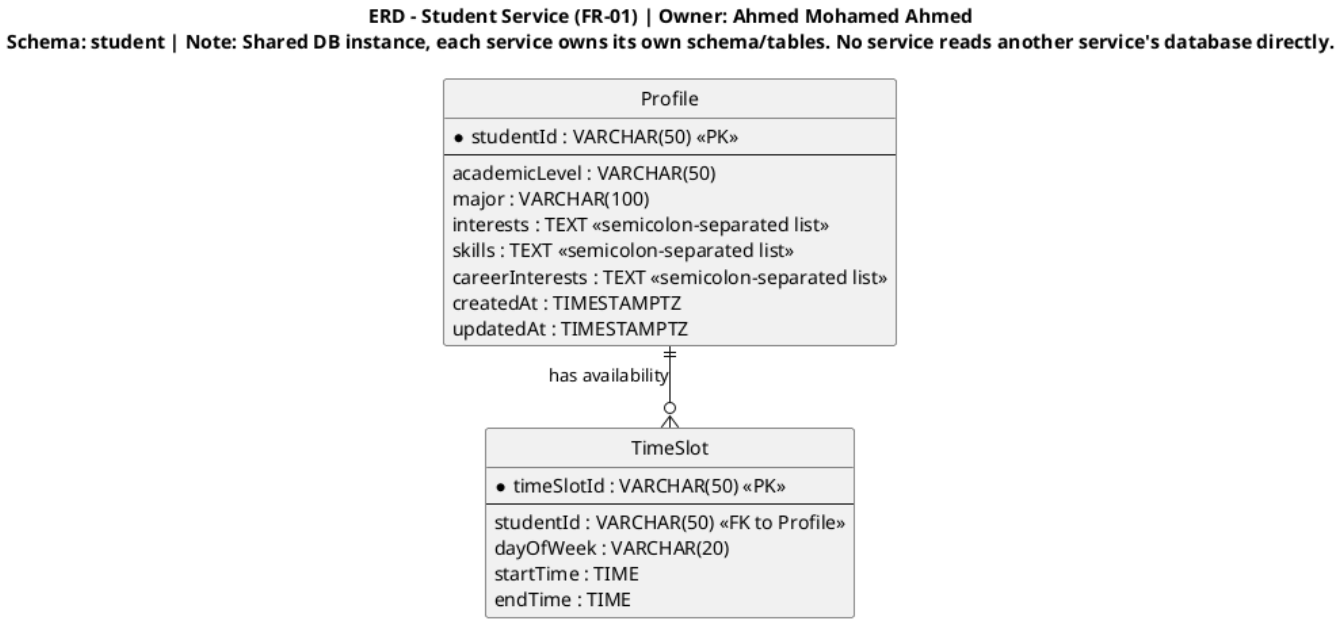
Source: `20-administration-service.puml`

A.6 ERDs

Each service owns its database schema. The ERDs show the tables, primary keys, fields, and relationships. Implemented ERDs match the EF Core migrations. Cross-service identifiers are logical references only; no foreign key crosses a service boundary.

01. Student Service — schema `student`

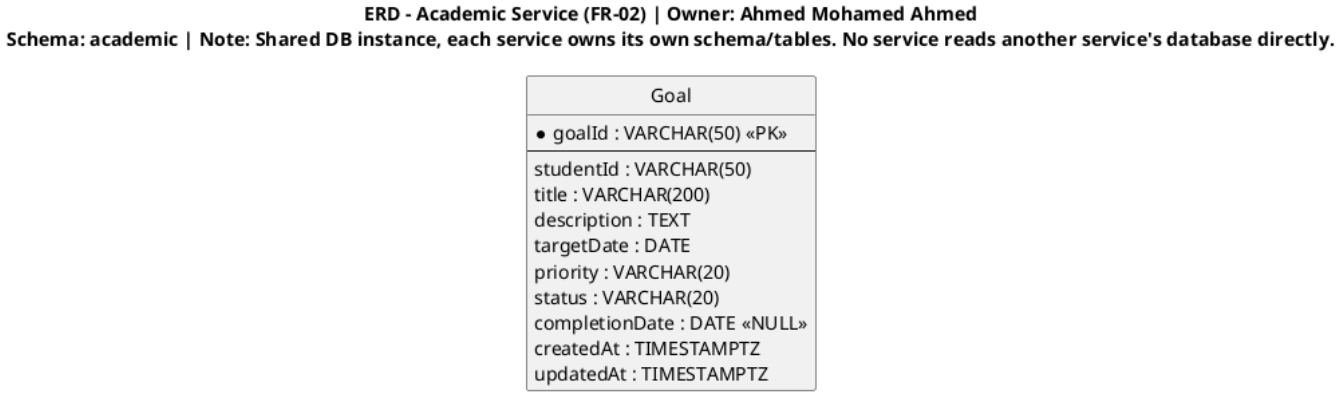
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `01-student-service-erd.puml`

02. Academic Service — schema `academic`

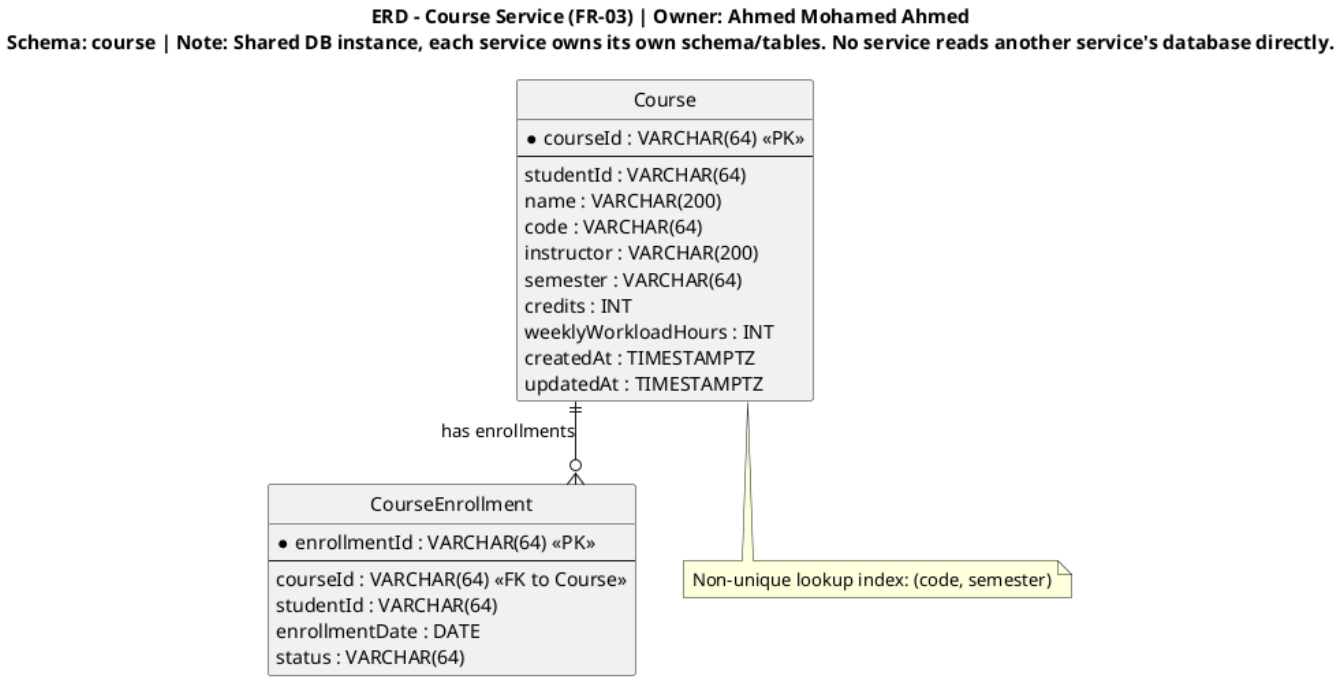
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `02-academic-service-erd.puml`

03. Course Service — schema `course`

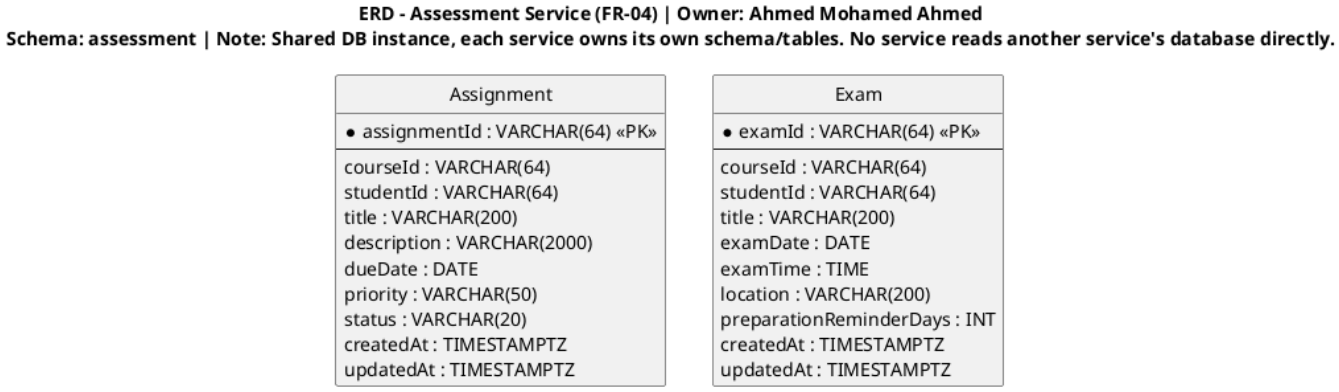
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `03-course-service-erd.puml`

04. Assessment Service — schema `assessment`

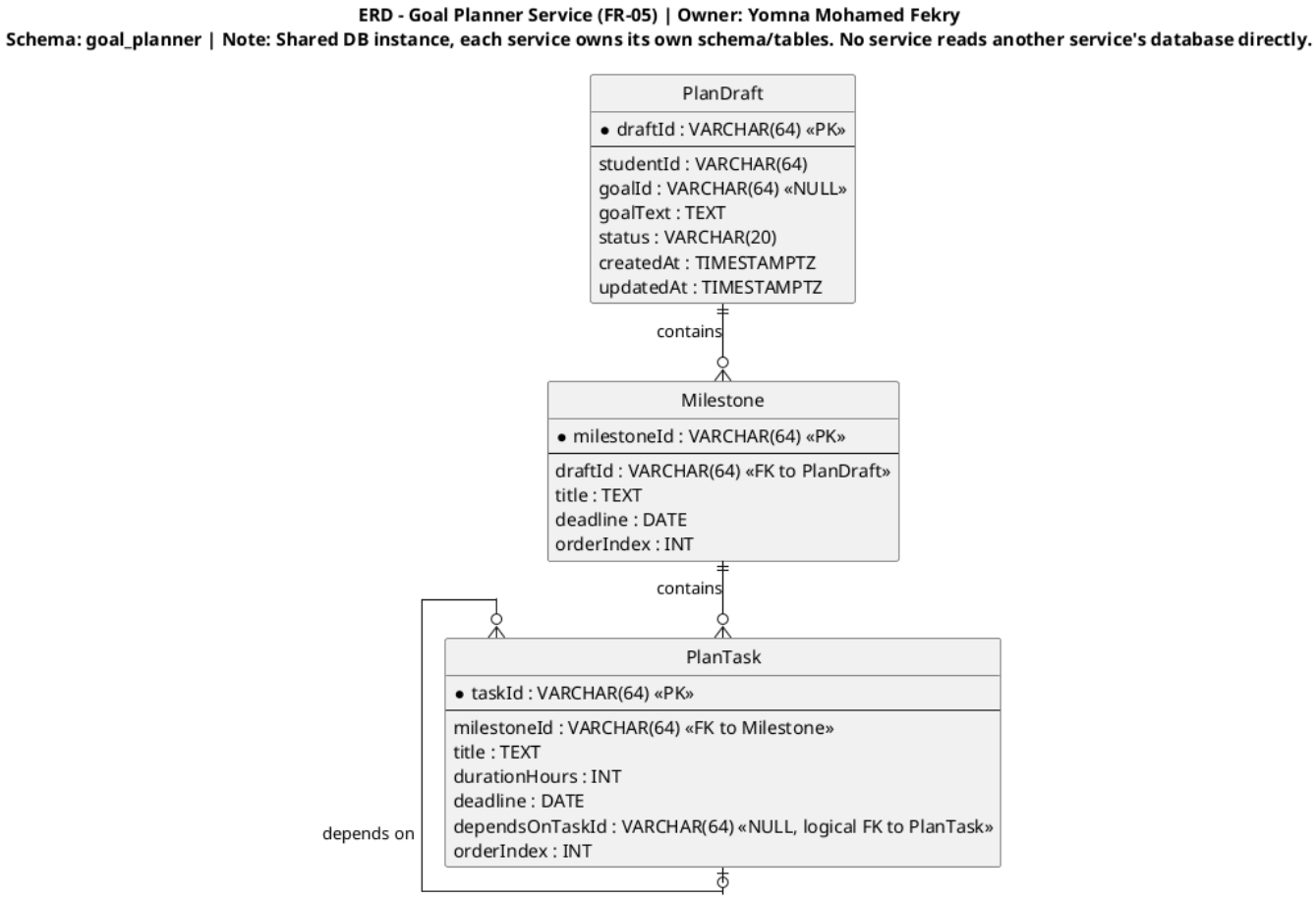
Owner: Ahmed Mohamed Ahmed · **Status:** Implemented in Sprint 1



Source: `04-assessment-service-erd.puml`

05. Goal Planner Service — schema `goal_planner`

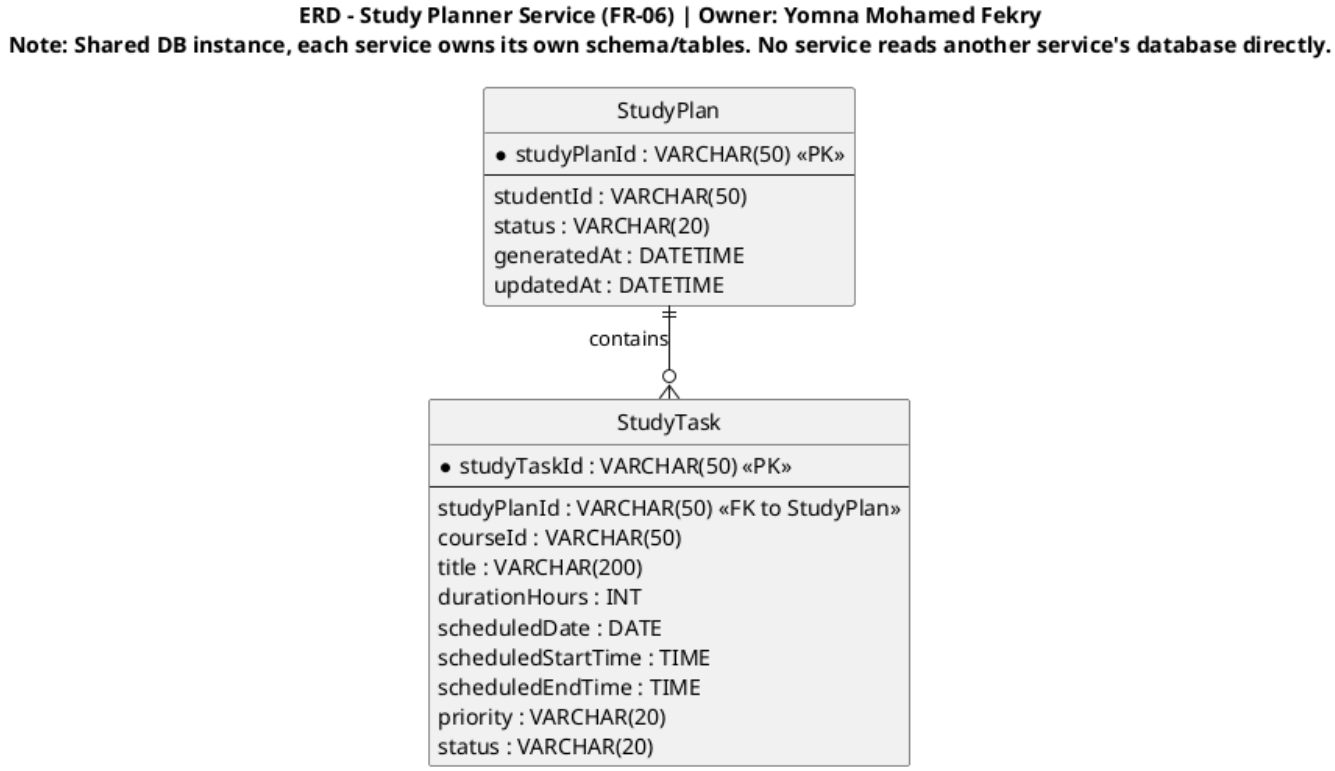
Owner: Yomna Mohamed Fekry · **Status:** Implemented in Sprint 1



Source: `05-goal-planner-service-erd.puml`

06. Study Planner Service — schema `study_planner`

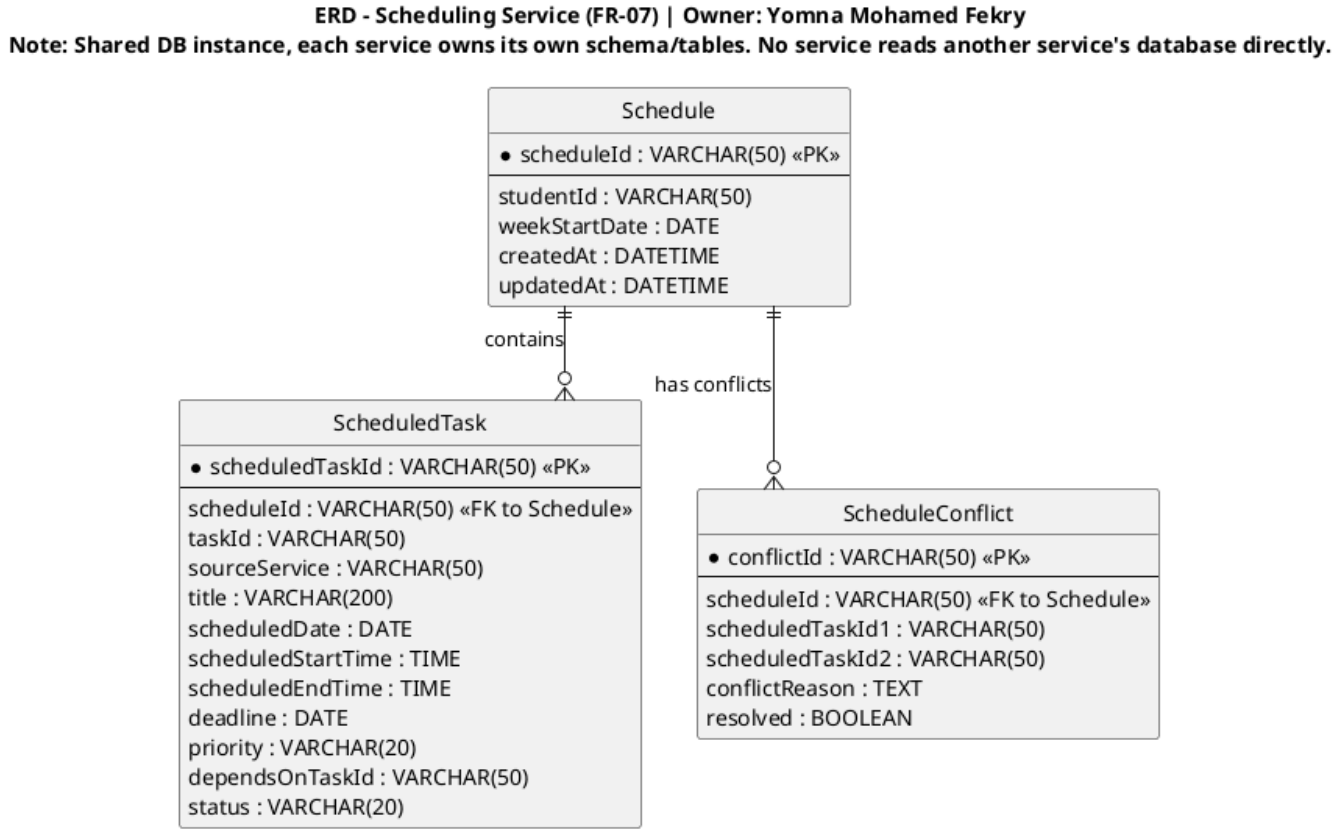
Owner: Yomna Mohamed Fekry · **Status:** Design complete; schema created in a later sprint



Source: `06-study-planner-service-erd.puml`

07. Scheduling Service — schema `scheduling`

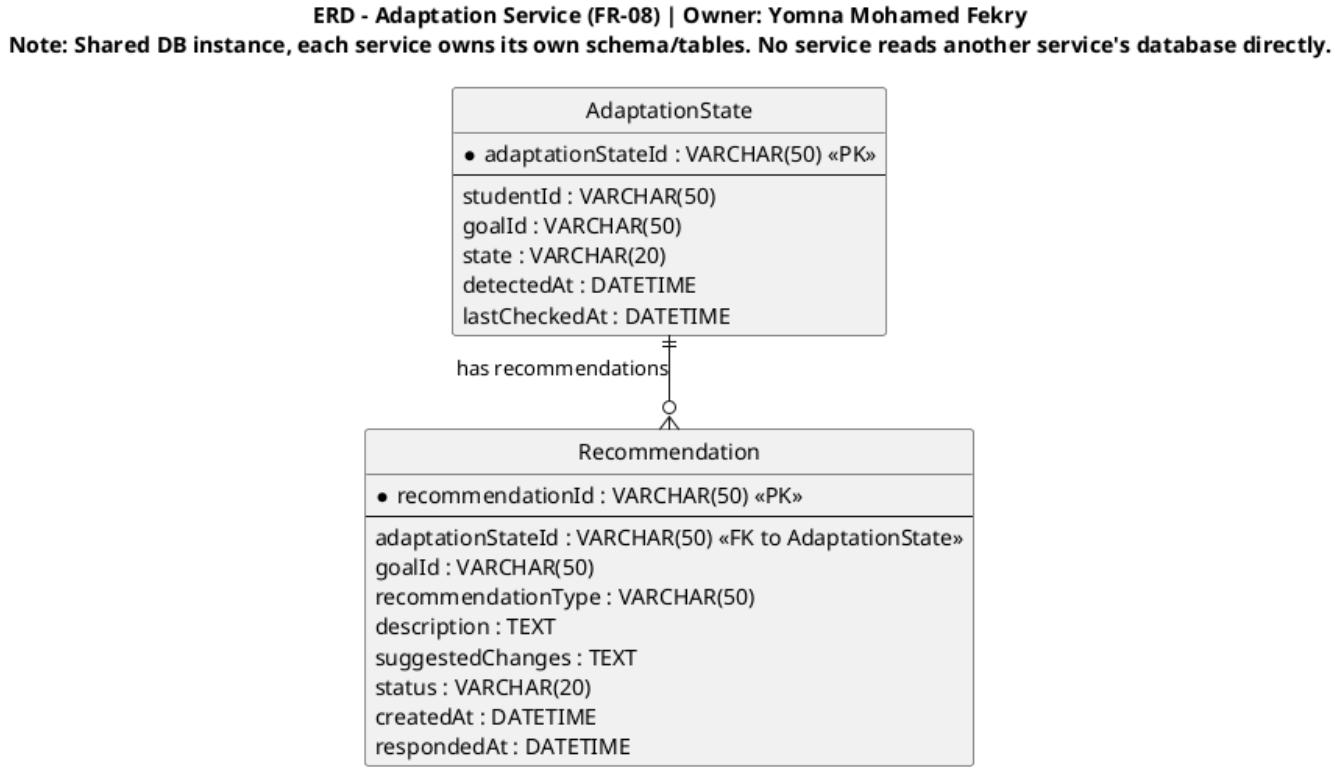
Owner: Yomna Mohamed Fekry · **Status:** Design complete; schema created in a later sprint



Source: `07-scheduling-service-erd.puml`

08. Adaptation Service — schema `adaptation`

Owner: Yomna Mohamed Fekry · **Status:** Design complete; schema created in a later sprint

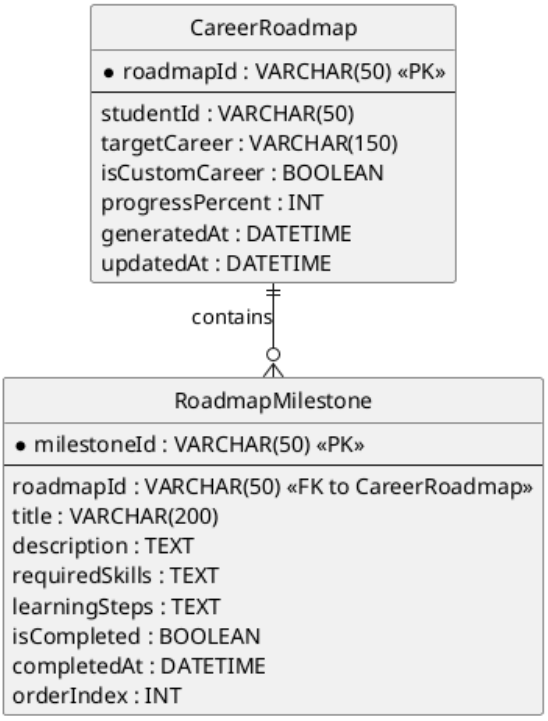


Source: `08-adaptation-service-erd.puml`

09. Career Service — schema `career`

Owner: Arwa Mohey Hamdy · **Status:** Design complete; schema created in a later sprint

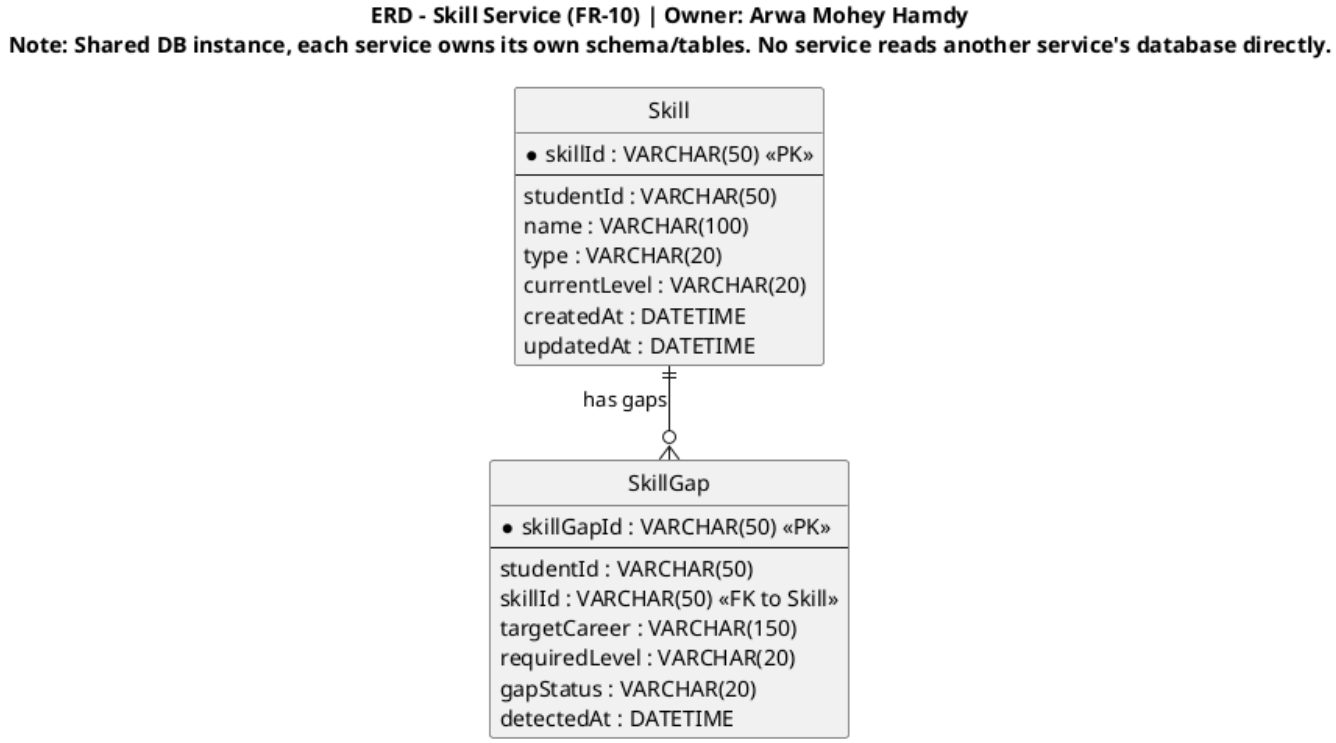
ERD - Career Service (FR-09) | Owner: Arwa Mohey Hamdy
Note: Shared DB instance, each service owns its own schema/tables. No service reads another service's database directly.



Source: `09-career-service-erd.puml`

10. Skill Service — schema `skill`

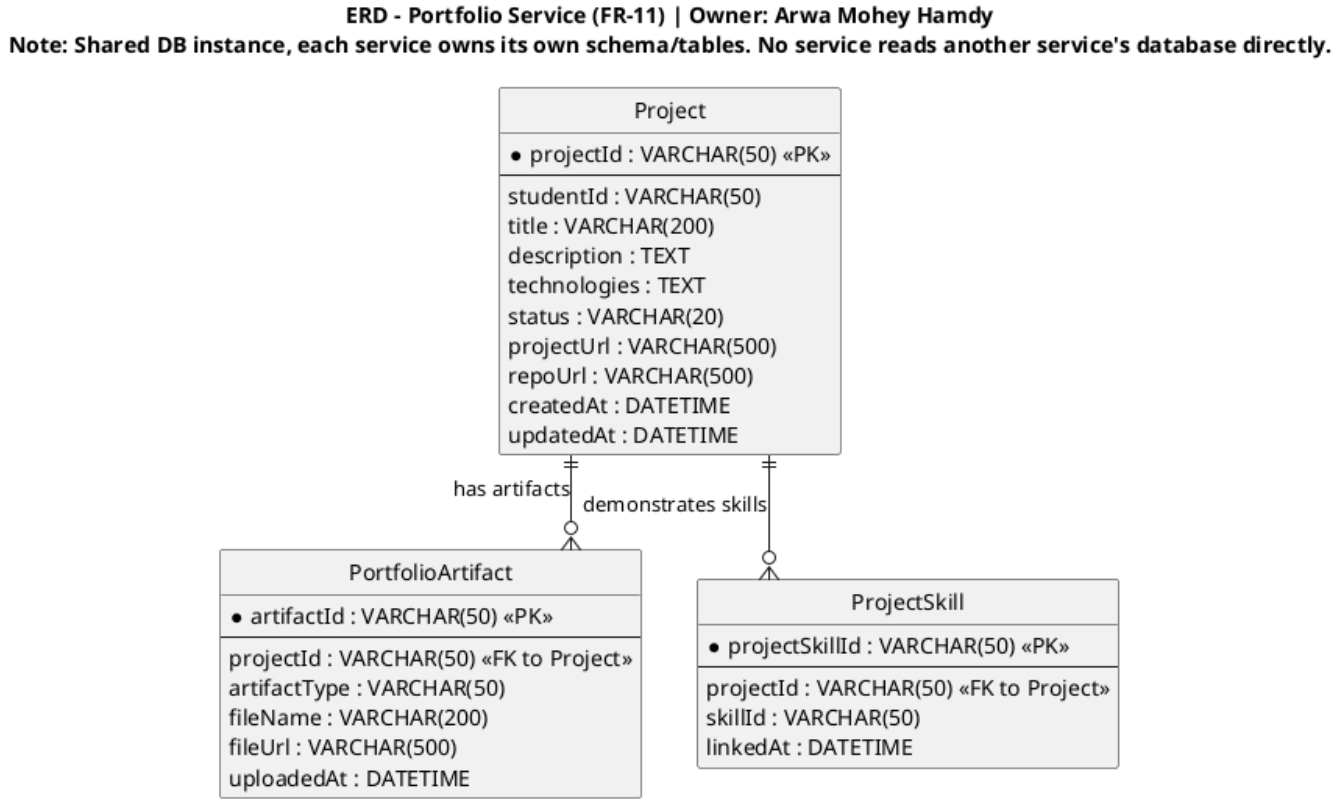
Owner: Arwa Mohey Hamdy · **Status:** Design complete; schema created in a later sprint



Source: `10-skill-service-erd.puml`

11. Portfolio Service — schema `portfolio`

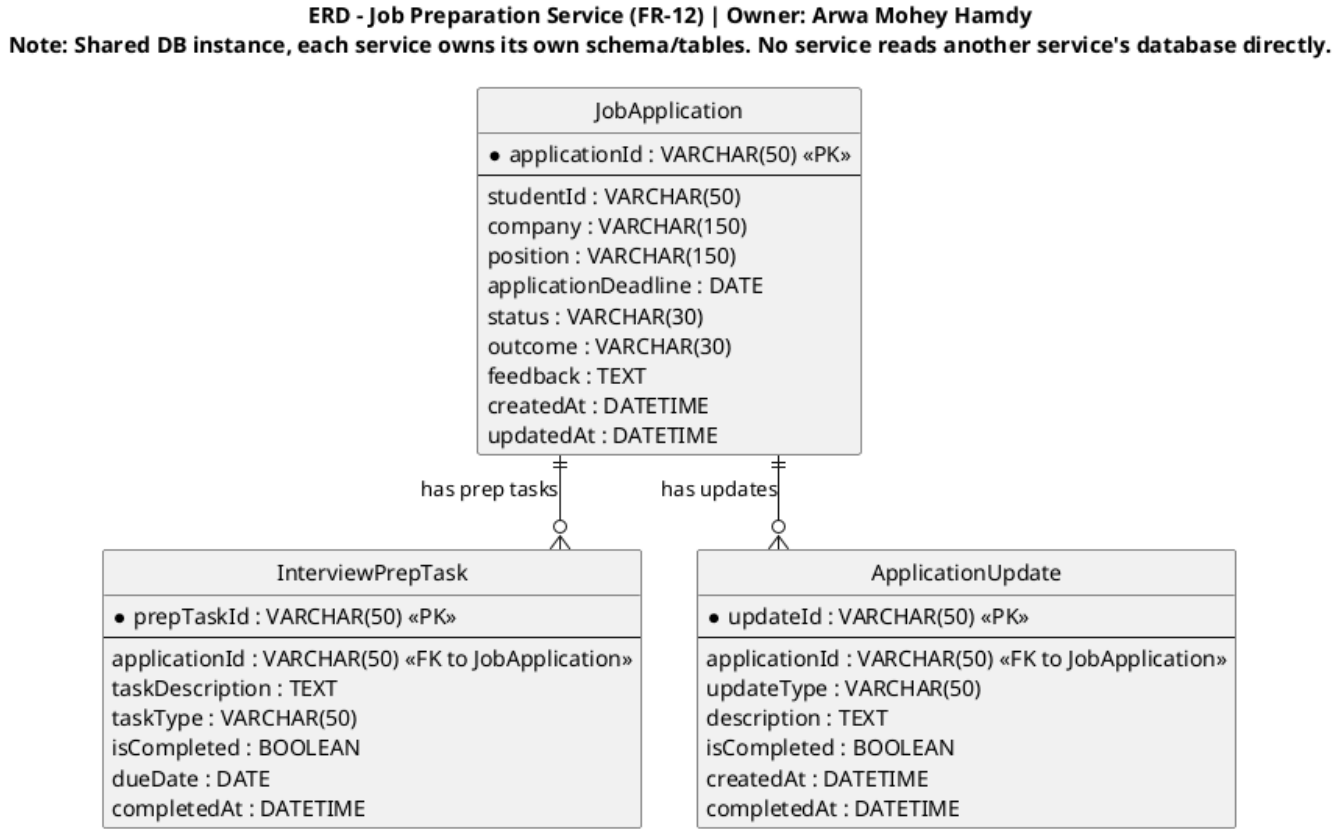
Owner: Arwa Mohey Hamdy · **Status:** Design complete; schema created in a later sprint



Source: `11-portfolio-service-erd.puml`

12. Job Preparation Service — schema `job_preparation`

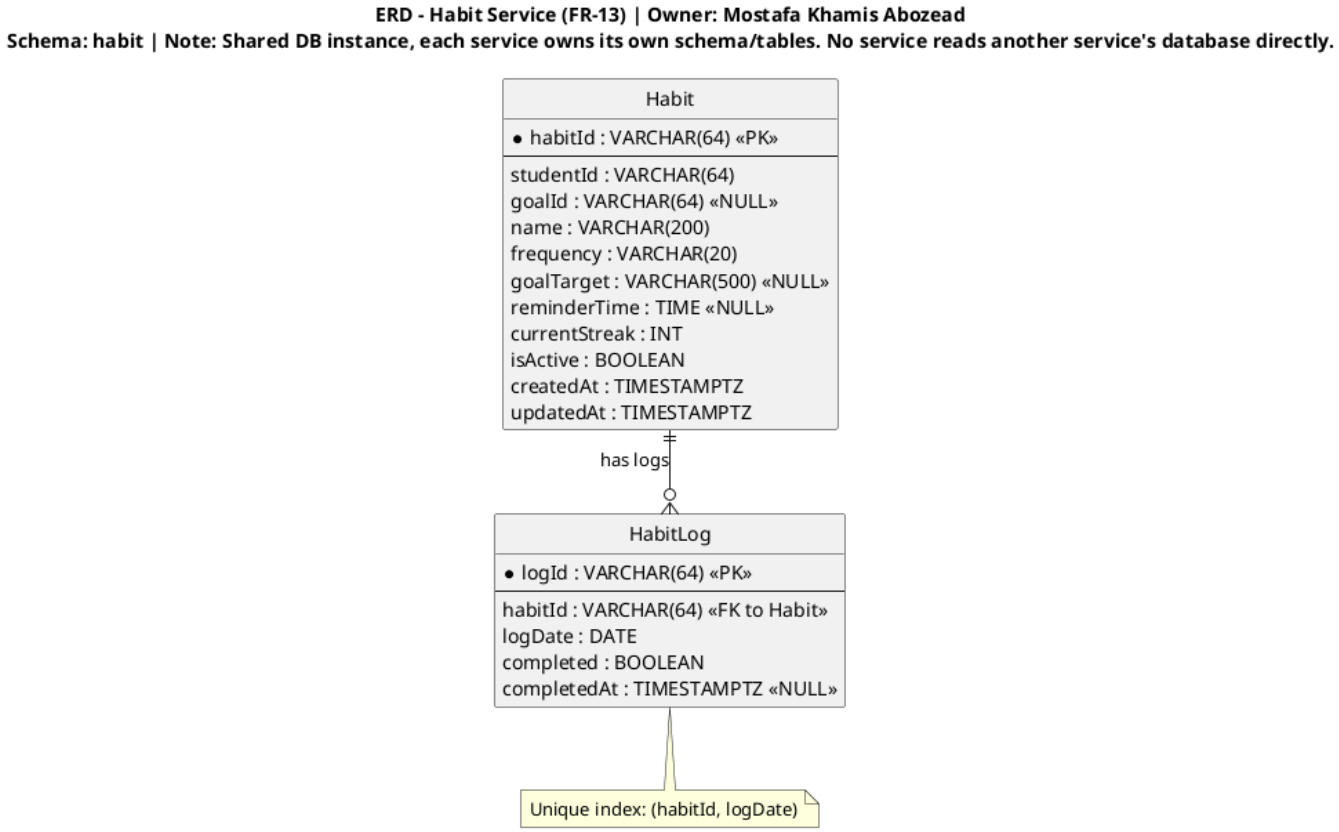
Owner: Arwa Mohey Hamdy · **Status:** Design complete; schema created in a later sprint



Source: `12-job-preparation-service-erd.puml`

13. Habit Service — schema `habit`

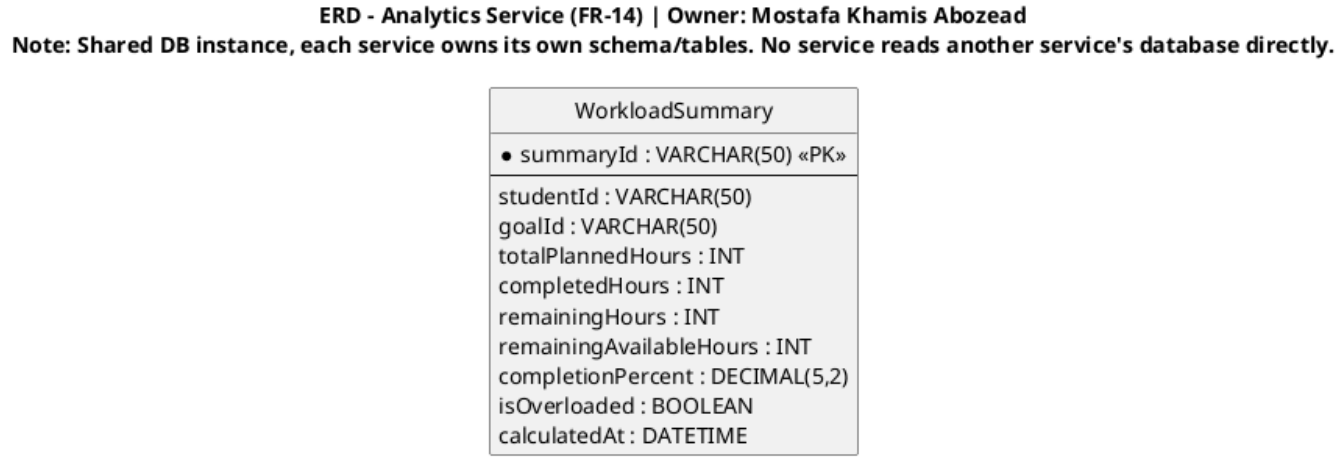
Owner: Mostafa Khamis Abozead · **Status:** Implemented in Sprint 1



Source: `13-habit-service-erd.puml`

14. Analytics Service — schema `analytics`

Owner: Mostafa Khamis Abozead · **Status:** Design complete; schema created in a later sprint

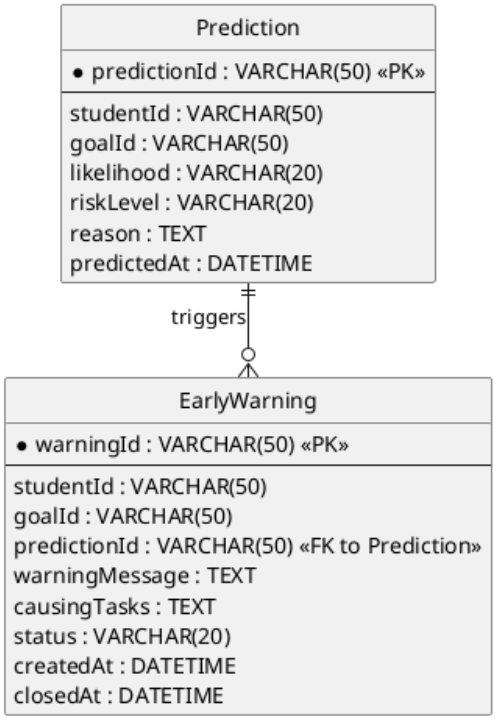


Source: `14-analytics-service-erd.puml`

15. Prediction Service — schema `prediction`

Owner: Mostafa Khamis Abozead · **Status:** Design complete; schema created in a later sprint

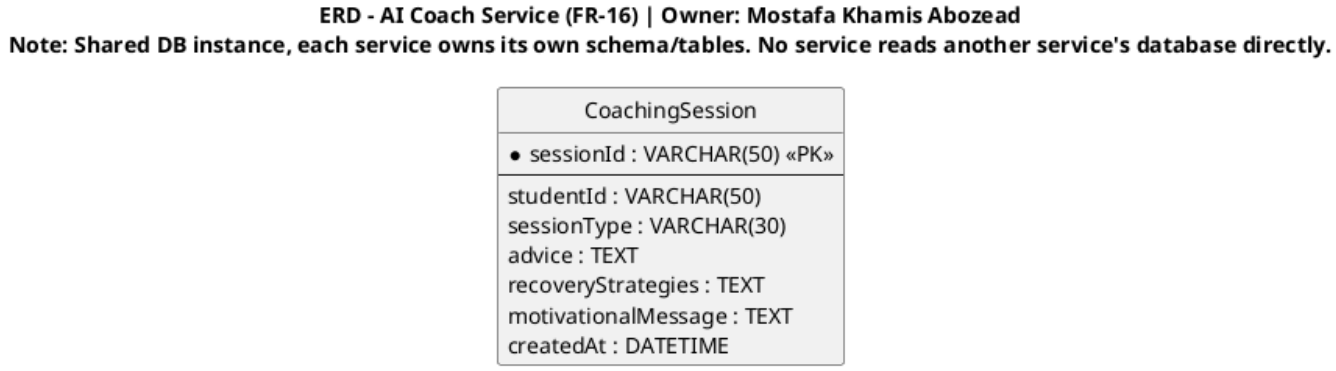
ERD - Prediction Service (FR-15) | Owner: Mostafa Khamis Abozead
Note: Shared DB instance, each service owns its own schema/tables. No service reads another service's database directly.



Source: `15-prediction-service-erd.puml`

16. AI Coach Service — schema `ai_coach`

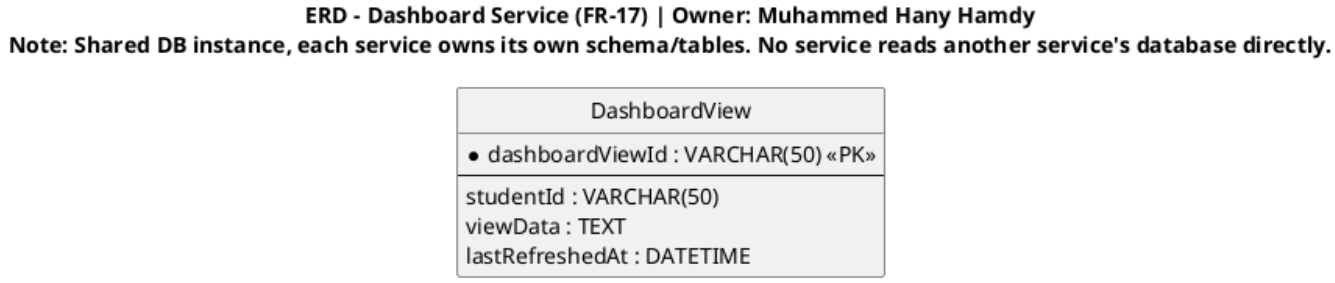
Owner: Mostafa Khamis Abozead · **Status:** Design complete; schema created in a later sprint



Source: `16-ai-coach-service-erd.puml`

17. Dashboard Service — schema `dashboard`

Owner: Muhammed Hany Hamdy · **Status:** Design complete; schema created in a later sprint



Source: `17-dashboard-service-erd.puml`

18. Notification Service — schema `notification`

Owner: Muhammed Hany Hamdy · **Status:** Design complete; schema created in a later sprint

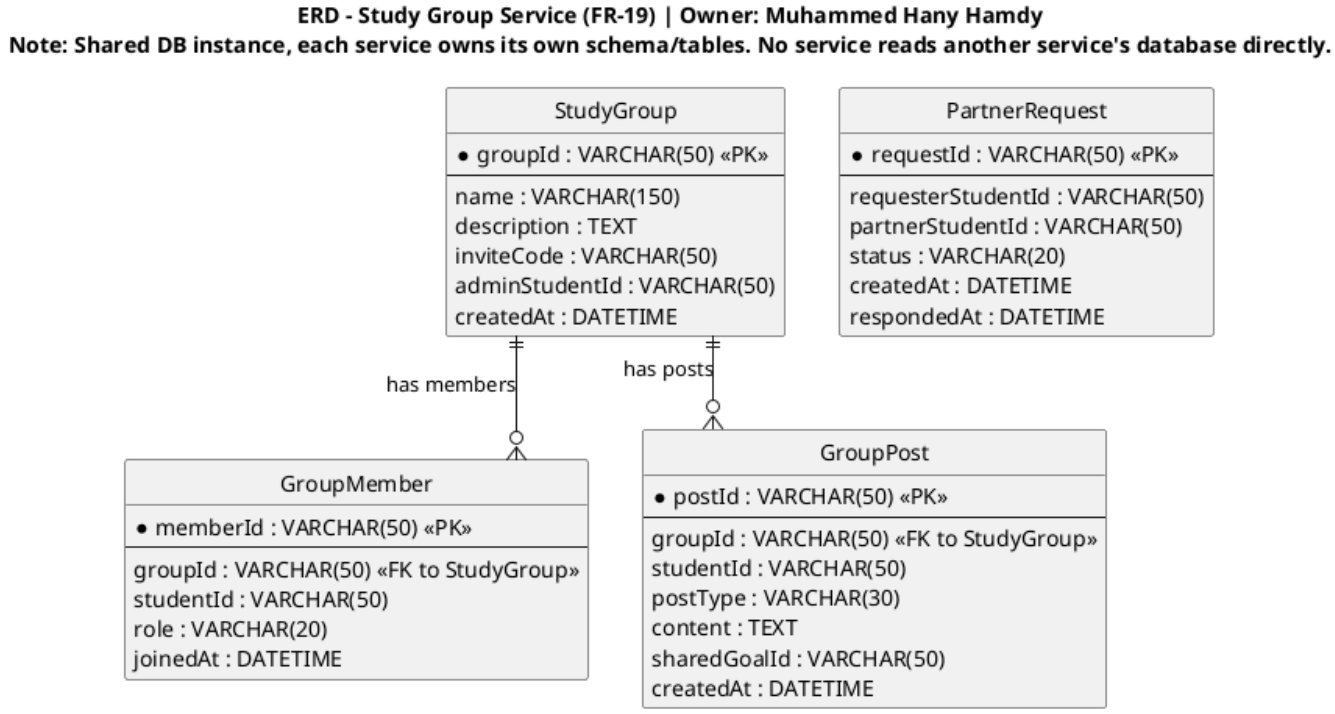
ERD - Notification Service (FR-18) | Owner: Muhammed Hany Hamdy
Note: Shared DB instance, each service owns its own schema/tables. No service reads another service's database directly.



Source: `18-notification-service-erd.puml`

19. Study Group Service — schema `study_group`

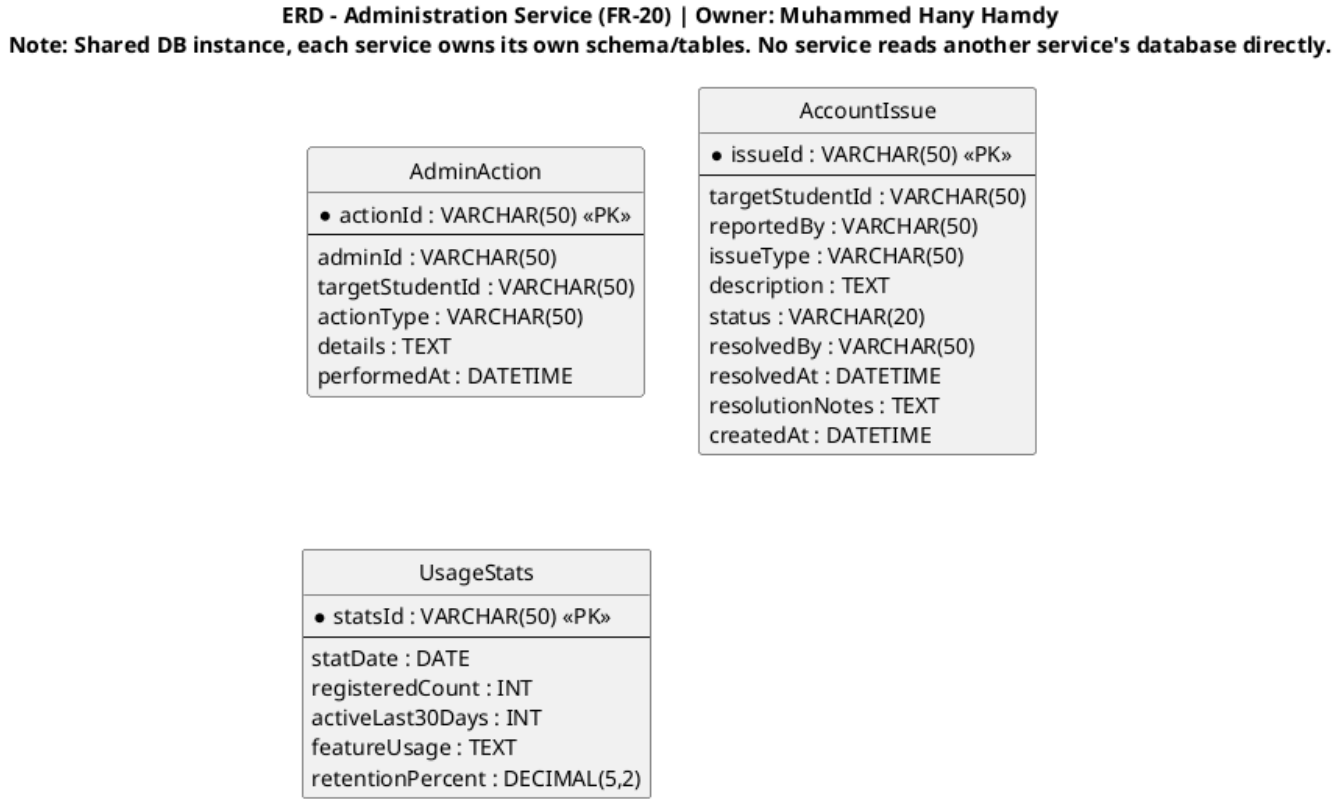
Owner: Muhammed Hany Hamdy · **Status:** Design complete; schema created in a later sprint



Source: `19-study-group-service-erd.puml`

20. Administration Service — schema `administration`

Owner: Muhammed Hany Hamdy · **Status:** Design complete; schema created in a later sprint



Source: `20-administration-service-erd.puml`

---

3. Part B — DevOps

B.1 Docker

Every service has a multi-stage Dockerfile. The six Sprint 1 services use production-ready .NET 10 multi-stage builds; the remaining 14 Dockerfiles are placeholders for their later sprints. The API Gateway also has a multi-stage Dockerfile.

- All service Dockerfiles
- API Gateway Dockerfile
- docker-compose.yml
- .env.example

The implemented stack starts with one command:

```
docker compose up --build -d
```

Service	Host port	Container port	Schema	Status
Student Service	5011	8080	`student`	Implemented
Academic Service	5012	8080	`academic`	Implemented
Course Service	5013	8080	`course`	Implemented
Assessment Service	5014	8080	`assessment`	Implemented
Goal Planner Service	5015	8080	`goal_planner`	Implemented
Habit Service	5016	8080	`habit`	Implemented
PostgreSQL 16	5432	5432	shared instance	Implemented

Each service exposes `GET /health`, returning:

```
{"status":"healthy"}
```

The database is a single PostgreSQL instance. Each service owns a separate schema and its own `__EFMigrationsHistory` table. Cross-service reads use HTTP through the documented interfaces, never direct database access.

B.2 GHCR and GitHub Actions

The GitHub Actions workflow has two jobs:

1. **test** — restores, builds, and runs the full `Aspirra.slnx` test suite in Release mode.
2. **docker** — builds the Sprint 1 service images and pushes them to GHCR on pushes to `develop` and `main`.

Image coordinates:

```
ghcr.io/aiu-softwave/aspirra/student-service:latest
ghcr.io/aiu-softwave/aspirra/academic-service:latest
ghcr.io/aiu-softwave/aspirra/course-service:latest
ghcr.io/aiu-softwave/aspirra/assessment-service:latest
ghcr.io/aiu-softwave/aspirra/goal-planner-service:latest
ghcr.io/aiu-softwave/aspirra/habit-service:latest
```

Each image is also tagged with the commit SHA. The latest successful CI run completed all image builds:

- GitHub Actions run 40673152122
- CI workflow source

---

4. Part C — Project Management

C.1 Sprint Plan

Sprint	Goal	Start	End	State
Sprint 1 — Foundation	Deliver the core academic foundation: profile, goals, courses, assessments, habits, and the first AI goal planner.	2026-10-22	2026-11-04	Closed

Sprint 1 — Stories and Story Points

Jira Key	User Story	Summary	Assignee	Status	Points	FR / Service
SCRUM-104	US-01.1	Create Profile Story	Ahmed Mohamed Ahmed	Done	3	FR-01 / Student
SCRUM-102	US-01.3	View Profile Story	Ahmed Mohamed Ahmed	Done	1	FR-01 / Student
SCRUM-103	US-01.2	Update Profile Story	Ahmed Mohamed Ahmed	Done	2	FR-01 / Student
SCRUM-101	US-02.1	Create Goal Story	Ahmed Mohamed Ahmed	Done	3	FR-02 / Academic
SCRUM-105	US-02.2	Edit Goal Story	Ahmed Mohamed Ahmed	Done	2	FR-02 / Academic
SCRUM-106	US-02.3	Complete Goal Story	Ahmed Mohamed Ahmed	Done	1	FR-02 / Academic
SCRUM-108	US-03.1	Add Course Story	Ahmed Mohamed Ahmed	Done	3	FR-03 / Course
SCRUM-109	US-03.2	View Course Progress Story	Ahmed Mohamed Ahmed	Done	3	FR-03 / Course
SCRUM-113	US-04.1	Add Assignment Story	Ahmed Mohamed Ahmed	Done	3	FR-04 / Assessment
SCRUM-114	US-04.2	Add Exam Story	Ahmed Mohamed Ahmed	Done	2	FR-04 / Assessment
SCRUM-115	US-04.3	Track Assignment Status Story	Ahmed Mohamed Ahmed	Done	2	FR-04 / Assessment
SCRUM-117	US-05.1	Generate Goal Plan Story	Yomna Mohamed Fekry	Done	8	FR-05 / Goal Planner
SCRUM-118	US-05.2	Review and Edit AI Plan Story	Yomna Mohamed Fekry	Done	5	FR-05 / Goal Planner
SCRUM-119	US-05.3	Approve AI Plan Story	Yomna Mohamed Fekry	Done	2	FR-05 / Goal Planner
SCRUM-143	US-13.1	Create Habit Story	Mostafa Khamis Abozead	Done	3	FR-13 / Habit
SCRUM-144	US-13.2	Complete Habit and Keep Streak Story	Mostafa Khamis Abozead	Done	3	FR-13 / Habit

Total stories: 16
Total story points: 46

C.2 Velocity

Sprint	Planned velocity	Actual velocity	Estimation error
Sprint 1 — Foundation (closed)	46 SP	46 SP	0%

C.3 Sprint Burndown

Day	Guideline remaining (SP)	Actual remaining (SP)	Notes
Day 1 (2026-10-22)	46	46	Sprint started; all stories in sprint with points
Mid-sprint	~23	~23	Work in progress
End (2026-11-04)	0	0	All stories completed; sprint closed

The Jira burndown chart for Sprint 1 is available from the board reports:

- Jira SCRUM board
- Jira project

C.4 Jira Configuration

Field	Value
Site	https://aspirra.atlassian.net
Project	Aspirra Project (`SCRUM`)
Board	SCRUM board, board ID 1
Issue type	Task (stories labeled `story`)
Epics	`SCRUM-10`–`SCRUM-27`, one per FR-01–FR-20
Workflow	Backlog → Ready → In Progress → In Review → Done
Sprint 1	Closed; 16 stories; 46 story points

---

5. Part D — Traceability

Every diagram maps to a real artifact: code, configuration, Jira, or the database.

D.1 Architecture Diagrams

Diagram	Artifact(s)	Notes
System Context	`README.md`, Jira project `SCRUM`	Actors, Aspirra box, provided interface list, external systems, required interfaces `IAIProvider` / `IEmailGateway`
Interfaces	`docs/proposal/04-B4-functional-requirements.md`, Jira epics/stories, `src/Services/*/Interfaces/`	High-level and low-level FRs per interface; every required operation annotated with `(used by: FR-XX.Y)`
Container	`docker-compose.yml`, `src/Services/*`, `src/ApiGateway/`, EF Core migrations	One container per service; provided/required interfaces; shared PostgreSQL with per-service schemas
Components	`src/Services/*/`, `src/ApiGateway/`	Per-service internal components, interfaces, and ownership
System Class	All services in `src/Services/`	SOLID applied; cross-service dependencies through interfaces
Service Classes	Corresponding `src/Services/<Service>/`	Implemented services reflect the real classes and methods
ERDs	`src/Services/*/Data/*DbContext.cs`, `Models/`, `Data/Migrations/`	Per-service schemas; EF Core migrations; no cross-service database reads

D.2 Sprint 1 Code Mapping

User Story (Jira)	Service	Code artifacts	Design artifacts
US-01.1/01.2/01.3 (SCRUM-104/103/102)	Student Service	ProfilesController, IStudentProfile, StudentProfileService, ProfileRepository, StudentDbContext, Migrations, tests	Component `01`, Class `01`, ERD `01`
US-02.1/02.2/02.3 (SCRUM-101/105/106)	Academic Service	GoalsController, IGoal, GoalService, GoalRepository, AcademicDbContext, Migrations, tests	Component `02`, Class `02`, ERD `02`
US-03.1/03.2 (SCRUM-108/109)	Course Service	CoursesController, ICourse, IAssessmentClient, CourseService, HttpAssessmentClient, CourseRepository, CourseDbContext, Migrations, tests	Component `03`, Class `03`, ERD `03`
US-	Assessment	Controllers, IAssessment, AssessmentService,	Component `04`,

User Story (Jira)	Service	Code artifacts	Design artifacts
04.1/04.2/04.3 (SCRUM-113/114/115)	Service	EfAssessmentRepository, AssessmentDbContext, Migrations, tests	Class `04`, ERD `04`
US-05.1/05.2/05.3 (SCRUM-117/118/119)	Goal Planner Service	PlansController, IGoalPlanner, GoalPlannerService, MockAIProvider, HttpGoalClient, PlanDraftRepository, GoalPlannerDbContext, Migrations, tests	Component `05`, Class `05`, ERD `05`
US-13.1/13.2 (SCRUM-143/144)	Habit Service	HabitsController, IHabit, IHabitRepository, HabitService, HabitRepository, HabitDbContext, Migrations, tests	Component `13`, Class `13`, ERD `13`

D.3 DevOps and Project Management

Artifact	Maps to	Notes
Multi-stage Dockerfiles	`src/Services/*/Dockerfile`, `src/ApiGateway/Dockerfile`	One per service/container; Sprint 1 services production-ready, 14 placeholders
Docker Compose	`docker-compose.yml`	Six Sprint 1 services + PostgreSQL; remaining services commented until their sprints
Environment	`.env.example`	Database, AI provider, and email gateway settings
GitHub Actions	`.github/workflows/ci.yml`	Test, build, and push images to GHCR
Jira	Project `SCRUM`, board 1	Epics per FR, stories labeled `story`, Sprint 1 closed
Backlog and estimation	`docs/proposal/08-C8-product-backlog.md`, `docs/proposal/09-C9-estimation-and-sprint-plan.md`	US-XX.Y, story points, sprint plan

---

6. Build and Verification

Check	Result
`dotnet build Aspirra.slnx -c Release`	Passed
`dotnet test Aspirra.slnx -c Release`	74 passed, 0 failed, 0 skipped
Student Service tests	14 passed
Academic Service tests	16 passed
Course Service tests	10 passed
Assessment Service tests	11 passed
Goal Planner Service tests	13 passed
Habit Service tests	10 passed
Docker Compose stack	6 services + PostgreSQL healthy
Health checks	`GET /health` returns `{"status":"healthy"}` on all six services
Rendered diagrams	63 service/architecture PNGs
CI	Latest GitHub Actions run successful

Commands:

```
dotnet test Aspirra.slnx -c Release
docker compose up --build -d
docker compose ps
```

Implemented service endpoints:

```
GET http://localhost:5011/health   # Student
GET http://localhost:5012/health   # Academic
GET http://localhost:5013/health   # Course
GET http://localhost:5014/health   # Assessment
GET http://localhost:5015/health   # Goal Planner
GET http://localhost:5016/health   # Habit
```

---

7. Team Ownership

Member	GitHub	Owned services	Sprint 1 contribution
Ahmed Mohamed Ahmed (Team Leader)	`AbooSalh`	Student, Academic, Course, Assessment	11 stories, 25 SP; four implemented services
Yomna Mohamed Fekry	`YomnaElmenawy`	Goal Planner, Study Planner, Scheduling, Adaptation	3 stories, 15 SP; Goal Planner implemented
Arwa Mohey Hamdy	`ArwaMohey`	Career, Skill, Portfolio, Job Preparation	Design artefacts for four services
Mostafa Khamis Abozead	`Mostafa-abozead`	Habit, Analytics, Prediction, AI Coach	2 stories, 6 SP; Habit implemented
Muhammed Hany Hamdy	`muhammedhanii`	Dashboard, Notification, Study Group, Administration	Design artefacts for four services

Appendix A — Full Interfaces Specification

Reproduced from `Interfaces.md` for submission convenience.

Provided Interfaces (exposed by Aspirra service containers)

IStudentProfile — Student Service — Owner: Ahmed Mohamed Ahmed

IStudentProfile — high level: - Manage Student Profile (FR-01)

IStudentProfile — low level: Manage Student Profile: - `createProfile(studentId, data)` (FR-01.1, FR-01.4) - `updateProfile(studentId, data)` (FR-01.2, FR-01.4) - `getProfile(studentId)` (FR-01.3) - `getAvailability(studentId)` (FR-01.3)

IGoal — Academic Service — Owner: Ahmed Mohamed Ahmed

IGoal — high level: - Manage Academic Goals (FR-02)

IGoal — low level: Manage Academic Goals: - `createGoal(studentId, data)` (FR-02.1) - `updateGoal(goalId, data)` (FR-02.2, FR-02.3) - `completeGoal(goalId)` (FR-02.4) - `archiveGoal(goalId)` (FR-02.5) - `restoreGoal(goalId)` (FR-02.5) - `deleteGoal(goalId)` (FR-02.6) - `getGoal(goalId)` / `getGoals(studentId, status)` (FR-02.2, FR-02.4)

ICourse — Course Service — Owner: Ahmed Mohamed Ahmed

ICourse — high level: - Manage Courses (FR-03)

ICourse — low level: Manage Courses: - `addCourse(studentId, data)` (FR-03.1, FR-03.2) - `updateCourse(courseId, data)` (FR-03.4) - `removeCourse(courseId)` (FR-03.5) - `getCourse(courseId)` / `getCourses(studentId)` (FR-03.4) | Track Course Progress: - `getCourseProgress(courseId)` (FR-03.3)

IAssessment — Assessment Service — Owner: Ahmed Mohamed Ahmed

IAssessment — high level: - Manage Assignments and Exams (FR-04)

IAssessment — low level: Manage Assignments and Exams: - `addAssignment(data)` (FR-04.1) - `addExam(data)` (FR-04.2) | Track Assignment Status: - `setAssignmentStatus(assignmentId, status)` (FR-04.3) - `getAssessments(studentId)` / `getCourseAssignments(courseId)` (FR-04.3) | Manage Exam Reminders: - `setExamReminder(examId, daysBefore)` (FR-04.4) - `getUpcomingExams(studentId, days)` (FR-04.4)

IGoalPlanner — Goal Planner Service — Owner: Yomna Mohamed Fekry

IGoalPlanner — high level: - Plan Goals with AI (FR-05)

IGoalPlanner — low level: Plan Goals with AI: - `generatePlan(studentId, goalText)` (FR-05.1, FR-05.2) - `updateDraft(draftId, edits)` (FR-05.3, FR-05.4, FR-05.5) - `approvePlan(draftId)` (FR-05.6) - `discardPlan(draftId)` (FR-05.6) - `getDraft(draftId)` (FR-05.3) - `getTasks(goalId)` (FR-05.2)

IStudyPlanner — Study Planner Service — Owner: Yomna Mohamed Fekry

IStudyPlanner — high level: - Generate Study Plans (FR-06)

IStudyPlanner — low level: Generate Study Plans: - `generateStudyPlan(studentId, courses, availability)` (FR-06.1, FR-06.2) - `regenerateStudyPlan(studentId)` (FR-06.3) - `validateStudyPlan(plan)` (FR-06.4) - `retainPreviousPlan(studentId)` (FR-06.5)

IScheduling — Scheduling Service — Owner: Yomna Mohamed Fekry

IScheduling — high level: - Enable Smart Scheduling (FR-07)

IScheduling — low level: Enable Smart Scheduling: - `scheduleTasks(studentId, tasks)` (FR-07.1) - `detectConflicts(schedule)` (FR-07.2) - `suggestAlternatives(conflict)` (FR-07.3) - `adjustScheduledTask(taskId, newSlot)` (FR-07.4) - `markTaskDone(taskId, completed)` (FR-07.5) - `getSchedule(studentId, range)` (FR-07.1)

IAdaptation — Adaptation Service — Owner: Yomna Mohamed Fekry

IAdaptation — high level: - Enable Adaptive Planning (FR-08)

IAdaptation — low level: Enable Adaptive Planning: - `detectFallingBehind(studentId)` (FR-08.1) - `recommendRescheduling(studentId)` (FR-08.2) - `recommendReprioritization(studentId)` (FR-08.3) - `approveRecommendation(recommendationId)` (FR-08.4) - `acceptRecommendation(recommendationId)` (FR-08.5) - `rejectRecommendation(recommendationId)` (FR-08.6) - `modifyRecommendation(recommendationId, changes)` (FR-08.7)

ICareer — Career Service — Owner: Arwa Mohey Hamdy

ICareer — high level: - Manage Career Roadmaps (FR-09)

ICareer — low level: Manage Career Roadmaps: - `selectTargetCareer(studentId, career)` (FR-09.1, FR-09.2) - `generateRoadmap(studentId, targetCareer)` (FR-09.3) - `trackRoadmapProgress(studentId)` (FR-09.4) - `updateTargetCareer(studentId, career)` (FR-09.5) - `regenerateRoadmap(studentId)` (FR-09.6)

ISkill — Skill Service — Owner: Arwa Mohey Hamdy

ISkill — high level: - Track Skills (FR-10)

ISkill — low level: Track Skills: - `addSkill(studentId, skill)` (FR-10.1) - `compareSkills(studentId)` (FR-10.2) - `getSkillGaps(studentId)` (FR-10.3) - `updateSkillLevel(skillId, level)` (FR-10.4)

IPortfolio — Portfolio Service — Owner: Arwa Mohey Hamdy

IPortfolio — high level: - Manage Projects and Portfolios (FR-11)

IPortfolio — low level: Manage Projects and Portfolios: - `addProject(studentId, project)` (FR-11.1) - `linkProjectToSkills(projectId, skillIds)` (FR-11.2) - `viewPortfolio(studentId)` (FR-11.3) - `addProjectArtifacts(projectId, links, documentation)` (FR-11.4)

IJobPrep — Job Preparation Service — Owner: Arwa Mohey Hamdy

IJobPrep — high level: - Manage Internship and Job Preparation (FR-12)

IJobPrep — low level: Manage Internship and Job Preparation: - `trackApplication(studentId, application)` (FR-12.1) - `trackInterviewPrep(applicationId)` (FR-12.2) - `trackCvUpdates(applicationId)` (FR-12.3) - `trackPortfolioUpdates(applicationId)` (FR-12.4) - `recordOutcome(applicationId, outcome)` (FR-12.5) - `recordFeedback(applicationId, feedback)` (FR-12.6)

IHabit — Habit Service — Owner: Mostafa Khamis Abozead

IHabit — high level: - Manage Habits (FR-13)

IHabit — low level: Manage Habits: - `createHabit(studentId, data)` (FR-13.1) - `completeHabit(habitId, date)` (FR-13.2) - `getStreaks(studentId)` (FR-13.3) - `linkHabitToGoal(habitId, goalId)` (FR-13.4) - `getHabits(studentId)` (FR-13.1)

IAnalytics — Analytics Service — Owner: Mostafa Khamis Abozead

IAnalytics — high level: - Analyze Study Workload (FR-14)

IAnalytics — low level: Analyze Study Workload: - `calculateRemainingWorkload(studentId)` (FR-14.1) - `calculateAvailableStudyTime(studentId)` (FR-14.2) - `calculateCompletionPercentage(studentId)` (FR-14.3) - `getWorkloadSummary(studentId)` (FR-14.4) - `detectOverloadedGoal(studentId)` (FR-14.5)

IPrediction — Prediction Service — Owner: Mostafa Khamis Abozead

IPrediction — high level: - Predict and Warn Early (FR-15)

IPrediction — low level: Predict and Warn Early: - `estimateCompletionLikelihood(studentId, goalId)` (FR-15.1) - `generateEarlyWarning(goalId)` (FR-15.2) - `identifyRiskTasks(goalId)` (FR-15.3) - `suggestRecoveryActions(goalId)` (FR-15.4)

ICoach — AI Coach Service — Owner: Mostafa Khamis Abozead

ICoach — high level: - Coach Students with AI (FR-16)

ICoach — low level: Coach Students with AI: - `getPersonalizedRecommendations(studentId)` (FR-16.1) - `getStudyAdvice(studentId)` (FR-16.2) - `getRecoveryStrategy(studentId)` (FR-16.3) - `getMotivationalGuidance(studentId)` (FR-16.4)

IDashboard — Dashboard Service — Owner: Muhammed Hany Hamdy

IDashboard — high level: - Display Student Dashboard (FR-17)

IDashboard — low level: Display Student Dashboard: - `getTodayAndDeadlines(studentId)` (FR-17.1) - `getGoalStatus(studentId)` (FR-17.2) - `getGoalCompletion(studentId)` (FR-17.3) - `getHabitStreaks(studentId)` (FR-17.4) - `getWorkloadSummary(studentId)` (FR-17.5) - `getCareerProgress(studentId)` (FR-17.6)

INotification — Notification Service — Owner: Muhammed Hany Hamdy

INotification — high level: - Send Notifications (FR-18)

INotification — low level: Send Notifications: - `sendDeadlineReminder(studentId, deadline)` (FR-18.1, FR-18.5) - `sendTaskReminder(studentId, task)` (FR-18.2) - `sendHabitReminder(studentId, habit)` (FR-18.3) - `sendEarlyWarning(studentId, warning)` (FR-18.4) - `sendStreakMilestone(studentId, habit)` (FR-18.6)

IStudyGroup — Study Group Service — Owner: Muhammed Hany Hamdy

IStudyGroup — high level: - Manage Study Groups and Accountability (FR-19)

IStudyGroup — low level: Manage Study Groups and Accountability: - `createStudyGroup(studentId, group)` (FR-19.1) - `joinStudyGroup(studentId, groupId)` (FR-19.2) - `shareGoal(groupId, goalId)` (FR-19.3) - `trackGroupProgress(groupId)` (FR-19.4) - `findAccountabilityPartner(studentId)` (FR-19.5) - `connectAccountabilityPartner(studentId, partnerId)` (FR-19.6) - `shareProgressUpdate(groupId)` (FR-19.7) - `sendSupportMessage(groupId, message)` (FR-19.8)

IAdministration — Administration Service — Owner: Muhammed Hany Hamdy

IAdministration — high level: - Manage Administration (FR-20)

IAdministration — low level: Manage Administration: - `viewStudentAccount(studentId)` (FR-20.1) - `createStudentAccount(data)` (FR-20.2) - `updateStudentAccount(studentId, data)` (FR-20.3) - `suspendStudentAccount(studentId)` (FR-20.4) - `getUsageStatistics()` (FR-20.5, FR-20.6, FR-20.7) - `investigateAccountIssue(studentId)` (FR-20.8) - `resolveAccountIssue(studentId, resolution)` (FR-20.9) - `getRetentionMetrics()` (FR-20.10)

Required Interfaces (consumed by Aspirra services or external systems)

IAIProvider — External AI Provider

IAIProvider — high level (required): - Provide AI capabilities (supports FR-05, FR-06, FR-09, FR-16)

IAIProvider — low level: - `generate(prompt): AIResponse` (used by: FR-05.1, FR-05.2 — generate goal roadmap; FR-06.1 — generate study plan; FR-09.3 — generate career roadmap; FR-16.1, FR-16.2, FR-16.3 — personalized recommendations, study advice, recovery strategy)

IEmailGateway — External Email Gateway

IEmailGateway — high level (required): - Send emails (supports FR-18)

IEmailGateway — low level: - `send(to, subject, body): void` (used by: FR-18.1 — assignment and exam deadline reminders; FR-18.2 — scheduled task reminders; FR-18.3 — scheduled habit reminders; FR-18.4 — early warning notifications; FR-18.5 — goal milestone reminders; FR-18.6 — habit streak milestone notifications)

IStudentProfile — required by Scheduling, Career, Analytics, Study Group, and Administration services

IStudentProfile — high level (required): - Manage Student Profile (supports FR-07, FR-09, FR-14, FR-19, FR-20)

IStudentProfile — low level: - `getProfile(studentId)` (used by: FR-09.1, FR-09.2 — choose a target career; FR-19.3 — share goals in a study group; FR-20.1 — view student account) - `getAvailability(studentId)` (used by: FR-07.1 — schedule around availability; FR-14.2 — calculate remaining available study time) - `createProfile(studentId, data)` (used by: FR-20.2 — create a student account) - `updateProfile(studentId, data)` (used by: FR-20.3 — update student account details)

IGoal — required by Goal Planner, AI Coach, Dashboard, and Study Group services

IGoal — high level (required): - Manage Academic Goals (supports FR-05, FR-16, FR-17, FR-19)

IGoal — low level: - `getGoal(goalId)` (used by: FR-05.2 — generate a plan for a goal; FR-05.6 — apply an approved plan) - `getGoals(studentId, status)` (used by: FR-16.1 — personalized recommendations; FR-17.2, FR-17.3 — dashboard goal status and completion; FR-19.3 — share goals in a study group)

ICourse — required by Study Planner Service

ICourse — high level (required): - Manage Courses (supports FR-06)

ICourse — low level: - `getCourses(studentId)` (used by: FR-06.1 — generate a study plan from enrolled courses) - `getCourseProgress(courseId)` (used by: FR-06.1 — weigh remaining course work in the plan)

IAssessment — required by Course, Scheduling, and Notification services

IAssessment — high level (required): - Manage Assignments and Exams (supports FR-03, FR-07, FR-18)

IAssessment — low level: - `getCourseAssignments(courseId)` (used by: FR-03.3 — view completed and outstanding assignments) - `getAssessments(studentId)` (used by: FR-07.1 — schedule around assessment deadlines; FR-18.1 — assignment deadline reminders) - `getUpcomingExams(studentId, days)` (used by: FR-18.1 — exam deadline reminders)

IGoalPlanner — required by Scheduling Service

IGoalPlanner — high level (required): - Plan Goals with AI (supports FR-07)

IGoalPlanner — low level: - `getTasks(goalId)` (used by: FR-07.1 — schedule approved goal tasks) - `getDraft(draftId)` (used by: FR-07.1 — read task dependencies before scheduling)

IScheduling — required by Study Planner, Adaptation, Analytics, Dashboard, and Notification services

IScheduling — high level (required): - Enable Smart Scheduling (supports FR-06, FR-08, FR-14, FR-17, FR-18)

IScheduling — low level: - `scheduleTasks(studentId, tasks)` (used by: FR-06.2 — distribute study tasks across available time slots) - `getSchedule(studentId, range)` (used by: FR-08.1 — detect falling behind; FR-14.1, FR-14.3 — calculate remaining workload and completed duration; FR-17.1 — display today’s tasks; FR-18.2 — scheduled task reminders) - `markTaskDone(taskId, completed)` (used by: FR-08.1 — detect falling behind from unfinished tasks)

IAdaptation — required by Dashboard Service

IAdaptation — high level (required): - Enable Adaptive Planning (supports FR-17)

IAdaptation — low level: - `getRecommendations(studentId)` (used by: FR-17.2 — show whether a goal needs attention) - `detectFallingBehind(studentId)` (used by: FR-17.2 — show goal risk status)

ICareer — required by Skill and Dashboard services

ICareer — high level (required): - Manage Career Roadmaps (supports FR-10, FR-17)

ICareer — low level: - `getTargetCareer(studentId)` (used by: FR-10.2 — compare skills against the target career) - `getRequiredSkills(studentId)` (used by: FR-10.2, FR-10.3 — compare skills and identify gaps) - `getRoadmapProgress(studentId)` (used by: FR-17.6 — display career progress)

ISkill — required by Portfolio Service

ISkill — high level (required): - Track Skills (supports FR-11)

ISkill — low level: - `getSkills(studentId)` (used by: FR-11.2 — link projects to demonstrated skills) - `updateSkillLevel(skillId, level)` (used by: FR-11.2 — reflect skills demonstrated by a linked project)

IHabit — required by AI Coach, Dashboard, and Notification services

IHabit — high level (required): - Manage Habits (supports FR-16, FR-17, FR-18)

IHabit — low level: - `getHabits(studentId)` (used by: FR-16.1 — personalized recommendations; FR-18.3 — habit reminders; FR-18.6 — streak milestone notifications) - `getStreaks(studentId)` (used by: FR-16.1 — coaching context; FR-17.4 — display habit streaks) - `completeHabit(habitId, date)` (used by: FR-16.1 — update coaching context after a completion)

IAnalytics — required by Prediction, AI Coach, Dashboard, and Study Group services

IAnalytics — high level (required): - Analyze Study Workload (supports FR-15, FR-16, FR-17, FR-19)

IAnalytics — low level: - `getGoalWorkload(studentId, goalId)` (used by: FR-15.1 — estimate completion likelihood; FR-15.2 — generate an early warning) - `getOverloadedGoals(studentId)` (used by: FR-15.2, FR-15.3 — raise warnings and identify risk tasks) - `getWorkloadSummary(studentId)` (used by: FR-16.2, FR-16.3 — deadline-aware advice and recovery strategy; FR-17.5 — dashboard workload summary; FR-19.4 — track group progress)

IPrediction — required by Notification Service

IPrediction — high level (required): - Predict and Warn Early (supports FR-18)

IPrediction — low level: - `getEarlyWarnings(studentId)` (used by: FR-18.4 — send early warning notifications) - `getCompletionLikelihood(studentId, goalId)` (used by: FR-18.4 — include risk level in the early warning)

Interface Notes

- Every service container exposes one provided interface; other containers consume it only through the required-interface operations listed above.
- Each service owns its data and schema. Interfaces exchange identifiers and DTOs; no service reads another service's database directly.
- Implemented interfaces for Sprint 1 are traceable to code:
  - `src/Services/StudentService/Interfaces/IStudentProfile.cs`
  - `src/Services/AcademicService/Interfaces/IGoal.cs`
  - `src/Services/CourseService/Interfaces/ICourse.cs`
  - `src/Services/AssessmentService/Interfaces/IAssessment.cs`
  - `src/Services/GoalPlannerService/Interfaces/IGoalPlanner.cs`
  - `src/Services/HabitService/Interfaces/IHabit.cs`
- The external required interfaces map to configuration placeholders in `.env.example` (`AI_PROVIDER_API_KEY`) and the Notification Service's email settings (`EMAIL_GATEWAY_API_KEY`).